

Scalable zkSNARKs for Matrix Computations

A Generic Framework for Verifiable Deep Learning

Mingshu Cong¹, Sherman S. M. Chow², S.-M. Yiu¹, and Tsz Hon Yuen³

¹ School of Comput. & Data Sci., The University of Hong Kong, Hong Kong
`{mscong,smyiu}@cs.hku.hk`

² Dept. of Info. Engg., The Chinese University of Hong Kong, Hong Kong
`smchow@ie.cuhk.edu.hk`

³ Faculty of Inf. Technol., Monash University, Melbourne, Australia
`john.tszhonyuen@monash.edu`

Abstract. Sublinear proof sizes have recently become feasible in verifiable machine learning (VML), yet no approach achieves the trio of strictly linear prover time, logarithmic proof size and verification time, and architecture privacy. Hurdles persist because we lack a succinct commitment to the full neural network and a framework for heterogeneous models, leaving verification dependent on architecture knowledge. Existing limits motivate our new approach: a unified proof-composition framework that casts VML as the design of zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs) for matrix computations. Representing neural networks with linear and non-linear layers as a directed acyclic graph of atomic matrix operations enables topology-aware composition without revealing the graph. Modeled this way, we split proving into a reduction layer and a compression layer that attests to the reduction with a proof of proof. At the reduction layer, inspired by reduction of knowledge (Crypto '23), root-node proofs are reduced to leaf-node proofs under an interface standardized for heterogeneous linear and non-linear operations. Next, a recursive zkSNARK compresses the transcript into a single proof while preserving architecture privacy. Complexity-wise, for a matrix expression with M atomic operations on $n \times n$ matrices, the prover runs in $O(Mn^2)$ time while proof size and verification time are $O(\log(Mn))$, outperforming known VML systems. Honed for this framework, we formalize relations directly in matrices or vectors—a more intuitive form for VML than traditional polynomials. Our LiteBullet proof, an inner-product proof built on folding and its connection to sumcheck (Crypto '21), yields a polynomial-free alternative. With these ingredients, we reconcile heterogeneity, zero knowledge, succinctness, and architecture privacy in a single VML system.

Keywords: zkSNARK · Matrix · Zero-Knowledge Machine Learning

1 Introduction

Succinctness in zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs) allows a prover to convince a verifier of the validity of an NP statement with logarithmic-time verification while leaking no extra information [11].

Table 1. Architecture/operation-specific proofs (D -layer NN inference, $n \times n$ matrices)

Protocol	Arch. ^a	Op. ^{a, b}	Prover Time	Proof Size	Verifier Time	Hiding ^a
Zen [36]	MLP/CNN	Conv.	$O(Dn^4 \log n)^c$	$O(1)$	$O(1)$	Arch.&Par.
pvCNN [82]	CNN	Conv.	$O(Dn^4 \log n)^c$	$O(1)$	$O(1)$	Arch.&Par.
vCNN [60]	CNN	Conv.	$O(Dn^2 \log n)$	$O(D)$	$O(D)$	Par.
KMC18 [53]	MLP	Basics	$O(Dn^2 \log n)$	$O(D \log n)$	$O(D \log n)$	Par.
Trustless [49]	MLP	Basics	$O(Dn^2)$	$O(D \log n)$	$O(D \log n)$	Par.
zkCNN [63]	CNN	Conv.	$O(Dn^2)$	$O(D \log^2 n)$	$O(Dn)^d$	Par.
zkLLM [74]	LLM	Softmax	$O(Dn^2)$	$O(D \log n)$	$O(Dn)^d$	Par.
Evalyn	General	General	$O(Dn^2)$	$O(\log(Dn))$	$O(\log(Dn))$	Arch.&Par.

^a Shorthand: Arch.: Architecture; Op.: Operation; Par.: Parameter.

^b Matrix multiplication and ReLU are the basics and are omitted. Conv.: Convolution.

^c Complexities for $n \times n$ matrix multiplication/convolution itself are $O(n^3)/O(n^4)$.

^d zkCNN/zkLLM use Hyrax PCS; verifier time complexity larger than its proof size.

Mainstream zkSNARKs target all NP statements by considering the arithmetic circuit satisfiability (Circ-SAT) problem, *e.g.*, [14,16,23,31,32,33,38,45,48,71,72]. Compiling NP statements into arithmetic circuits can nonetheless be inefficient.

Scaling general zkSNARKs to machine learning models poses challenges. Handling neural networks (NNs), with both linear (*e.g.*, matrix operations) and non-linear (*e.g.*, activation) layers, is specifically difficult due to their scale and structural complexities. Establishing verifiable machine learning (VML) means producing a proof for the statement: “the inference output of an NN f on matrix input $\mathbf{X}$ yields $\mathbf{Y}$.” Routine matrix operations such as multiplication and convolution are inherently superlinear; their massive parameter counts in real-world models further strain Circ-SAT zkSNARKs. Monolithic integration of zkSNARK protocols into a single arithmetic circuit yields prohibitively large circuit sizes with billions of gates even after optimizations (Appendix A.2), exceeding the practical (*e.g.*, $<2^{30}$ -gate) limits of Circ-SAT-based zkSNARKs, motivating specialized zkSNARK designs for VML [1,30,36,49,53,60,63,74,80,82,88]. As a baseline, we expect prover time to scale linearly with Dn^2 , where D is the NN depth (number of layers) and $n \times n$ is the matrix dimension, while proof size and verifier time remain logarithmic. Necessarily, achieving that linear scaling demands linear-prover protocols for superlinear matrix operations.

Moving past monolithic circuits, recent work adopts a layer-by-layer proving strategy to improve scalability. In particular, vCNN [60] uses Groth16 [48] for both convolution and the basic rectified linear unit (ReLU) activation, while Keuffner *et al.* (KMC18) [53] and “Trustless DNN” [49] use the Goldwasser–Kalai–Rothblum (GKR) protocol for multiplication, with ReLU handled by Groth16 and Halo2 [87], respectively. Notably, these approaches achieve $O(n^2 \log n)$ per-layer prover time, where the logarithmic factor arises from the use of fast Fourier transforms (FFT) in Groth16. Going further, zkCNN [63] and zkLLM [74] use modified GKR protocols to cover not only multiplication/convolution but also

non-linear activations, including ReLU and Softmax, achieving strictly linear prover time. Still, their per-layer proof sizes and verifier times are $O(\log^2 n)$ and $O(\log n)$, respectively. Hence, despite these gains, all these layer-by-layer approaches are not succinct overall (Table 1). Ultimately, with D per-layer proofs, the aggregate size and verification time scale at least linearly with the depth D .

Considering verification, these layer-by-layer methods require explicit knowledge of the NN structure. Sans this information, the verifier cannot identify where to check each operation. Monolithic-circuit approaches, by contrast, ensure architecture privacy by committing to the circuit without leaking its details.

Open Problem. We are left in an unsatisfactory state of affairs: monolithic circuit-based VML methods retain all the advantages of zkSNARKs: succinct proof size, fast verifier time, and full protection of the NN architecture, but suffer from prohibitively slow prover time (either superlinear or linear with large constants). Conversely, specialized VML protocols for specific NNs achieve strictly linear prover time yet sacrifice succinctness, the defining property of zkSNARKs, and fail to hide the NN architecture. Recently, zkAudit [79] (extending prior work [49] from inference to training) highlights hiding the NN architecture—not only the parameters—as “an exciting future research.” Finally, these tailor-made solutions cannot adapt to the rapidly evolving NN landscape. This dilemma leads to a central question: *Can we design a generic zkSNARK system that (i) supports all NN architectures with architecture privacy, while achieving (ii) strictly linear prover time and (iii) logarithmic proof size and verifier time?*

Challenges. Each requirement is easy in isolation, but achieving all three together is challenging. Architecture privacy demands a proof system that applies uniformly across heterogeneous architectures and operations. Proving NNs generically [36,80,82] is significantly more costly than specialized ones. Such a system must unify linear and non-linear operations while handling structural complexities, such as layers feeding into multiple subsequent layers and the interplay between linear and non-linear components⁴. With a hidden architecture, the verifier is blind to operational dependencies and must rely on a commitment to the network’s topology rather than its explicit structure. Meanwhile, logarithmic verification requires a succinct commitment binding all local checks to avoid per-layer overhead. Such succinct commitments remain an open challenge. Moreover, achieving strictly linear prover time in VML requires specialized atomic proofs for superlinear matrix operations (*e.g.*, matrix multiplication and convolution). However, recursively applying these specialized protocols causes proof size and verifier time to grow linearly with the number of operations.

Our Solution. We propose an intuitive design without sacrificing efficiency. Proofs for atomic matrix operations serve as “Lego blocks”—modular units with a unified interface—so heterogeneous proofs compose cleanly. Shared values across these blocks are limited to a few field elements. These atomic proofs are composed in topological order to capture the dependency structure of NN

⁴ For example, GForce [66] formulates a layer that fuses dequantization with requantization across non-linear ReLU or MaxPool while preserving equivalence.

operations, with each proof corresponding to a specific operation. The verifier sees only a commitment to the architecture, so the topology remains hidden. To achieve succinctness, we compress the entire composite proof with a proof-of-proof (PoP) zkSNARK. This maintains strictly linear prover time, as the cost is dominated by the linear-time atomic proofs. The PoP resolves the verifier’s overhead by collapsing many per-layer checks into a single, succinct verification.

1.1 Our Contributions

Unified DAG-Based VML Framework. We conceptualize NNs as matrix expressions represented as a directed acyclic graph (DAG) whose nodes are atomic matrix operations and whose edges are data dependencies. This abstraction generalizes across NN architectures, covering multilayer perceptron (MLP), convolutional neural networks (CNNs), and transformer architectures such as large language models (LLMs) explored in prior VML work. It also extends to unexplored architectures with *multi-parent* edges such as recurrent networks (RNNs) and residual networks (ResNets). The DAG view separates local arithmetic correctness from global linkage and architecture privacy. This formalization underlies the proofs that follow. Our model-agnostic approach contrasts with both prior VML systems and the cryptography literature on the secure NN evaluation [67], where advances often rely on function-specific protocols (*e.g.*, **Softmax** and **GELU** [51]) or on optimizations to homomorphic encryption schemes.

Proof Composition for VML. If each node admits a linear-time proof and nodes share only a logarithmic number of field elements, then composing [43] along the DAG yields linear prover time, but proof size and verification remain $O(M \log n)$, which we reduce to $O(\log(Mn))$ with a dual-layer framework:

- *Proof reduction layer:* Recursively reduce root proofs to leaf-node proofs, aligning with the reduction of knowledge (RoK) framework [54].
- *Proof-of-proof layer:* Compress the non-succinct, non-zero-knowledge reduction proof into a succinct, zero-knowledge proof using an outer zkSNARK, which hides the topology by proving only a topology commitment and a single digest that links all challenges.

With this dual-layer composition, we achieve “the best of both worlds” for generic matrix expressions: strictly linear prover time, logarithmic proof size and verifier time, zero knowledge, and architecture privacy. To unify heterogeneous matrix operations, we introduce the standardized **Evalyn** interface for:

- *Type-1 operations:* Directly compatible with the proof reduction framework.
- *Type-2 operations:* Transformable into Type-1 operations.

This formulation homogenizes atomic operations by reducing them to a unified matrix-expression framework, thereby facilitating a uniform proof strategy.

Foundational Contribution: We formalize a unified dual-layer proof composition framework for generating zkSNARKs for generic matrix expressions.

Strictly Linear Prover Time via Proof Reduction. Let M be the number of atomic matrix operations and let each matrix have dimension at most $n \times n$. For depth- D NNs, $M = O(D)$. By recursively generating atomic proofs through the unified interface and composing along the DAG, we achieve $O(Mn^2)$ prover time, previously attainable only for specific NNs, now in a model-agnostic setting.

Key Advancement 1: We achieve $O(Mn^2)$ prover time for generic matrix expressions with M operations on $n \times n$ matrices.

Succinctness and Architecture Privacy via Proof of Proof. Proof reduction alone yields a proof of size $O(M \log n)$ and requires explicit topology for verification. We add an outer PoP layer, as a zkSNARK over the verification circuit of a prior proof, to ensure succinctness and architecture privacy. Since proof reduction involves only sublinear field and hashing operations, PoP adds an acceptable additional prover overhead. Performance is improved further using batch proofs for matrix commitment openings of the PoP circuit. To our knowledge, this is the first approach to achieve succinctness, zero knowledge covering the architecture, and strictly linear prover time simultaneously.

Key Advancement 2: PoP restores logarithmic proof size and verifier time.

Key Advancement 3: We enable efficient commitment for NN architectures.

Polynomial-Free LiteBullet Proof: Folding-Based Inner-Product Proof.

Polynomials play a central role in zkSNARK research, with many systems relying on the sumcheck protocol [64] over multilinear extension (MLE) polynomials and polynomial commitment schemes (PCS), *e.g.*, [14,16,45,50,59,78,85]. In VML scenarios, reliance on polynomials introduces unnecessary barriers for practitioners translating between matrix and polynomial representations.

Motivated by the link between sumcheck and folding techniques [17], we propose LiteBullet, a folding-based inner product proof that eliminates polynomial dependencies. LiteBullet operates directly on vectors and matrices, avoiding polynomial conversion while matching the concrete efficiency of sumcheck.

Key Advancement 4: We propose a folding-based inner product proof of independent interest, enabling a polynomial-free zkSNARK design paradigm.

1.2 Related Work

Proof composition [43] facilitates modular protocol design by formally showing when zero knowledge and soundness are preserved under sequential and parallel composition. In zkSNARKs, IVC [77] and PCD [34,22] are canonical frameworks that maintain a proof as the computation advances, via recursive verification across steps in IVC, or via accumulation along a message graph in PCD. Representative systems include Fractal [33], Halo [20], Nova [57], SuperNova [55], and HyperNova [56]. IVC and PCD schemes support two styles of composition.

- *Proof of proof.* Recursive composition [10,12,33] proves the current step and a proof that the previous step was proven correctly.
- *Proof accumulation.* Each step updates an accumulator using an accumulation scheme [20,22,25], and the verifier checks the accumulator at the end.

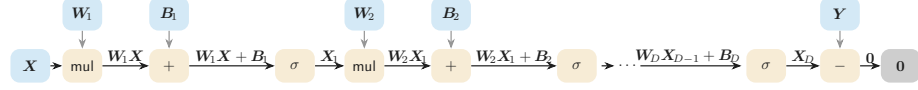


Fig. 1. NN inference represented as a DAG of matrix operations (gray, blue, and yellow rectangles denote the root, leaf, and intermediate nodes, respectively.)

RoK [54] reduces a proof of knowledge for one relation to a simpler one. This perspective also applies to proof reduction for composite functions [53].

LegoSNARK [26] develops modular composition using matrix commitments to decompose computations into smaller parts that are proven separately. However, composing LegoSNARK with PoP incurs high overhead due to many group-element commitments that must be carried and verified across steps.

Our method is inspired by IVC and PCD but departs in three ways. (i) We apply PoP only as an outer compression of the reduction transcript rather than at every intermediate step. (ii) We eliminate intermediate-state commitments between blocks by sharing only a logarithmic number of field elements across the interface. (iii) We aggregate in the dependency order of the DAG rather than in a purely temporal order. See Appendix A for further comparisons and related work on general-purpose zkSNARKs, VML, folding techniques, and PCS.

2 Technical Overview

2.1 Matrix Expression as Abstraction for Neural Networks

We model VML as a zkSNARK for DAG-structured matrix expressions built from atomic operations. Modeling the forward pass as a DAG of atomic matrix operations provides a single compilation interface from NN layers to field-level constraints, and it fixes the target zkSNARK relation in Eq. (1).

A feedforward NN with D fully connected layers (Appendix E.5) is defined by $\mathbf{X}_0 := \mathbf{X}$, $\mathbf{X}_i = \sigma(\mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i)$, $i \in \{1, \dots, D\}$, where $\{\mathbf{W}_i\}$ are the weights, $\{\mathbf{B}_i\}$ are the biases, and σ is an element-wise activation. The output is $\mathbf{Y} := \mathbf{X}_D$. Each layer decomposes into three atomic operations: multiplication mul , addition $+$, and a non-linear activation σ . The computation forms a DAG as in Fig. 1, with leaf nodes $\mathbf{X}, \mathbf{W}_1, \mathbf{B}_1, \dots, \mathbf{W}_D, \mathbf{B}_D, \mathbf{Y}$. As a computation graph, the DAG encodes the composite matrix map g , where each node is an atomic operation and edges carry data dependencies. Correctness is expressed by the residual matrix function $f = g(\mathbf{X}, \mathbf{W}_1, \mathbf{B}_1, \dots, \mathbf{W}_D, \mathbf{B}_D) - \mathbf{Y}$.

A VML prover proves knowledge of matrices $(\mathbf{a}_1, \dots, \mathbf{a}_k) := (\mathbf{X}, \mathbf{W}_1, \mathbf{B}_1, \dots, \mathbf{W}_D, \mathbf{B}_D, \mathbf{Y})$ that satisfy $f(\cdot) = \mathbf{0}$, where $k = 2D + 2$. The target relation is:

$$\mathcal{R}_{\text{main}} := \left\{ \left(\begin{array}{l} k, n \in \mathbb{N}, \\ f \in \mathcal{E}, C_a \in \mathbb{G} \end{array} \right) : \left(\begin{array}{l} \mathbf{a}_1, \dots, \mathbf{a}_k \\ \in \mathbb{F}^{n \times n} \end{array} \right) \left| \left(\begin{array}{l} f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0} \wedge \\ C_a = \text{Comm}(\mathbf{a}_1 \| \dots \| \mathbf{a}_k) \end{array} \right) \right. \right\}. \quad (1)$$

Here, $\mathcal{E}$ is the set of well-typed matrix expressions over prime field $\mathbb{F}$ built from mul , $+$, and σ . We fix a uniform bound n , attach the shape metadata $(n_{\text{rows}}, n_{\text{cols}})$

to each matrix, and apply constraints and commitments only to entries within that shape, while treating the remainder as padded zeros.⁵ All matrices are bound to a single commitment C_a using a matrix commitment scheme **Comm** with length-tagged encoding⁶ over a prime-order elliptic-curve group $\mathbb{G}$.

This paper is dedicated to designing an efficient zkSNARK for $\mathcal{R}_{\text{main}}$.

2.2 Type-2-to-Type-1 Transformation

Among matrix operations, an operation is called Type-1 if it admits a strictly-linear-prover reduction proof (in the number of matrix entries). Type-1 operations include addition, matrix multiplication, transpose, Hadamard (element-wise) product, Frobenius inner product, and Kronecker product. An expression built only from Type-1 operations is Type-1. Operations outside this class are Type-2, and we compile each Type-2 node into auxiliary variables and equalities that use only Type-1 operators, yielding a single linear-prover-cost framework.

For illustration, consider convolution $\mathbf{c} = \text{Conv}(\mathbf{a}, \mathbf{b})$ with $\mathbf{a}, \mathbf{b} \in \mathbb{F}^{n \times n}$ and $\mathbf{c} \in \mathbb{F}^{(2n-1) \times (2n-1)}$. Let $\vec{\mathbf{l}} = (1, y, \dots, y^{2n-2})$ and $\vec{\mathbf{r}} = (1, y', \dots, y'^{2n-2})$ for random challenges $y, y' \in \mathbb{F}^*$, and let $(\cdot)_{[0:n]}$ take the first n entries of a vector. The generating-function identity gives $\vec{\mathbf{l}}^\top \mathbf{c} \vec{\mathbf{r}} = (\vec{\mathbf{l}}_{[0:n]}^\top \mathbf{a} \vec{\mathbf{r}}_{[0:n]}) \cdot (\vec{\mathbf{l}}_{[0:n]}^\top \mathbf{b} \vec{\mathbf{r}}_{[0:n]})$, so verifying convolution reduces to two matrix-vector multiplications and one field multiplication, all Type-1, with soundness error at most $O(n/|\mathbb{F}|)$ by Schwartz–Zippel for a bivariate equality. This pattern also covers depthwise and separable convolutions by factoring $\mathbf{a}$ or $\mathbf{b}$ into structured blocks.

2.3 Polynomial-Free Inner-Product Proof to Replace Sumcheck

Conventional zkSNARK systems map matrix equations to polynomial equations via MLEs, the unique multilinear polynomials that match the matrix entries on Boolean inputs.⁷ The sumcheck protocol [64] on these MLEs reduces the verification of a matrix equation to checking polynomial evaluations. These evaluations are subsequently proven using a vector–matrix–vector product (projection) argument provided by many PCS [61], *e.g.*, Hyrax [78], Dory [59], and Orion [85]. MLEs and this pipeline are widely used in proof systems, *e.g.*, [32,44,45,71,76].

To keep verification in the matrix domain, the MLE–sumcheck detour is avoided by introducing **LiteBullet**, a folding-based inner-product proof. Boyle *et al.* [17, Appendix A] reveal that sumcheck mirrors folding techniques.

⁵ Shape-aware verification is required when padding to keep data zeros distinct from padding zeros. One can commit to a shape-metadata tag $(n_{\text{rows}}, n_{\text{cols}})$ and adapt the constraints so that checks apply only to the tagged region, while proving that entries outside are zero. The PoP layer should hide these tags and any architecture information while proving the correctness for the unpadded matrices.

⁶ Metadata includes $(n_{\text{rows}}, n_{\text{cols}})$ for each matrix. We apply a canonical serialization of its entries and domain separation labels for inputs, parameters, and outputs.

⁷ Consider the MLE of the vector $(2, 3, 4, 5)$ on $(x_1, x_2) \in \{0, 1\}^2$, we have $\psi(x_1, x_2) = 2(1 - x_1)(1 - x_2) + 3(1 - x_1)x_2 + 4x_1(1 - x_2) + 5x_1x_2$.

Guided by this insight, we present the folding identity that derives LiteBullet. It proves an inner product claim via a single bilinear identity that folds the vectors in half each round. Specifically, $c = \vec{a} \bullet \vec{b}$ for scalar $c \in \mathbb{F}$ and vectors $\vec{a}, \vec{b} \in \mathbb{F}^m$ is proven by sampling a random challenge $x \in \mathbb{F}^*$ and checking:

$$(\vec{a}_{\text{LH}} + x^{-1}\vec{a}_{\text{RH}}) \bullet (\vec{b}_{\text{LH}} + x\vec{b}_{\text{RH}}) = c + x(\vec{a}_{\text{LH}} \bullet \vec{b}_{\text{RH}}) + x^{-1}(\vec{a}_{\text{RH}} \bullet \vec{b}_{\text{LH}}).$$

In each folding step, $\vec{a}$ is split into left and right halves $\vec{a}_{\text{LH}}$ and $\vec{a}_{\text{RH}}$, and likewise for $\vec{b}_{\text{LH}}$ and $\vec{b}_{\text{RH}}$. The recursion halves the length over $\log_2 m$ rounds, until both sides are scalars, similar to the technique in Bulletproofs [21]. This similarity motivates the name LiteBullet proof.

Distinct from Bulletproofs, LiteBullet is not an argument of knowledge (AoK). Vectors $\vec{a}$ and $\vec{b}$ are public, and verification involves computing the projections (inner products) $\vec{a} \bullet \vec{\xi}^{-1}$ and $\vec{b} \bullet \vec{\xi}$ of these public vectors on tensor-structured random vectors $\vec{\xi}^{-1}$ and $\vec{\xi}$ derived from $\log_2 m$ challenges $x_{(1)}, \dots, x_{(\log_2 m)}$ and the Kronecker (tensor) product operation $\otimes$: $\vec{\xi} = (1, x_{(1)}) \otimes \dots \otimes (1, x_{(\log_2 m)}) \in \mathbb{F}^m$. We refer to $\vec{\xi} = \text{ts}(\mathbf{x})$ as the *tensor span* of $\mathbf{x} := (x_{(1)}, \dots, x_{(\log_2 m)})$. The inner products $\vec{a} \bullet \vec{\xi}^{-1}$ and $\vec{b} \bullet \vec{\xi}$ are verified via additional succinct proof.

Replacing sumcheck with LiteBullet enables polynomial-free proof systems.

2.4 Atomic Type-1 Operation Proof with Unified Interface

Matrix expressions are built from atomic matrix operations. To support all DAG architectures, we introduce a unified Evalyn interface for each Type-1 operation $\otimes$, with four methods ($\otimes.\text{Prove}$, $\otimes.\text{Eval}$, $\otimes.\text{Link}$, $\otimes.\text{Verify}$). Given matrices $\mathbf{a}, \mathbf{b}$, a scalar v , and public vectors $\vec{l}$ and $\vec{r}$, $\otimes.\text{Prove}$ produces a proof π of the scalar projection $v = \vec{l}^\top (\mathbf{a} \otimes \mathbf{b}) \vec{r}$. $\otimes.\text{Verify}$ checks π , $\otimes.\text{Eval}$ extracts child projection scalars v_L and v_R of the left and right inputs $\mathbf{a}$ and $\mathbf{b}$ from π , and $\otimes.\text{Link}$ computes $(\vec{l}_L, \vec{r}_L)$ and $(\vec{l}_R, \vec{r}_R)$ using $(\vec{l}, \vec{r})$ and verifier challenges.

An Evalyn proof reduces the parent projection v to simpler projections $v_L = \vec{l}_L^\top \mathbf{a} \vec{r}_L$ and $v_R = \vec{l}_R^\top \mathbf{b} \vec{r}_R$. Deterministic $\otimes.\text{Eval}$ and $\otimes.\text{Link}$ (*cf.*, Definition 5) serve as the Reduce routine in RoK [54] (Appendix B.2) by deriving the child statements from the parent statement and transcript-defined challenges.

Proofs for Type-1 operations are wrapped by the Evalyn interface as follows:

$$\left\{ \begin{array}{ll} \mathcal{P} \text{ produces a } \otimes\text{-specific proof } \pi \text{ for } v = \vec{l}^\top (\mathbf{a} \otimes \mathbf{b}) \vec{r} & (\text{Prove}) \\ \mathcal{V} \text{ checks } \pi \text{ and:} & (\text{Verify}) \\ \{v_L = \vec{l}_L^\top \mathbf{a} \vec{r}_L \wedge v_R = \vec{l}_R^\top \mathbf{b} \vec{r}_R\} \text{ (recursively proven), where:} & \\ v_L \text{ and } v_R \text{ are the final elements of } \pi, \text{ and} & (\text{Eval}) \\ \vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R \text{ are derived from } \vec{l}, \vec{r}, \text{ and the randomness from } \mathcal{V} & (\text{Link}) \end{array} \right\}.$$

Eval reads the transcript to extract child projection scalars, while Link maps parent projecting vectors to child projecting vectors using random challenges.

2.5 Proof Reduction Through DAG Traversal

For a *Type-1 expression* f composed solely of Type-1 operations, proofs are generated by a pre-order traversal from the root to the leaves of the DAG representing f , where nodes are operations and edges denote data dependencies. To prove $f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}$, we apply the Schwartz–Zippel lemma (Appendix B.3) and check that $\vec{\mathbf{l}}^\top f(\mathbf{a}_1, \dots, \mathbf{a}_k) \vec{\mathbf{r}} = 0$ for random tensor-structured $\vec{\mathbf{l}}$ and $\vec{\mathbf{r}}$.

Starting from this root scalar projection, the parties recursively execute the $\otimes$.(Prove, Eval, Link, Verify) algorithms at each node in the DAG. In the pre-order traversal, the prover calls $\otimes$.Prove at each node to emit a proof π . The verifier runs $\otimes$.Verify on π and aborts immediately if its output is **false**. Otherwise, the verifier uses $\otimes$.Eval to derive child scalars v_L and v_R from π , and applies $\otimes$.Link to compute projecting vectors $(\vec{\mathbf{l}}_L, \vec{\mathbf{r}}_L)$ and $(\vec{\mathbf{l}}_R, \vec{\mathbf{r}}_R)$ for the left and right child, respectively. These vectors and scalars are passed from parent to child nodes. Upon completing the traversal, each leaf $i \in [k]$ receives a scalar v_i and projecting vectors $\vec{\mathbf{l}}_i$ and $\vec{\mathbf{r}}_i$, which are then checked via matrix-commitment openings.

2.6 Batch Opening Leaf Node Projections for Succinctness

To keep the proof succinct, the k leaf node projections $v_i = \vec{\mathbf{l}}_i^\top \mathbf{a}_i \vec{\mathbf{r}}_i, i \in [k]$ are batch-opened. Otherwise, the proof would require at least $O(k)$ group elements.

This batch opening uses an extra Fiat–Shamir random challenge $\theta \in \mathbb{F}^*$, and one additional LiteBullet proof for the inner product relation:

$$\begin{aligned} \sum_{i=1}^k \theta^{i-1} \cdot v_i &= \sum_{i=1}^k \theta^{i-1} \cdot (\vec{\mathbf{l}}_i^\top \mathbf{a}_i \vec{\mathbf{r}}_i) \\ &= \text{vec}(\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k) \bullet \text{vec}(\vec{\mathbf{l}}_1 \otimes \vec{\mathbf{r}}_1^\top \parallel \theta \cdot \vec{\mathbf{l}}_2 \otimes \vec{\mathbf{r}}_2^\top \parallel \dots \parallel \theta^{k-1} \cdot \vec{\mathbf{l}}_k \otimes \vec{\mathbf{r}}_k^\top). \end{aligned}$$

We write $\text{vec}(\cdot)$ for the vectorization map, and we use the standard identity $\vec{\mathbf{l}}^\top \mathbf{a} \vec{\mathbf{r}} = \text{vec}(\mathbf{a}) \bullet \text{vec}(\vec{\mathbf{l}} \otimes \vec{\mathbf{r}}^\top)$. The relation is proved by checking an opening such that the commitment encodes the concatenated matrix $(\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k)$ and one LiteBullet inner product proof whose size is $O(\log(kn))$ for matrices $\mathbf{a}_i \in \mathbb{F}^{n \times n}$.

This LiteBullet proof combines k projections of $n \times n$ matrices into a single projection of the larger concatenated matrix $(\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k)$, with its matrix commitment opening, at proof size $O(\log(kn))$.

2.7 Proof of Proof for Succinctness and Zero Knowledge

During proof reduction, each of the M nodes in the DAG emits a proof, so the total proof size and verification time grow linearly with M , which breaks the target of logarithmic succinctness. Also, LiteBullet proofs are not zero knowledge, and the proof-reduction verifier depends on architecture information (topology, operation types, and matrix-shape metadata), compromising NN privacy.

We add a proof of proof (PoP) using a zkSNARK to recover these properties. The PoP proves knowledge of M valid proofs $\{\pi_i\}_{i \in [M]}$ such that $\mathcal{V}_i(\pi_i) = \text{true}$

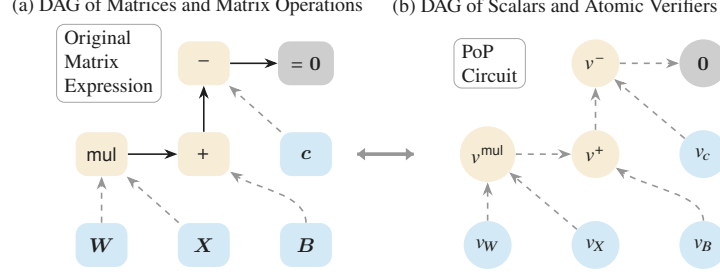


Fig. 2. DAG for the matrix expression $c = WX + B$ and its PoP circuit: By projecting each matrix into a scalar, the topology of the matrix DAG is captured by a scalar DAG, which corresponds to the PoP circuit. (Legend: **root**, **leaf**, and **operation**)

for each atomic verifier $\mathcal{V}_i$. The M proofs together contain $O(M \log n)$ field elements and hash outputs, so a PoP prover time of $O(M \log n)$ is small compared with the $O(Mn^2)$ time of proof reduction for matrices of size $n \times n$.

Our reduction maps each matrix to a scalar, and the atomic verifier circuits link these scalars according to the original DAG topology, which yields the PoP circuit (Fig. 2). Committing to the PoP circuit commits to the DAG. As the PoP circuit is much smaller than the monolithic circuit for the full matrix expression, it enables more efficient proving using Circ-SAT-based zkSNARKs.

2.8 Commit and Prove

The proof reduction produces a sublinear proof for language $\mathcal{L}_{\text{main}}$ below.

$$\{(k, n, f, \mathbf{a}_1, \dots, \mathbf{a}_k) \mid k, n \in \mathbb{N}, f \in \mathcal{E}, \mathbf{a}_1, \dots, \mathbf{a}_k \in \mathbb{F}^{n \times n}, f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}\}. \quad (2)$$

However, this proof is not an AoK for the relation $\mathcal{R}_{\text{main}}$ in Eq. (1), because it treats $\mathbf{a}_1, \dots, \mathbf{a}_k$ as public inputs known to the verifier.

Prior work shows that the *commit-and-prove* technique, combining a proof system with a commitment scheme, yields an AoK [89,90]. The knowledge soundness error of this approach stems from the soundness error of the proof system, the knowledge error of the proof of knowledge for a commitment opening, and the binding advantage of the commitment scheme. The prover initially commits to the leaf nodes of the DAG, then performs proof reduction, and finally opens the leaf commitments.

2.9 Putting Everything Together

The Evalyn framework in Fig. 3 comprises four subalgorithms executed in order.

- *Type-2 commitment* (**Comm**) converts Type-2 operations into Type-1 expressions by adding new leaf nodes to the DAG, commits to these leaves, and outputs a commitment C_{type2} sent to the verifier.

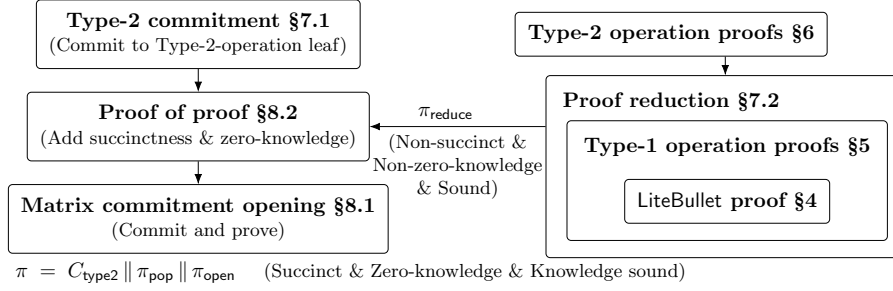


Fig. 3. Architecture of Evalyn

- *Proof reduction* (Reduce) traverses the DAG in a topological order that obeys dependencies and reduces root projections to leaf projections.
- *Proof of proof* (PoP) runs a zkSNARK that proves knowledge of a valid reduction proof π_{reduce} and outputs π_{pop} , so the verifier needs only π_{pop} .
- *Matrix commitment opening* (Open) opens the committed leaves at the positions required by verification and outputs proof π_{open} for those projections.

The verifier accepts if and only if the PoP proof accepts with C_{type2} bound as a public input and the requested openings are valid.

Fig. 4 illustrates our pipeline and contrasts it with IVC/PCD. Our reduction runs as a backward pass from roots to leaves—a topological order from outputs to inputs—then applies a single-shot PoP that subsumes π_{reduce} , and finally proves the required openings of committed leaves. In contrast, IVC/PCD composes proofs stepwise along the forward execution, which introduces recursive overhead and per-step verification interfaces that our single-shot design avoids.

3 Preliminaries

3.1 Notations

Starting with matrices over an order- p Galois field $\mathbb{F}$ with $\mathbb{F}^*$ as its multiplicative group, matrices are denoted by bold letters, *e.g.*, $\mathbf{a}, \mathbf{b}, \mathbf{c}$, while vectors are indicated by arrows, *e.g.*, row and column vectors $\vec{l}^\top$ and $\vec{r}$. Scalar field elements are written in lowercase, *e.g.*, a, b, c . Matrix or vector entries are indexed by subscripts, *e.g.*, $\mathbf{a} = \{a_{ij}\}$ and $\vec{l} = \{l_i\}$. Canonical index set $[n]$ represents the set $\{1, \dots, n\}$. $\phi(\mathbf{x})$ and $\psi(\mathbf{x})$ represent multilinear polynomials, with μ variables and evaluation values v . M is the number of matrix operations. Matrix/vector dimensions are denoted by m, n , indices by i, j, s, τ , and random challenges by x, y, z . $\mathcal{E}$ is the set of matrix expressions with $\otimes$ as a matrix operation, covering:

- Addition ($\mathbf{c} = \mathbf{a} + \mathbf{b}$), Multiplication ($\mathbf{c} = \mathbf{a}\mathbf{b}$), Concatenation ($\mathbf{c} = (\mathbf{a} \parallel \mathbf{b})$),
- Convolution ($\mathbf{c} = \text{Conv}(\mathbf{a}, \mathbf{b})$), Frobenius inner product ($c = \mathbf{a} \bullet \mathbf{b}$),
- Hadamard product ($\mathbf{c} = \mathbf{a} \circ \mathbf{b}$) and Kronecker product ($\mathbf{c} = \mathbf{a} \otimes \mathbf{b}$).

Similarly, $\mathbf{c} = \otimes \mathbf{a}$ can represent:

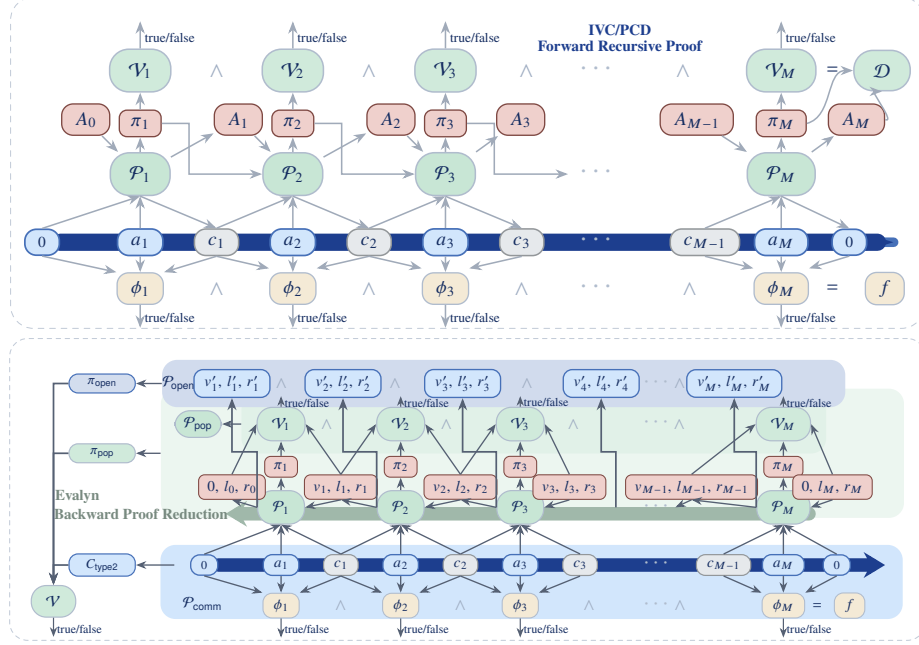


Fig. 4. Comparing IVC/PCD and Evalyn for a matrix expression $\mathbf{0} = f(\mathbf{a}_1, \dots, \mathbf{a}_M)$, decomposed as $\{c_i = \phi_i(c_{i-1}, \mathbf{a}_i)\}_{i \in [M]}$ with $c_0 = c_M = \mathbf{0}$: IVC/PCD uses a forward recursive proof generation, while updating the accumulator A_i ; decider $\mathcal{D}$ verifies M predicates using accumulator state A_M along with the final proof π_M . Evalyn uses backward proof reduction: each node produces a proof π_i and passes a scalar projection v_i of intermediate matrices c_i ; verification of M predicates checks all π_i from all provers and the matrix projections $\{v_i\}_{i \in [M]}$ of matrices $\mathbf{a}_i$ on vectors $\vec{l}_i$ and $\vec{r}_i$. $\mathcal{P}_{\text{pop}}$ produces a zkSNARK for $\{V_i\}_{i \in [M]}$. (See Appendix A.1 for more details.)

- Scalar multiplication ($\mathbf{c} = b \cdot \mathbf{a}$), Transpose ($\mathbf{c} = \mathbf{a}^\top$), Inversion ($\mathbf{c} = \mathbf{a}^{-1}$),
- Element-wise inversion ($\mathbf{c} = \frac{1}{\mathbf{a}}$) and Non-linear function ($\mathbf{c} = \sigma(\mathbf{a})$).

Their definitions are available in standard linear algebra textbooks. Without loss of generality, we focus on $n \times n$ square matrices with zero-padding.

Definition 1 (Matrix Expression). A matrix expression $f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}$ is a DAG whose leaves are the input matrices $\mathbf{a}_1, \dots, \mathbf{a}_k$, the root is the output matrix $\mathbf{0}$, and other nodes represent matrix operations as below.

- **Type-1 operations:** Addition, subtraction, scalar multiplication, transpose, matrix multiplication, (Frobenius) inner product, Hadamard (element-wise) product, element-wise inversion, Kronecker (tensor) product, concatenation, and truncation. A matrix expression composed solely of Type-1 operations is referred to as a Type-1 expression.
- **Type-2 operations:** Inversion, non-linear functions, convolution, max pooling, and other NN operations. They are transformed into Type-1 expressions.

Definition 2 (Tensor Span). *Given μ random values $\mathbf{x} \in \mathbb{F}^\mu$, their tensor span is the 2^μ -vector $\text{ts}(\mathbf{x}) \in \mathbb{F}^{2^\mu}$ below, where $\otimes$ is the Kronecker product:*

$$\text{ts}(\mathbf{x}) := (1, x_1) \otimes (1, x_2) \otimes \cdots \otimes (1, x_\mu) \in \mathbb{F}^{2^\mu}.$$

3.2 Proof, Argument, and Argument of Knowledge

A proof/argument system consists of three probabilistic polynomial-time (PPT) algorithms: $(\text{Setup}, \mathcal{P}, \mathcal{V})$. **Setup** produces public parameters pp . Given pp and a statement u , prover $\mathcal{P}$ with witness w outputs a transcript π to convince the verifier that u is true. In this paper, when we refer to proof/argument, it is related to a language $\mathcal{L}$ involving a statement u as $\mathcal{L} = \{u : \text{Constraint}(u) = \text{true}\}$. For an argument of knowledge, it pertains to a relation $\mathcal{R}$ involving an input u and a witness (knowledge) w as $\mathcal{R} = \{u : w \mid \text{Constraint}(u, w) = \text{true}\}$. For an input u alone, we write $u \in \mathcal{R}$ to denote that there exists a witness w such that $(u, w) \in \mathcal{R}$. A proof, an argument, and an AoK satisfy soundness, computational soundness, and knowledge soundness, respectively. They all possess perfect completeness. Round-by-round soundness (Appendix B.6) is stronger than soundness, and special soundness (Appendix B.7) is stronger than knowledge soundness. Detailed definitions are provided in Appendix B.1.

Our scheme instantiates RoK. Its definition, sequential/parallel composition, and applications to DAGs are detailed in Appendix B.2. Other zkSNARK-related definitions, including perfect zero knowledge and succinctness, are in Appendix B.4. Adaptive Fiat-Shamir transformation is recalled in Appendix B.8.

3.3 Matrix Commitment Scheme

A matrix commitment scheme is a specific type of general commitment scheme (with binding and hiding, as defined in Appendix B.5) that mirrors the PCS for polynomials. It directly provides a proof about the vector-matrix-vector product (projection) of a committed matrix. Since modern PCS verifies polynomial evaluations through matrix projections, *e.g.*, [14, 16, 45, 59, 78, 85], they all function as matrix commitment schemes.

Definition 3 (Matrix Commitment). *A matrix commitment scheme for $m \times n$ matrices is a tuple of algorithms $(\text{Setup}, \text{Comm}, \text{Prove}_{\text{open}}, \text{Verify}_{\text{open}})$:*

- $\text{Setup}(1^\lambda, m, n) \rightarrow \text{pp}$: *Given a unary string 1^λ , output public parameter pp .*
- $\text{Comm}(\text{pp}, \mathbf{a}; \tilde{\mathbf{a}}) \rightarrow C$: *Take as input a matrix $\mathbf{a}$ over a field $\mathbb{F}$ output a commitment C using randomness $\tilde{\mathbf{a}}$. (For brevity, pp or $\tilde{\mathbf{a}}$ may be omitted.)*
- $\text{Prove}_{\text{open}}(\text{pp}, C, C_v, \vec{\mathbf{l}}, \vec{\mathbf{r}}; \mathbf{a}, \tilde{\mathbf{a}}, \tilde{\mathbf{v}}) \rightarrow \pi_{\text{open}}$: *For our usage, we directly define an algorithm that prove about two commitments C and C_v that should satisfy the relation below, where the corresponding openings $\tilde{\mathbf{a}}$ and $\tilde{\mathbf{v}}$ are made implicit:*

$$\mathcal{R}_{\text{open}} = \{(\text{pp}, C, C_v, \vec{\mathbf{l}}, \vec{\mathbf{r}}) : \mathbf{a} \mid C_v = \text{Comm}(\vec{\mathbf{l}}^\top \mathbf{a} \vec{\mathbf{r}}) \wedge C = \text{Comm}(\mathbf{a})\}.$$

- $\text{Verify}_{\text{open}}(\text{pp}, C, C_v, \vec{\mathbf{l}}, \vec{\mathbf{r}}; \pi_{\text{open}}) \rightarrow \{\text{true}, \text{false}\}$: *Output true iff π_{open} is valid.*

Algorithm 1. LiteBullet Proof

Input: $m \in \mathbb{N}; c \in \mathbb{F}; \vec{a}, \vec{b} \in \mathbb{F}^m$;
Output: $\mathcal{P}$: $\pi := (\iota_{(1)}, \dots, \iota_{(\log_2 m)}, \rho_{(1)}, \dots, \rho_{(\log_2 m)}, \hat{a}, \hat{b})$; $\mathcal{V}$: true or false

- 1 **foreach** $j \in \{1, \dots, \log_2 m\}$; // with zero padding, m is a power of two
- 2 **do**
- 3 $\mathcal{P}$ splits $\vec{a}$ and $\vec{b}$ into halves $(\vec{a}_{\text{LH}} \parallel \vec{a}_{\text{RH}}) \xleftarrow{\text{half}} \vec{a}$; $(\vec{b}_{\text{LH}} \parallel \vec{b}_{\text{RH}}) \xleftarrow{\text{half}} \vec{b}$;
- 4 $\mathcal{P}$ computes $\iota_{(j)} \leftarrow (\vec{a}_{\text{LH}} \bullet \vec{b}_{\text{RH}}) \in \mathbb{F}$, $\rho_{(j)} \leftarrow (\vec{a}_{\text{RH}} \bullet \vec{b}_{\text{LH}}) \in \mathbb{F}$;
- 5 $\mathcal{P} \rightarrow \mathcal{V}$: $\iota_{(j)}, \rho_{(j)} \in \mathbb{F}$;
- 6 $\mathcal{V} \rightarrow \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathbb{F}^*$;
- 7 $\mathcal{P}$ folds $\vec{a} \leftarrow \vec{a}_{\text{LH}} + x_{(j)}^{-1} \vec{a}_{\text{RH}} \in \mathbb{F}^{m/2^j}$, $\vec{b} \leftarrow \vec{b}_{\text{LH}} + x_{(j)} \vec{b}_{\text{RH}} \in \mathbb{F}^{m/2^j}$;
- 8 $\mathcal{P} \rightarrow \mathcal{V}$: $\hat{a} \leftarrow \vec{a}[1] \in \mathbb{F}$; $\hat{b} \leftarrow \vec{b}[1] \in \mathbb{F}$; // $\mathcal{P}$ folded $\vec{a}, \vec{b}$ into scalars $\hat{a}, \hat{b}$
- 9 $\mathcal{P}$ and $\mathcal{V}$ compute the tensor challenges $\vec{\xi}^{-1} \leftarrow \text{ts}(x^{-1})$, $\vec{\xi} \leftarrow \text{ts}(x)$;
- 10 $\mathcal{V}$ checks $\hat{a}\hat{b} \stackrel{?}{=} c + \sum_{j=1}^{\log_2 m} (x_{(j)} \iota_{(j)} + x_{(j)}^{-1} \rho_{(j)})$;
- 11 $\mathcal{V}$ checks $\hat{a} \stackrel{?}{=} \vec{a} \bullet \vec{\xi}^{-1}$ and $\hat{b} \stackrel{?}{=} \vec{b} \bullet \vec{\xi}$; // w.r.t. unfolded $\vec{a}, \vec{b}$
- 12 **return** true if all of the above checks return true, else **return** false;

4 Lightweight Inner Product Proof

Inner product is essential for matrix computations. We present an inner product proof, LiteBullet, a polynomial-free alternative to the well-known sumcheck protocol (see Appendix E.3 for a recap). LiteBullet underpins the development of proofs for other matrix operations in subsequent sections.

Definition 4 (Inner Product Proof). *Given a value $c \in \mathbb{F}$ and vectors $\vec{a}, \vec{b} \in \mathbb{F}^m$, an inner product proof $(\text{Setup}_{\text{ip}}, \mathcal{P}_{\text{ip}}, \mathcal{V}_{\text{ip}})$ is a proof (Definition 6, Appendix B) for $\mathcal{L}_{\text{ip}}$:*

$$\mathcal{L}_{\text{ip}} = \left\{ (m \in \mathbb{N}_{\geq 1}, c \in \mathbb{F}, \vec{a}, \vec{b} \in \mathbb{F}^m) : c = \vec{a} \bullet \vec{b} \right\}. \quad (3)$$

4.1 LiteBullet Proof

LiteBullet is inspired by Bulletproofs [21] (Appendix E.1), a folding-based inner product argument, but it drops knowledge extractability, which requires group operations (Lines 5–6 and 11, Algorithm 4, Appendix E.1) that are slower than field operations. By relaxing extractability, LiteBullet uses only field operations, which can match or surpass the efficiency of the sumcheck protocol.

Algorithm 1 details the LiteBullet protocol for $\mathcal{L}_{\text{ip}}$ in Eq. (3). Its Line 4 (the counterpart of Lines 5–6 of Algorithm 4), only computes field inner products $\iota_{(j)} = \vec{a}_{\text{LH}} \bullet \vec{b}_{\text{RH}}$ and $\rho_{(j)} = \vec{a}_{\text{RH}} \bullet \vec{b}_{\text{LH}}$, avoiding group operations. Vectors $\vec{a}_{\text{LH}}$ and $\vec{a}_{\text{RH}}$ are the left and right halves of $\vec{a}$, and similarly for $\vec{b}$. At Line 11, the verifier checks $\hat{a} = \vec{a} \bullet \vec{\xi}^{-1}$ and $\hat{b} = \vec{b} \bullet \vec{\xi}$. These checks can be done directly or

proven using additional proofs, which lets **LiteBullet** disentangle proofs about $\vec{a}$ and $\vec{b}$ and enables recursive reductions across related operations.

The **LiteBullet** protocol in Algorithm 1 is presented in its interactive form. It can be converted to a non-interactive version via the Fiat–Shamir transform (Definition 23, Appendix B.8), with the following interface:

- **LiteBullet.Prove**($c, \vec{a}, \vec{b}$) $\rightarrow \pi$: Outputs a non-interactive proof π ;
- **LiteBullet.Verify**($\pi, c, \vec{a}, \vec{b}$) $\rightarrow \text{true/false}$: Verifies the proof π .

In later sections, we employ a simplified verifier interface **LiteBullet.Verify**(π, c) $\rightarrow \text{true/false}$, which corresponds to the non-interactive verifier of Algorithm 1 without executing the final projection check at Line 11. These projections are verified by other protocols, thus ensuring that $\vec{a}$ and $\vec{b}$ remain hidden from the verifier.

Soundness stems from round-by-round soundness as in Definition 19, Appendix B.6. Given public $\vec{a}$ and $\vec{b}$, define $\vec{a}_{(j)}$, $\vec{b}_{(j)}$, and state predicates **State_j** as:

$$\begin{aligned} \vec{a}_{(0)} &\leftarrow \vec{a}, \vec{b}_{(0)} \leftarrow \vec{b}, \\ \vec{a}_{(j)} &\leftarrow \vec{a}_{(j-1)\text{LH}} + x_{(j)}^{-1} \vec{a}_{(j-1)\text{RH}}, \vec{b}_{(j)} \leftarrow \vec{b}_{(j-1)\text{LH}} + x_{(j)} \vec{b}_{(j-1)\text{RH}}, \quad j \in [\log_2 m], \\ \text{State}_0 &\leftarrow (c = \vec{a} \bullet \vec{b}), \text{State}_j \leftarrow \left(c + \sum_{t=1}^j (x_{(j)} \iota_j + x_{(j)}^{-1} \rho_{(j)}) = \vec{a}_{(j)} \bullet \vec{b}_{(j)} \right). \end{aligned}$$

For each iteration $j \in [\log_2 m]$, the probability that **State_{j-1}** = **false** but **State_j** = **true** is negligible ($2/(p-1)$), so does the probability that **State₀** = **false** ($c \neq \vec{a} \bullet \vec{b}$) but **State_{log₂ m}** = **true** (i.e., **LiteBullet** returns **true**) by a union bound.

Theorem 1. *Algorithm 1 is a proof for $\mathcal{L}_{\text{ip}}$ with soundness error $\frac{2 \log_2 m}{p-1}$.*

Its proof is in Appendix F.1. **LiteBullet** mirrors the widely used sumcheck protocol but relies on folding rather than polynomials. Appendix A.4 develops this connection and shows how **LiteBullet** avoids converting matrices into polynomials and back, simplifying proof design for matrix operations.

4.2 Batch Opening Multiple Matrix Projections

Distinct Matrix Projections. **LiteBullet** enables batch opening of k matrix projections $v_i = \vec{l}_i^\top \mathbf{a}_i \vec{r}_i, i \in [k]$ with a single projection on a concatenated matrix $(\mathbf{a}_1 \parallel \dots \parallel \mathbf{a}_k)$. This is essential for the succinctness of our proof system. Specifically, **LiteBullet** provides a proof for:

$$\mathcal{L}_{\text{concat}} = \left\{ \left((n, k), v_i, \vec{l}_i, \vec{r}_i, \mathbf{a}_i \in \mathbb{F}^{n \times n}, i \in [k] \right) : \left(v_i = \vec{l}_i^\top \mathbf{a}_i \vec{r}_i, \forall i \in [k] \right) \right\}.$$

Upon a random challenge $\theta \in \mathbb{F}^*$, $\mathcal{L}_{\text{concat}}$ is proven via a linear combination:

$$\begin{aligned} \sum_{i=1}^k \theta^{i-1} \cdot v_i &= \sum_{i=1}^k \theta^{i-1} \cdot (\vec{l}_i^\top \mathbf{a}_i \vec{r}_i) \\ &= \text{vec}(\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k) \bullet \text{vec}(\vec{l}_1 \otimes \vec{r}_1^\top \parallel \theta \cdot \vec{l}_2 \otimes \vec{r}_2^\top \parallel \dots \parallel \theta^{k-1} \cdot \vec{l}_k \otimes \vec{r}_k^\top). \end{aligned}$$

$\text{vec}(\mathbf{a}) \bullet \text{vec}(\vec{\mathbf{l}} \otimes \vec{\mathbf{r}}^\top) = \mathbf{a} \bullet (\vec{\mathbf{l}} \otimes \vec{\mathbf{r}}^\top) = \sum_{i=1}^n \sum_{j=1}^n a_{ij} l_i r_j = \vec{\mathbf{l}}^\top \mathbf{a} \vec{\mathbf{r}}$, where $\text{vec}(\mathbf{a})$ flattens a matrix $\mathbf{a}$ into a vector.

A LiteBullet proof then reduces the verification of $\mathcal{L}_{\text{concat}}$ to checking one inner product of $\text{vec}(\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k)$ with a tensor-structured vector $\vec{\xi}^{-1} = \text{ts}(\mathbf{x}) \in \mathbb{F}^{kn^2}$, where $\mathbf{x} \in \mathbb{F}^{\log_2(kn^2)}$ are random challenges sampled by LiteBullet.

Since $\vec{\xi}^{-1}$ is a Kronecker tensor, it is decomposed into $\vec{\xi}^{-1} = \vec{\mathbf{l}} \otimes \vec{\mathbf{r}}$, where $\vec{\mathbf{l}} \in \mathbb{F}^n$ and $\vec{\mathbf{r}} \in \mathbb{F}^{kn}$. The reduced inner product is equivalent to the matrix projection $\vec{\mathbf{l}}^\top (\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k) \vec{\mathbf{r}}$. Thus, we design a ConcatProj proof for $\mathcal{L}_{\text{concat}}$.

ConcatProj proof for $v_i = \vec{\mathbf{l}}_i^\top \mathbf{a}_i \vec{\mathbf{r}}_i, \forall i \in [k]$, creating proof π

Input: $n, k \in \mathbb{N}, v_i \in \mathbb{F}, \vec{\mathbf{l}}_i, \vec{\mathbf{r}}_i \in \mathbb{F}^n, \mathbf{a}_i \in \mathbb{F}^{n \times n}, i \in [k]$. **Output:** (true, $v, \vec{\mathbf{l}}, \vec{\mathbf{r}}$) or false

- $\mathcal{P} \rightarrow \mathcal{V}: \theta \xleftarrow{\$} \mathbb{F}^*$; $\mathcal{P}$ and $\mathcal{V}$ compute $v \leftarrow \sum_{i=1}^k \theta^{i-1} \cdot v_i$;
- $\mathcal{P}$ and $\mathcal{V}$ run $\pi \leftarrow \text{LiteBullet.Prove}()$ for v being the inner product as defined below:
 $v = \text{vec}(\mathbf{a}_1 \parallel \mathbf{a}_2 \parallel \dots \parallel \mathbf{a}_k) \bullet \text{vec}(\vec{\mathbf{l}}_1 \otimes \vec{\mathbf{r}}_1^\top \parallel \theta \cdot \vec{\mathbf{l}}_2 \otimes \vec{\mathbf{r}}_2^\top \parallel \dots \parallel \theta^{k-1} \cdot \vec{\mathbf{l}}_k \otimes \vec{\mathbf{r}}_k^\top)$;
- $\mathcal{P}$ and $\mathcal{V}$ compute $\vec{\mathbf{l}} \in \mathbb{F}^n$ and $\vec{\mathbf{r}} \in \mathbb{F}^{kn}$ using the random challenges in LiteBullet;
- Run $\text{ok} := \text{LiteBullet.Verify}(\pi, v)$; if $\text{ok} = \text{true}$, **return** (ok, $\pi, v, \vec{\mathbf{l}}, \vec{\mathbf{r}}$); false otherwise;

Same Matrix Projected onto Multiple Vectors. Similarly, LiteBullet provides a batched proof for k projections of the same matrix $\mathbf{a} \in \mathbb{F}^{n \times n}$ onto multiple vectors $\vec{\mathbf{l}}_i, \vec{\mathbf{r}}_i, i \in [k]$. The target language $\mathcal{L}_{\text{batch}}$ is:

$$\mathcal{L}_{\text{batch}} = \left\{ \left((n, k), \{v_i, \vec{\mathbf{l}}_i, \vec{\mathbf{r}}_i\}_{i \in [k]}, \mathbf{a} \in \mathbb{F}^{n \times n} \right) : \left(v_i = \vec{\mathbf{l}}_i^\top \mathbf{a} \vec{\mathbf{r}}_i, \forall i \in [k] \right) \right\},$$

This is achieved by using a random challenge $\theta \in \mathbb{F}^*$ and checking:

$$\sum_{i=1}^k \theta^{i-1} \cdot v_i = \sum_{i=1}^k \theta^{i-1} \cdot (\vec{\mathbf{l}}_i^\top \mathbf{a} \vec{\mathbf{r}}_i) = \text{vec}(\mathbf{a}) \bullet \text{vec}\left(\sum_{i=1}^k \theta^{i-1} \cdot (\vec{\mathbf{l}}_i \otimes \vec{\mathbf{r}}_i^\top)\right).$$

Thus, we design the following BatchProj proof for $\mathcal{L}_{\text{batch}}$.

BatchProj proof for $v_i = \vec{\mathbf{l}}_i^\top \mathbf{a} \vec{\mathbf{r}}_i, \forall i \in [k]$, creating proof π

Input: $n, k \in \mathbb{N}, v_i \in \mathbb{F}, \vec{\mathbf{l}}_i, \vec{\mathbf{r}}_i \in \mathbb{F}^n, i \in [k], \mathbf{a} \in \mathbb{F}^{n \times n}$. **Output:** (true, $v, \vec{\mathbf{l}}, \vec{\mathbf{r}}$) or false

- $\mathcal{P} \rightarrow \mathcal{V}: \theta \xleftarrow{\$} \mathbb{F}^*$; $\mathcal{P}$ and $\mathcal{V}$ computes $v \leftarrow \sum_{i=1}^k \theta^{i-1} \cdot v_i$;
- $\mathcal{P}$ and $\mathcal{V}$ run $\pi \leftarrow \text{LiteBullet.Prove}(v, \text{vec}(\mathbf{a}), \text{vec}(\sum_{i=1}^k \theta^{i-1} \cdot (\vec{\mathbf{l}}_i \otimes \vec{\mathbf{r}}_i^\top)))$;
- $\mathcal{P}$ and $\mathcal{V}$ compute $\vec{\mathbf{l}} \in \mathbb{F}^n$ and $\vec{\mathbf{r}} \in \mathbb{F}^n$ using the random challenges in LiteBullet;
- Run $\text{ok} := \text{LiteBullet.Verify}(\pi, v)$; if $\text{ok} = \text{true}$, **return** (ok, $\pi, v, \vec{\mathbf{l}}, \vec{\mathbf{r}}$); false otherwise;

Claim 1. *The soundness errors of the ConcatProj and BatchProj proofs for k projections of $n \times n$ matrices are $\frac{k+4 \log_2 n + 2 \log_2 k}{p-1}$ and $\frac{k+4 \log_2 n}{p-1}$, respectively. (The proof is provided in Appendix F.2.)*

5 Proofs for Atomic Type-1 Operations

We use LiteBullet to develop atomic Evalyn proofs for matrix operations that will compose into larger proofs in later sections. Operations sharing the Evalyn interface are classified as Type-1, including addition, subtraction, scalar multiplication, transpose, matrix multiplication, Frobenius inner product, Hadamard (element-wise) product, Kronecker product, concatenation, and truncation.

For a binary Type-1 operation $\otimes$, an atomic Evalyn proof is designed to prove the projection $v = \vec{l}^\top (\mathbf{a} \otimes \mathbf{b}) \vec{r}$ via the language:

$$\mathcal{L}_\otimes = \left\{ (v, \vec{l}, \vec{r}, \mathbf{a}, \mathbf{b}) : v = \vec{l}^\top (\mathbf{a} \otimes \mathbf{b}) \vec{r} \right\}.$$

Given public $v \in \mathbb{F}$ and tensor-structured random vectors $\vec{l}$ and $\vec{r}$, the proof reduces the projection v to independently checking the projections $v_L = \vec{l}_L^\top \mathbf{a} \vec{r}_L$ and $v_R = \vec{l}_R^\top \mathbf{b} \vec{r}_R$ for the left and right input matrices. At each node of the reduction tree, v_L and v_R are derived from the prover's messages. The verifier uses $\vec{l}$, $\vec{r}$, and the random challenge to compute $(\vec{l}_L, \vec{r}_L)$ and $(\vec{l}_R, \vec{r}_R)$. Proofs for unary Type-1 operations, $v = \vec{l}^\top (\otimes \mathbf{a}) \vec{r}$, proceed in a similar manner.

Definition 5 (Atomic Evalyn Proof). *The Evalyn interface for a matrix operation $\otimes$ is an algorithm tuple $(\otimes.\text{Prove}, \otimes.\text{Eval}, \otimes.\text{Link}, \otimes.\text{Verify})$.*

- $\otimes.\text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b}) \rightarrow \pi$: Given scalar v , tensor-structured public vectors $\vec{l}$ and $\vec{r}$, and matrices $\mathbf{a}$ and $\mathbf{b}$, produce a transcript π to be sent to the verifier.
- $\otimes.\text{Eval}(\pi) \rightarrow (v_L, v_R)$: Given a transcript π , output v_L and v_R that correspond to the projections of $\mathbf{a}$ and $\mathbf{b}$, respectively.
- $\otimes.\text{Link}(\vec{l}, \vec{r}; \pi) \rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$: Derive Fiat–Shamir randomness $\mathbf{x}$ from π , and use $\vec{l}, \vec{r}$, and $\mathbf{x}$ to compute projecting vectors $(\vec{l}_L, \vec{r}_L)$ and $(\vec{l}_R, \vec{r}_R)$.
- $\otimes.\text{Verify}(v, \pi, v_L, v_R) \rightarrow \text{true/false}$: Given the transcript π and (v_L, v_R) , check that π is valid and consistent with v and (v_L, v_R) .

These algorithms compose into an atomic Evalyn proof for $\mathcal{L}_\otimes$ as follows:

Atomic Evalyn proof for $\mathcal{L}_\otimes = \{(v, \vec{l}, \vec{r}, \mathbf{a}, \mathbf{b}) : v = \vec{l}^\top (\mathbf{a} \otimes \mathbf{b}) \vec{r}\}$ creating proof π

- $\mathcal{P} \rightarrow \mathcal{V}$: $\pi \leftarrow \otimes.\text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b})$;
- $\mathcal{V}$ runs $(v_L, v_R) \leftarrow \otimes.\text{Eval}(\pi)$ and $(\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R) \leftarrow \otimes.\text{Link}(\vec{l}, \vec{r}; \pi)$;
- $\mathcal{V}$ runs $\text{true/false} \leftarrow \otimes.\text{Verify}(v, \pi, v_L, v_R)$ and checks $v_L \stackrel{?}{=} \vec{l}_L^\top \mathbf{a} \vec{r}_L$ and $v_R \stackrel{?}{=} \vec{l}_R^\top \mathbf{b} \vec{r}_R$;
return true iff all checks return true;

Atomic Evalyn Proof as RoK. Atomic Evalyn proofs mirror the Kothapalli–Parno RoK [54] (Appendix B.2) RoK reduces a witness knowledge check in a relation $\mathcal{R}_1$ on a statement u_1 to a simpler one in $\mathcal{R}_2$ on statement u_2 via a Reduce algorithm over the transcript π . In Evalyn, Eval and Link together implement Reduce by processing the prover messages and verifier challenges of transcript π ,

respectively. While Evalyn natively performs proof reduction, adding a commitment converts it into a full RoK.

The rest of this section constructs atomic Evalyn proofs for multiplication and the Kronecker product, deferring other Type-1 operations (addition, scalar multiplication, transpose, Frobenius inner product, Hadamard product, concatenation, and truncation) to Appendix C. We will use `LiteBullet.Prove( $c, \vec{a}, \vec{b}$ )` and `LiteBullet.Verify( $\pi, c$ )` for $c = \vec{a} \bullet \vec{b}$ with proof π described in Section 4.1.

5.1 Matrix Multiplication

Matrix multiplication is proven using an inner product proof. We know:

$$\left\{ v = \vec{l}^\top (ab) \vec{r} \right\} \iff \left\{ v = (\vec{l}^\top a) \bullet (b \vec{r}) \right\}.$$

Given $\vec{l}$ and $\vec{r}$, the prover computes $\vec{a}_l = \vec{l}^\top a$ and $\vec{b}_r = b \vec{r}$. The matrix projection $v = \vec{l}^\top (ab) \vec{r}$ is valid iff the inner product equation $v = \vec{a}_l \bullet \vec{b}_r$ holds. Therefore, an inner-product proof for $v = \vec{a}_l \bullet \vec{b}_r$ serves as a proof for $v = \vec{l}^\top (ab) \vec{r}$.

LiteBullet reduces the verification of $v = \vec{a}_l \bullet \vec{b}_r$ to checking $\hat{a} = \vec{a}_l \bullet \vec{\xi}^{-1}$ and $\hat{b} = \vec{b}_r \bullet \vec{\xi}$ for a tensor-structured probe $\vec{\xi} \leftarrow \text{ts}(\mathbf{x})$ derived from the verifier challenges (Line 11, Algorithm 1). This is equivalent to verifying $\hat{a} = \vec{l}^\top a \vec{\xi}^{-1}$ and $\hat{b} = \vec{\xi}^\top b \vec{r}$. In the LiteBullet transcript, $\hat{a}$ and $\hat{b}$ are the final elements.

Thus, we design the atomic Evalyn proof for matrix multiplication as follows:

- `mul.Prove( $v, \vec{l}, \vec{r}; a, b$ )` $\rightarrow \pi$. The prover computes $\vec{a}_l \leftarrow \vec{l}^\top a$ and $\vec{b}_r \leftarrow b \vec{r}$. Then, it invokes $\pi \leftarrow \text{LiteBullet.Prove}(v, \vec{a}_l, \vec{b}_r)$ and outputs π .
- `mul.Eval( $\pi$ )` $\rightarrow (v_L, v_R)$. It outputs the final two scalars of π .
- `mul.Link( $\vec{l}, \vec{r}, \pi$ )` $\rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$. Using the randomness $\mathbf{x}$ derived from π , it computes $\vec{\xi} \leftarrow \text{ts}(\mathbf{x})$ and outputs $\vec{l}_L \leftarrow \vec{l}, \vec{r}_L \leftarrow \vec{\xi}^{-1}, \vec{l}_R \leftarrow \vec{\xi}$, and $\vec{r}_R \leftarrow \vec{r}$.
- `mul.Verify( $v, \pi, v_L, v_R$ )` $\rightarrow \text{true/false}$. It returns `true` if v_L and v_R are the last two elements of π , and `LiteBullet.Verify( $v, \pi$ )` returns `true`.

5.2 Kronecker Product

The Kronecker product proof uses direct factorization rather than an inner product proof. We rely on the equivalence:

$$\left\{ v = \vec{l}^\top (a \otimes b) \vec{r} \text{ s.t. } \vec{l} = \vec{l}_L \otimes \vec{l}_R, \vec{r} = \vec{r}_L \otimes \vec{r}_R \right\} \iff \left\{ v = (\vec{l}_L^\top a \vec{r}_L) (\vec{l}_R^\top b \vec{r}_R) \right\}.$$

- $\otimes.\text{Prove}(v, \vec{l}, \vec{r}; a, b) \rightarrow \pi$. It finds $\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R$ s.t. $\vec{l} = \vec{l}_L \otimes \vec{l}_R, \vec{r} = \vec{r}_L \otimes \vec{r}_R$, computes $v_L \leftarrow \vec{l}_L^\top a \vec{r}_L, v_R \leftarrow \vec{l}_R^\top b \vec{r}_R$, and outputs $\pi \leftarrow (v_L, v_R)$.
- $\otimes.\text{Eval}(\pi) \rightarrow (v_L, v_R)$. It outputs the final two scalars of π .
- $\otimes.\text{Link}(\vec{l}, \vec{r}, \pi) \rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$. It outputs $\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R$ s.t. $\vec{l} = \vec{l}_L \otimes \vec{l}_R$ and $\vec{r} = \vec{r}_L \otimes \vec{r}_R$.
- $\otimes.\text{Verify}(v, \pi, v_L, v_R) \rightarrow \text{true/false}$. It checks $v \stackrel{?}{=} v_L \cdot v_R$ and $\pi \stackrel{?}{=} (v_L, v_R)$.

5.3 Complexity and Soundness Error of Type-1 Operation Proofs

Consider matrices with dimensions bounded by $n \times n$. Each atomic Evalyn proof uses at most one n^2 -inner-product proof. The round complexity is at most $2\log_2 n$. For all Type-1 matrix operations, $\otimes$.Prove requires at most $6n^2 + O(\log n)$ field operations, $\otimes$.Eval requires $O(1)$, and $\otimes$.Link and $\otimes$.Verify each take at most $O(\log n)$. The total prover time is capped at $O(n^2)$.

Claim 2. *All atomic Evalyn proofs for Type-1 operations satisfy perfect completeness and soundness over a field of size p . The soundness error of atomic Evalyn proofs for Type-1 operations is at most $\frac{4\log_2 n}{p-1}$. (The proof is provided in Appendix F.3.)*

6 Proofs for Type-2 Operations via Type-1 Expressions

Element-wise non-linear function, convolution, max pooling, and normalization are central in NN computations, yet their proofs do not fit the Evalyn interface and are classified as Type-2 operations. Even so, each can be reformulated as a Type-1 expression and handled within our unified framework. Below, we focus on activation functions. See Appendix C for other Type-2 operations.

6.1 Lookup Proof

Proofs for non-linear functions in Section 6.2 rely on lookup proofs, which allow an untrusted prover to commit to a vector $\vec{c} \in \mathbb{F}^m$ and prove that all entries of $\vec{c}$ reside in a predetermined table $\vec{v} \in \mathbb{F}^n$. Lasso [73] is a recent lookup proof that incorporates the memory-checking technique from Spartan [71]. Below, we refine the lookup proof of Lasso by reformulating it in terms of matrix equations.

To prove that all entries of $\vec{c} \in \mathbb{F}^m$ reside in $\vec{v} \in \mathbb{F}^n$, define $\vec{s} \in \mathbb{F}^n$ where $s_{i \in [n]}$ counts the occurrences of v_i in $\vec{c}$. Moreover, define $\vec{t} \in \mathbb{F}^m$ where $t_{i \in [m]}$ records how many times c_i appears in the previous entries $\{c_1, \dots, c_{i-1}\}$. By definition, $0 \leq s_i \leq m, \forall i \in [n]$ and $0 \leq t_i \leq m, \forall i \in [m]$. Committing to $\vec{s}$ and $\vec{t}$ is efficient as their entries are bounded and have short bitwidth. Verifying that $\vec{c}$ is looked up from $\vec{v}$ reduces to checking whether the below two vectors of scalar pairs are permutations of each other as multisets:

$$((\vec{c} \parallel \vec{v}), (\vec{t} \parallel \vec{s})), \quad ((\vec{v} \parallel \vec{c}), (\vec{0} \parallel (\vec{t} + \vec{1}))).$$

This check employs a random challenge $z \in \mathbb{F}^*$, ensuring the products of entries from the following vectors are identical:

$$\vec{1} + z(\vec{c} \parallel \vec{v}) + z^2(\vec{t} \parallel \vec{s}), \quad \vec{1} + z(\vec{v} \parallel \vec{c}) + z^2(\vec{0} \parallel (\vec{t} + \vec{1})).$$

We define “grand products” $\Theta_1, \Theta_2, \Theta_3, \Theta_4$ below and require $\Theta_1 \cdot \Theta_2 = \Theta_3 \cdot \Theta_4$:

$$\begin{aligned} \Theta_1 &= \prod_{(c_i, t_i) \in \{(c, t)\}} (1 + zc_i + z^2t_i), & \Theta_2 &= \prod_{(v_i, s_i) \in \{(v, s)\}} (1 + zv_i + z^2s_i), \\ \Theta_3 &= \prod_{(c_i, t_i) \in \{(c, t)\}} (1 + zc_i + z^2(t_i + 1)), & \Theta_4 &= \prod_{v_i \in \{v\}} (1 + zv_i). \end{aligned}$$

Proof for Grand Product. Lasso proves these grand products using the Quarks technique [72], which commits to full-bitwidth vectors of lengths $O(m)$ and $O(n)$, where m and n are the sizes of the target vector and the lookup table, respectively. This undermines the main benefit of Lasso of committing only to low-bitwidth values. We propose an alternative technique to prove that a scalar $c \in \mathbb{F}$ equals the product of entries of a vector $\vec{a} \in \mathbb{F}^m$, *i.e.*, $c = \prod_{i=1}^m a_i$, using a chain of Hadamard product proofs via LiteBullet (Appendix C.4):

$$\vec{a}_0 \leftarrow \vec{a}, \quad \vec{a}_{j+1} \leftarrow \vec{a}_{j\text{LH}} \circ \vec{a}_{j\text{RH}}, \quad j \in [0, \log_2 m - 1], \quad c \leftarrow \vec{a}_{\log_2 m}.$$

The prover time of the j -th Hadamard product proof is $O(m/2^j)$, leading to a total prover time of $O(m)$. The proof size and verifier time are both $O(\log^2 m)$.

This grand product proof transforms lookup proofs into Type-1 expressions, with leaf nodes $\mathbf{c}$, $\mathbf{v}$, $\mathbf{t}$, and $\mathbf{s}$ all low-bitwidth matrices.

Optimization for Structured Table. Lasso achieves sublinear prover time in the table size for structured tables $\vec{v}$. If $\vec{v} = \vec{v}_1 \otimes \vec{v}_2$, then proving that $\vec{c}$ is looked up from $\vec{v}$ can proceed by decomposing $\vec{c}$ into $(\vec{c}_1, \vec{c}_2)$. The prover proves $\vec{c} = \vec{c}_1 \circ \vec{c}_2$ and that $(\vec{c}_1, \vec{c}_2)$ is looked up from $(\vec{v}_1, \vec{v}_2)$. Such structure is common; *e.g.*, the range-proof table $(-2^k + 1, \dots, 0, \dots, 2^k - 1)$ is tensor-structured.

Rounding. Rounding of floating-point matrices can also use lookup proofs on structured tables. To prove that a lower-bitwidth matrix $\mathbf{c}$ with η_c -bit entries approximates a higher-bitwidth matrix $\mathbf{a}$ with η_a -bit entries, it suffices to show that all entries of $(2^{\eta_a - \eta_c} \cdot \mathbf{c} - \mathbf{a})$ lie in the table $\{-2^{\eta_a - \eta_c - 1} + 1, \dots, 0, \dots, 2^{\eta_a - \eta_c - 1}\}$. Rounding in quantized NNs uses a similar approach (see Appendix D).

6.2 Element-Wise Non-Linear Function

Lookup proofs can handle element-wise non-linear functions as in zkLLM [74]. Proving $c = \sigma(a)$ becomes a membership check for the pair (a, c) in a precomputed table. We improve zkLLM by minimizing the table size for a monotone increasing $\sigma(\cdot)$ —only the image set is stored in a value table $\mathbf{v}$, with two key tables $\mathbf{k}_1$ and $\mathbf{k}_2$ giving, for each value, the minimum and maximum keys:

$$\{(\mathbf{k}_1, \mathbf{k}_2, \mathbf{v})\} := \{(a_1, a_2, c) : c = \sigma(a_1) = \sigma(a_2) \wedge \forall b \in \sigma^{-1}(c), a_1 \leq b \leq a_2\}.$$

This configuration reduces the table size to the cardinality of the image of $\sigma(\cdot)$.

With these key and value tables, proving $\mathbf{c} = \sigma(\mathbf{a})$ reduces to proving the existence of matrices $\mathbf{a}_1$ and $\mathbf{a}_2$ such that

$$\mathbf{a}_1 \leq \mathbf{a} \leq \mathbf{a}_2 \quad \wedge \quad \{(\mathbf{a}_1, \mathbf{a}_2, \mathbf{c})\} \subset \{(\mathbf{k}_1, \mathbf{k}_2, \mathbf{v})\}.$$

With NN quantization techniques [29, 37, 82, 83, 86], the image of $\sigma(\cdot)$ becomes a small set. For example, with 8-bit quantization⁸ [86], the cardinality of the image of $\sigma(\cdot)$ is at most 2^8 . Consequently, the lookup proof for $\{(\mathbf{a}_1, \mathbf{a}_2, \mathbf{c})\} \subset$

⁸ 8-bit quantization is common for NN inference, as noted in a whitepaper [65, §1].

$\{(\mathbf{k}_1, \mathbf{k}_2, \mathbf{v})\}$ is efficient. The inequalities $\mathbf{a}_1 \leq \mathbf{a} \leq \mathbf{a}_2$, as observed by Lasso [73], are efficiently proven using range proofs via lookup proofs on structured tables.

Since lookup proofs can be transformed into Type-1 expressions, non-linear functions can also be transformed into Type-1 expressions, aligning with our Evalyn paradigm and enabling proof composition within our unified framework.

Comparison-based layers (*e.g.*, ReLU, MaxPool) are quite common across NN architectures. When revealing their presence is acceptable, our range proofs could be replaced by specialized constructions [62] tuned to different trade-offs.

Hiding the Non-Linear Function via Lookup Table Commitments. We treat the non-linear function as a model hyperparameter encoded as a committed lookup table. The tables are part of the model parameters as commitments rather than being provided by the prover. After the lookup proof, the prover provides a zero-knowledge proof that the table projection is consistent with the table commitment, without revealing the table itself.⁹

7 Composition of Atomic Operation Proofs

In this section, we combine atomic Evalyn proofs from previous sections to construct composite proofs for complex matrix expressions.

Consider the NN layer $\mathbf{X}_i = \sigma(\mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i)$. By introducing intermediate matrices $\mathbf{c}_{1,i}$ and $\mathbf{c}_{2,i}$, this is decomposed into three atomic operations:

$$\mathbf{c}_{1,i} = \mathbf{W}_i \mathbf{X}_{i-1}, \quad \mathbf{c}_{2,i} = \mathbf{c}_{1,i} + \mathbf{B}_i, \quad \mathbf{X}_i = \sigma(\mathbf{c}_{2,i}).$$

Given commitments to $\mathbf{X}_{i-1}$, $\mathbf{W}_i$, $\mathbf{B}_i$, and $\mathbf{X}_i$, the proofs are constructed recursively. It first proves $\mathbf{X}_i = \sigma(\mathbf{c}_{2,i})$ using lookup proof in Section 6.2 with table $(\mathbf{k}_1, \mathbf{k}_2, \mathbf{v})$ and auxiliary inputs, which reduces to projections of $\mathbf{c}_{2,i}$, $\mathbf{X}_i$, $\mathbf{k}_1$, $\mathbf{k}_2$, $\mathbf{v}$, and the auxiliary matrices. It then proves $\mathbf{c}_{2,i} = \mathbf{c}_{1,i} + \mathbf{B}_i$ with another proof, which reduces to projections of $\mathbf{c}_{1,i}$ and $\mathbf{B}_i$. Finally, it proves $\mathbf{c}_{1,i} = \mathbf{W}_i \mathbf{X}_{i-1}$ with a multiplication proof, reducing to projections of $\mathbf{W}_i$ and $\mathbf{X}_{i-1}$.

Moreover, the element-wise non-linear functions $\mathbf{X}_i = \sigma(\mathbf{c}_{2,i})$, $i \in [D]$, across all NN layers can be collectively proven by a single grand lookup proof for:

$$(\mathbf{X}_1 \parallel \cdots \parallel \mathbf{X}_D) = \sigma(\mathbf{c}_{2,1} \parallel \cdots \parallel \mathbf{c}_{2,D}).$$

This batch lookup proof is more efficient than proving D separate lookups. It reduces to projections of $(\mathbf{X}_1 \parallel \cdots \parallel \mathbf{X}_D)$ and $(\mathbf{c}_{2,1} \parallel \cdots \parallel \mathbf{c}_{2,D})$. The projection of $(\mathbf{X}_1 \parallel \cdots \parallel \mathbf{X}_D)$ is proven by using $\text{Prove}_{\text{open}}$ of the matrix commitment scheme. The projection of $(\mathbf{c}_{2,1} \parallel \cdots \parallel \mathbf{c}_{2,D})$, via the concatenation proof in Appendix C.2, reduces to projections of each $\mathbf{c}_{2,i}$. For each $i \in [D]$, the reduction for $\mathbf{c}_{2,i} = \mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i$ proceeds as above.

⁹ A potential leakage channel is the table size. Functions with small image size (*e.g.*, ReLU, where many inputs map to 0) admit shorter optimized tables. Interval-based lookup [52] can also reduce the table size. We thus fix the table size by the input and output bitwidths and avoid function-specific optimization.

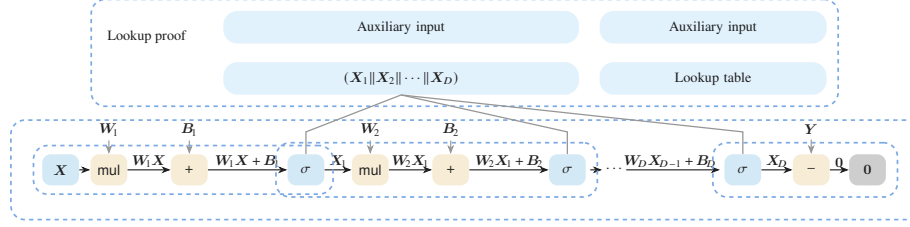


Fig. 5. Commitments to new leaf nodes (blue rectangles), resulting from a Type-2-to-Type-1 transformation, partition the DAG into independent subgraphs (dashed box)

7.1 Commitment for Type-2-to-Type-1 Transformation

In the NN example in Fig. 5, commitments to the outputs of non-linear operations cut the computational DAG into smaller subgraphs. This cut replaces each Type-2 node by committed leaf matrices. In general, transforming Type-2 operations into Type-1 expressions introduces additional leaf nodes.

Claim 3. *A matrix expression with M' Type-2 operations can be rewritten as a DAG with $k' = O(M')$ new leaf nodes and $J = O(M')$ subgraphs, containing $M = O(M')$ Type-1 operations. (The proof is provided in Appendix F.4.)*

Applying the Type-2 to Type-1 transformation and committing to the concatenated matrix of the new leaf nodes splits the original DAG into J Type-1 expressions. Thus, the language $\mathcal{L}_{\text{main}}$ is transformed to language $\mathcal{L}_{\text{type1}}$ below:

$$\{(J, n, k), (\{f_j \in \mathcal{E}_1\}_{j \in [J]}, \{\mathbf{a}_i \in \mathbb{F}^{n \times n}\}_{i \in [k]}) : \forall j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}\}, \quad (4)$$

where $\mathcal{E}_1$ represents the set of all Type-1 expressions. We next give the reduction algorithm for $\mathcal{E}_1$; Algorithm 2 (Section 7.2) specifies the traversal and its outputs.

7.2 Proof Reduction for Type-1 Expression through DAG Traversal

Starting at the output matrix, we recursively reduce projections to the input matrices, *i.e.*, a pre-order traversal of the DAG for a general matrix expression. Before proof generation, the prover evaluates the DAG in post-order to compute all node values; these become the inputs to the pre-order traversal.

We organize these traversals as Algorithm 2, which outputs the reduction transcript $\vec{\pi}_{\text{reduce}}$, the single batched leaf projection v , and the projecting vectors $\vec{l}$ and $\vec{r}$ that Section 8 later opens from the leaf commitment.

An Evalyn proof for a Type-1 operation $\otimes$ consists of $(\otimes.\text{Prove}, \otimes.\text{Eval}, \otimes.\text{Link}, \otimes.\text{Verify})$. $\otimes.\text{Eval}$ and $\otimes.\text{Link}$ reduce the projection of an output matrix to those of the input matrices. During a pre-order traversal, these algorithms are applied recursively to reduce the projection at the root into projections at the leaves.

By a variant of the Schwartz-Zippel lemma (Corollary 2, Appendix B.3), we have $f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}$ iff $\vec{l}^\top f(\mathbf{a}_1, \dots, \mathbf{a}_k) \vec{r} = 0$ for random tensor-structured

$\vec{l}$ and $\vec{r}$. During the pre-order DAG traversal, at each node (denoted by `node`¹⁰) with operation $\otimes$, the prover runs $\otimes.\text{Prove}$ to generate a proof π for the projection of the current `node`. Then $\otimes.\text{Eval}$ derives the child projections v_L and v_R from π , and $\otimes.\text{Link}$ maps $(\vec{l}, \vec{r})$ and the random challenges at `node` to the projecting vectors $(\vec{l}_L, \vec{r}_L)$ and $(\vec{l}_R, \vec{r}_R)$ for `node.leftChild` and `node.rightChild`. These values are stored in the respective child nodes and used when the traversal reaches them. The verifier checks the proof π at each node and verifies the projections at all leaf nodes. This proof reduction process is detailed in Algorithm 2.

If a node has multiple parents, the prover applies the **BatchProj** proof (Section 4.2) to consolidate multiple projections into a single projection (Line 10, Algorithm 2). At the end, the prover employs the **ConcatProj** proof (Section 4.2) to merge projections from all leaf nodes into a single projection of a concatenated matrix (Line 25), which is proven using the $\text{Prove}_{\text{open}}$ algorithm of the matrix commitment scheme, achieving logarithmic proof size and verifier time.

The prover traverses the DAG in pre-order and invokes the atomic proofs from Sections 4 and 5 at each node. As atomic Evalyn proofs instantiate RoK, a DAG of such reductions is sound.

Theorem 2. *Algorithm 2 is a proof for $\mathcal{L}_{\text{type1}}$ with a soundness error at most $\frac{10(M+k)\log_2 n}{p-2}$, where M is the total number of operations in the DAG, k is the number of leaves in the DAG, and $n \times n$ is the maximum matrix dimension.*

The proof is in Appendix F.5. Algorithm 2 can be converted into a non-interactive protocol by Fiat–Shamir transform. The non-interactive interface is:

- $\text{Prove}((J, n, k), (\{f_j\}_{j \in [J]}, \{a_i\}_{i \in [k]})) \rightarrow \vec{\pi}_{\text{reduce}}$: Produces the reduction proof $\vec{\pi}_{\text{reduce}}$ ending with the leaf-node projection v and projecting vectors $\vec{l}$ and $\vec{r}$;
- $\text{Verify}(\vec{\pi}_{\text{reduce}}, (J, n, k), (\{f_j\}_{j \in [J]}, \{a_i\}_{i \in [k]})) \rightarrow \text{true/false}$: Verifies $\vec{\pi}_{\text{reduce}}$.

Here, the verifier is assumed to know both the DAG structure $\{f_j\}_{j \in [J]}$ and the leaf-node values $\{a_i\}_{i \in [k]}$. To retain privacy, we hide the structural information using PoP: rather than allowing the verifier to check $\vec{\pi}_{\text{reduce}}$ directly, the verifier’s algorithm is compiled into a PoP circuit, with only the commitment to $\vec{\pi}_{\text{reduce}}$ sent to the verifier. To hide the leaf nodes, we commit to them and require the verifier to verify opening of a matrix commitment to verify the leaf-node projection in zero knowledge.

8 End-to-End Proof System: Commit-and-Prove and PoP

8.1 Commit and Prove with Binding Commitment

So far, the protocols are proofs with soundness rather than AoKs with knowledge soundness. In zkSNARKs, it is common to convert a proof into an AoK using

¹⁰ We represent each DAG node by a `Node` struct with fields: `op` (the type-1 operation of the node); `leftChild` (pointer to the left child, or `null` if the node is a leaf); `rightChild` (pointer to the right child, or `null` for leaves or unary `op`); `mat` (the matrix stored at the node); `v` (projection values propagated from all parents); `lvec` (projection vectors $\vec{l}$ received from all parents); `rvec` (projection vectors $\vec{r}$ received from all parents).

Algorithm 2. Proof for $f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}, \forall j \in [J]$

Output: $\mathcal{V}$: true or false; $\mathcal{P}$: $\vec{\pi}_{\text{reduce}} := \left(\{\pi_u^{\text{batch}}\}_{u \in \mathcal{N}_{\text{multi}}}, \{\pi_u^{\text{op}}\}_{u \in \mathcal{N}_{\text{int}}}, \pi_{\text{concat}} \right)$
 $(\mathcal{N}_{\text{int}} := \{u \in \text{DAG} : u \text{ is an internal node}\}, \mathcal{N}_{\text{multi}} := \{u \in \text{DAG} : |u.v| > 1\})$;

- 1 $\mathcal{P}$ performs a post-order traversal of the DAG of $\{f_j\}_{j \in [J]}$ to compute the matrix value `node.mat` for every node;
- 2 $\mathcal{P}$ and $\mathcal{V}$ perform a pre-order traversal of the DAG to verify projections:
- 3 $\mathcal{V} \rightarrow \mathcal{P}$: random vectors $\mathbf{x}_l \xleftarrow{\$} \mathbb{F}^{\log_2 n}$; $\mathbf{x}_r \xleftarrow{\$} \mathbb{F}^{\log_2 n}$;
- 4 $\mathcal{P}$ and $\mathcal{V}$ compute tensor spans $\vec{l} \leftarrow \text{ts}(\mathbf{x}_l) \in \mathbb{F}^n$; $\vec{r} \leftarrow \text{ts}(\mathbf{x}_r) \in \mathbb{F}^n$;
- 5 **foreach** `root` $\in$ DAG **do**
- 6 `root.v` $\leftarrow [0,]$; `root.lvec` $\leftarrow [\vec{l},]$; `root.rvec` $\leftarrow [\vec{r},]$;
- 7 **foreach** `node` $\in$ DAG **do**
- 8 **if** `|node.v|` > 1 **then**
- 9 $(\text{ok}, \pi_{\text{node}}^{\text{batch}}, v, \vec{l}, \vec{r}) \leftarrow \text{BatchProj}(\text{node.v}, \text{node.lvec}, \text{node.rvec}, \text{node.mat})$;
- 10 **if** $\neg \text{ok}$ **then return false**;
- 11 $(v, \vec{l}, \vec{r}) \leftarrow (\text{node.v}, \text{node.lvec}, \text{node.rvec})$;
- 12 **if** `node.op` is a binary operation **then**
- 13 $\mathcal{P}$: $\pi_{\text{node}}^{\text{op}} \leftarrow \otimes.\text{Prove}(v, \vec{l}, \vec{r}; \text{node.leftChild.mat}, \text{node.rightChild.mat})$;
- 14 $\mathcal{P}$ and $\mathcal{V}$ run $(v_L, v_R) \leftarrow \otimes.\text{Eval}(\pi_{\text{node}}^{\text{op}})$;
- 15 $\mathcal{P}$ and $\mathcal{V}$ run $(\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R) \leftarrow \otimes.\text{Link}(\vec{l}, \vec{r}, \pi_{\text{node}}^{\text{op}})$;
- 16 $\mathcal{V}$: **if** $\neg \otimes.\text{Verify}(v, \pi_{\text{node}}^{\text{op}}, v_L, v_R)$ **then return false**;
- 17 `node.leftChild.v.append`(v_L); `node.rightChild.v.append`(v_R);
- 18 `node.leftChild.lvec.append`($\vec{l}_L$); `node.leftChild.rvec.append`($\vec{r}_L$);
- 19 `node.rightChild.lvec.append`($\vec{l}_R$); `node.rightChild.rvec.append`($\vec{r}_R$);
- 20 **else if** `node.op` is a unary operation **then**
- 21 follow a process similar to binary operations (details omitted);
- 22 **foreach** $i \in [k]$ **do**
- 23 `leaf` $\leftarrow$ the i -th leaf node in DAG;
- 24 $\mathbf{a}_i \leftarrow \text{leaf.mat}$; $v_i \leftarrow \text{leaf.v}$; $\vec{l}_i \leftarrow \text{leaf.lvec}$; $\vec{r}_i \leftarrow \text{leaf.rvec}$;
- 25 $\mathcal{P}$ and $\mathcal{V}$ run $(\text{ok}, \pi_{\text{concat}}, v, \vec{l}, \vec{r}) \leftarrow \text{ConcatProj}(\{(v_i, \vec{l}_i, \vec{r}_i, \mathbf{a}_i)\}_{i \in [k]})$ to
 produce and verify the proof π_{concat} that batch verifies $v_i = \vec{l}_i^\top \mathbf{a}_i \vec{r}_i$;
- 26 **if** $\neg \text{ok}$ **then return false**;
- 27 $\mathcal{V}$ checks: $v = \vec{l}^\top (\mathbf{a}_1 \parallel \dots \parallel \mathbf{a}_k) \vec{r}$;
- 28 **return true** iff all of the above return true;

the commit-and-prove technique. For example, in the prevalent sumcheck+PCS paradigm, the sumcheck reduces verification to polynomial evaluations at random points, then a PCS verifies these evaluations, which yields an AoK. We extend this method and explore the knowledge soundness error.

Consider a relation $\mathcal{R}$ over an input u , a witness w , where u and w satisfy a constraint $\text{Constraint}(u, w)$, and w is committed by a commitment scheme Comm :

$$\mathcal{R} = \{(u, C_w) : w \mid \text{Constraint}(u, w) = \text{true} \wedge \text{Comm}(w) = C_w\}. \quad (5)$$

Let $\Pi_{\text{proof}}(u, w)$ be a proof system for the following $\mathcal{R}$ -derived language $\mathcal{L}(\mathcal{R})$:

$$\mathcal{L}(\mathcal{R}) = \{(u, w) : \text{Constraint}(u, w) = \text{true}\}. \quad (6)$$

Let $F_v(w) = \vec{l}^\top (\mathbf{a}_1 \| \dots \| \mathbf{a}_k) \vec{r}$ be the projection mapping, where w represents the leaf-node matrix values $(\mathbf{a}_1 \| \dots \| \mathbf{a}_k)$ as in the check at Line 27 of Algorithm 2. Given challenge $(\vec{l}, \vec{r})$, $F_v()$ is deterministic. This check can alternatively be verified without revealing the leaf-node values by opening a matrix commitment in Section 3.3. In abstract form, we define the opening relation as:

$$\mathcal{R}_{\text{open}} = \{v, F_v, C_w : w \mid v = F_v(w) \wedge C_w = \text{Comm}(w)\}. \quad (7)$$

The composite protocol of $\Pi_{\text{proof}}(u, w)$ and $\Pi_{\text{open}}(v, F_v, C_w, w)$ serves as an AoK for $\mathcal{R}$. The following theorem formalizes the commit-and-prove paradigm.

Theorem 3 (Knowledge Soundness with Commitment). *Provided:*

- A commitment scheme Comm for w with binding advantage κ_{com} ;
- A proof $\Pi_{\text{proof}}(u, w)$ for $\mathcal{L}(\mathcal{R})$ with computational soundness error $\kappa_{\text{proof}}^{\text{cs}}$;
- A fixed public deterministic mapping F_v ;
- An AoK $\Pi_{\text{open}}(v, F_v, C_w, w)$ for $\mathcal{R}_{\text{open}}$ with knowledge soundness error $\kappa_{\text{open}}^{\text{ks}}$.

(When the knowledge soundness error can be statement-dependent, we write $\kappa_{\text{proof}}^{\text{cs}}(u)$ or $\kappa_{\text{open}}^{\text{ks}}(u)$ instead.) The following interactive protocol Π functions as an AoK for $\mathcal{R}$ in Eq. (5):

- The parties run $\Pi_{\text{proof}}(u, w)$ to produce a proof π_{proof} for $\mathcal{L}(\mathcal{R})$ (Eq. (6)), with v included in π_{proof} ;
- The parties engage in $\Pi_{\text{open}}(v, F_v, C_w, w)$ to produce a proof π_{open} for $\mathcal{R}_{\text{open}}$ (Eq. (7)), where v is the public auxiliary value used by Π_{proof} for input u ;
- Given $\pi := (\pi_{\text{proof}}, \pi_{\text{open}})$ for $\mathcal{R}$, $\mathcal{V}$ returns **true** if both of $\mathcal{V}_{\text{proof}}(u, \pi_{\text{proof}})$ and $\mathcal{V}_{\text{open}}(v, F_v, C_w, \pi_{\text{open}})$ return **true**.

Its knowledge soundness error for $\mathcal{R}$ is $\kappa^{\text{ks}}(u) = \kappa_{\text{proof}}^{\text{cs}}(u) + 2\kappa_{\text{open}}^{\text{ks}}(u) + \kappa_{\text{com}}$. (The proof of this theorem is provided in Appendix F.6.)

An AoK for $\mathcal{R}_{\text{main}}$ in Eq. (1) is derived by combining the proof from Section 7 for $\mathcal{L}_{\text{main}}$ in Eq. (2) with the $\text{Prove}_{\text{open}}$ algorithm of a matrix commitment scheme.

We convert the protocol in the preceding theorem to a non-interactive argument using the adaptive Fiat–Shamir transformation (Appendix B.8), which relies on a correlation-intractable hash function [27,28]. Our choice of Poseidon [46] is heuristically assumed to be correlation intractable against low-degree relations, such as the quadratic relation $c = \text{Hash}(c \| \iota \| \rho) \cdot \iota + \text{Hash}(c \| \iota \| \rho)^{-1} \cdot \rho$ in LiteBullet. This forces adversaries to attack the transformation via brute-force queries. See Lemmas 9 and 10 for more details. A deeper analysis of correlation intractability is beyond our scope.

Corollary 1 (Non-Interactive Version of Theorem 3). *Let Comm be a binding commitment scheme Comm with binding advantage $\kappa_{\text{com}}(u)$, Π_{proof} a round-by-round sound interactive proof for $\mathcal{L}(\mathcal{R})$ with soundness error $\kappa_{\text{proof}}^{\text{cs}}(u)$, and Π_{open} a special-sound AoK protocol for $\mathcal{R}_{\text{open}}$ with soundness error $\kappa_{\text{open}}^{\text{ks}}(u)$. Fiat-Shamir transformation yields a non-interactive AoK for $\mathcal{R}$ with knowledge soundness error $\kappa^{\text{ks-fs}}(u) \leq (Q + 1)(\kappa_{\text{proof}}^{\text{cs}}(u) + 2\kappa_{\text{open}}^{\text{ks}}(u)) + \kappa_{\text{com}}(u)$ for any adversary making at most Q queries to the random oracle modeling the hash.*

8.2 Proof of Proof for Succinctness and Zero Knowledge

Our proof reduction (Section 7.2) emits M atomic Evalyn proofs for a DAG with M Type-1 operations, yielding $O(M)$ proof size and verifier time. Furthermore, these atomic Evalyn proofs and the composite proof are not zero knowledge since LiteBullet is not. To compile our system into a zkSNARK protocol, a proof-of-proof layer is added to prove this reduction *inside* a zkSNARK. For every Type-1 operations $\otimes$, our $\otimes.\text{Verify}$ algorithm (Section 5) uses only field and hash operations, enabling a small PoP circuit that can be efficiently encoded by the rank-1 constraint system (R1CS) [7]. We now specify the $\text{Verify}_{\text{reduce}}$ circuit, its R1CS form, and the batch linkage of all Fiat-Shamir challenges.

Consider a transcript $\vec{\pi}_{\text{reduce}} \in \mathbb{F}^{(2\mu+1)}$ of length $2\mu + 1$ from a μ -round proof reduction process in Algorithm 2 (where $\mu = O(M \log n)$) as follows:

$$\vec{\pi}_{\text{reduce}} := (u, \alpha_0, x_1, \alpha_1, x_2, \dots, \alpha_{\mu-1}, x_\mu, \alpha_\mu).$$

Here, u represents a hash value of f and C_a in $\mathcal{R}_{\text{main}}$, which is known by both parties, $\alpha_{i \in [0, \mu]}$ represent the prover's messages, and $x_1, \dots, x_\mu$ represent the Fiat-Shamir challenges computed by the adaptive Fiat-Shamir transformation (Definition 23, Appendix B.8). Without loss of generality, we assume each α_i contains 2 field elements, as in the LiteBullet proof.

At the conclusion of Algorithm 2 (Line 27), the prover sends the projection value v of the concatenated leaf node matrix and tensor-structured vectors $\vec{l} = \text{ts}(\vec{x}_l)$ and $\vec{r} = \text{ts}(\vec{x}_r)$. $\vec{x}_l$ and $\vec{x}_r$ must align with $\vec{\pi}_{\text{reduce}}$, recursively derived from the Fiat-Shamir challenges $x_1, \dots, x_\mu$ by the Link method of each atomic operation, and v is the last message of $\vec{\pi}_{\text{reduce}}$. These constraints are verified by a function $\text{Verify}_{\text{link}}(\vec{\pi}_{\text{reduce}}, v, \mathbf{x}_l, \mathbf{x}_r) = \text{true}$. Meanwhile, we define $\text{Verify}_{\text{reduce}}(\vec{\pi}_{\text{reduce}}, v, \mathbf{x}_l, \mathbf{x}_r)$ to run the Verify algorithm for M atomic operations:

$$\text{return } \text{Verify}_1(\vec{\pi}_{\text{reduce}}) \wedge \dots \wedge \text{Verify}_M(\vec{\pi}_{\text{reduce}}) \wedge \text{Verify}_{\text{link}}(\vec{\pi}_{\text{reduce}}, v, \mathbf{x}_l, \mathbf{x}_r).$$

Both $\text{Verify}_{\text{link}}$ and $\text{Verify}_{\text{reduce}}$ are compiled into a single arithmetic circuit that returns true when iff both return true.

As shown in Fig. 2, the PoP circuit preserves the topology of the original matrix DAG, mapping each matrix to a scalar. The PoP circuit size is $O(\mu) = O(M \log n)$, sublinear in the total number of matrix entries Mn^2 . The resulting

Circ-SAT instance is efficiently encoded as an R1CS equation (Appendix E.4):

$$\begin{bmatrix} \mathbf{a}_L^{\text{verify}} \\ v \\ \vec{x}_l \\ \vec{x}_r \\ \vec{w}_{\text{verify}} \end{bmatrix} \circ \begin{bmatrix} \vec{\pi}_{\text{reduce}} \\ v \\ \vec{x}_l \\ \vec{x}_r \\ \vec{w}_{\text{verify}} \end{bmatrix} = \begin{bmatrix} \vec{\pi}_{\text{reduce}} \\ v \\ \vec{x}_l \\ \vec{x}_r \\ \vec{w}_{\text{verify}} \end{bmatrix}. \quad (8)$$

All gate values in the PoP circuit are collected in $\vec{w}_{\text{verify}} \in \mathbb{F}^{O(\mu)}$.

Commitment to DAG Architecture. R1CS matrices $\{\mathbf{a}_i^{\text{verify}}\}_{i \in \{L, R, O\}} \in \mathbb{F}^{O(\mu) \times O(\mu)}$, which define the matrix expression f , allow for the recovery of the verifier's circuit. The PoP circuit is partitioned into subgraphs, each representing the verifier for an atomic operation (Fig. 2(b)). The subgraph structure determines the operation type, while their intersections encode dependencies between operations, making the original DAG architecture fully reconstructible (Fig. 2(a)). Since the DAG in turn uniquely determines the PoP circuit, this establishes a one-to-one correspondence. Committing to the R1CS matrices of the PoP circuit provides a hiding commitment to the DAG architecture.

Batch Proof for Fiat-Shamir Challenges. The PoP layer also batch proves the Fiat-Shamir challenges, which, by the adaptive Fiat-Shamir transformation (Definition 23, Appendix B.8), must satisfy the following with $x_0 := u$:

$$x_i = \text{Hash}(x_{i-1}, \alpha_{i-1}), \quad i \in [\mu].$$

Let the R1CS matrices for a single hash function be $\{\mathbf{a}_i^{\text{hash}}\}_{i \in \{L, R, O\}} \in \mathbb{F}^{\eta \times \eta}$, where η represents the number of multiplication gates in the hash function (*e.g.*, for the Poseidon hash [46], $\eta \approx 2^8$). Then, the i -th hash function is proven using:

$$\begin{bmatrix} \mathbf{a}_L^{\text{hash}} \\ x_{i-1} \\ \alpha_{i-1} \\ x_i \\ \vec{w}_i \end{bmatrix} \circ \begin{bmatrix} \vec{\pi}_{\text{reduce}} \\ x_{i-1} \\ \alpha_{i-1} \\ x_i \\ \vec{w}_i \end{bmatrix} = \begin{bmatrix} \vec{\pi}_{\text{reduce}} \\ x_{i-1} \\ \alpha_{i-1} \\ x_i \\ \vec{w}_i \end{bmatrix}, \quad i \in [\mu].$$

In the i -th hash circuit, $\vec{w}_i$ represents its gate values. These equations are consolidated into a single equation by defining matrices $\mathbf{h}$ and $\mathbf{w}_{\text{hash}}$ as follows:

$$\mathbf{h} := \begin{pmatrix} u & x_1 & x_2 & \cdots & x_{\mu-1} \\ \alpha_0 & \alpha_1 & \alpha_2 & \cdots & \alpha_{\mu-1} \\ x_1 & x_2 & x_3 & \cdots & x_\mu \end{pmatrix} \in \mathbb{F}^{3 \times \mu}, \quad \mathbf{w}_{\text{hash}} = (\vec{w}_1 \ \vec{w}_2 \ \cdots \ \vec{w}_\mu) \in \mathbb{F}^{\eta \times \mu}.$$

Then, stacking μ hashes can be verified using a single matrix equation:

$$\begin{bmatrix} \mathbf{a}_L^{\text{hash}} \\ \mathbf{h} \\ \mathbf{w}_{\text{hash}} \end{bmatrix} \circ \begin{bmatrix} \vec{\pi}_{\text{reduce}} \\ \mathbf{h} \\ \mathbf{w}_{\text{hash}} \end{bmatrix} = \begin{bmatrix} \vec{\pi}_{\text{reduce}} \\ \mathbf{h} \\ \mathbf{w}_{\text{hash}} \end{bmatrix}. \quad (9)$$

R1CS matrices $\{\mathbf{a}_i^{\text{hash}}\}_{i \in \{L, R, O\}}$ here are independent of μ , the number of hashes. For Poseidon hash, which requires $\sim 2^8$ arithmetic gates, each R1CS matrix has

size $\sim 2^8 \times 2^8$. In large NNs with $M \simeq 2^{10}$ and $n \simeq 2^{10}$, $\mathbf{w}_{\text{hash}}$ has size $\sim 2^{17} \times 2^8$, which remains moderate compared to the original problem scale $Mn^2 \simeq 2^{30}$.

Furthermore, since all entries of $\mathbf{h}$ reside within $\vec{\pi}_{\text{reduce}}$, we introduce a Boolean matrix $\mathbf{b} \in \{0, 1\}^{(3\mu) \times (2\mu+1)}$ to relate the vectorized $\text{vec}(\mathbf{h})$ to $\vec{\pi}_{\text{reduce}}$:

$$\text{vec}(\mathbf{h}) = \mathbf{b}\vec{\pi}_{\text{reduce}}. \quad (10)$$

The PoP Protocol, with private inputs $(\vec{\pi}_{\text{reduce}}, v)$ and public inputs:

- $C_b = \text{Comm}(\mathbf{b}), C_\pi, C_v, \mathbf{x}_l, \mathbf{x}_r,$
 $\{C_i^{\text{hash}} = \text{Comm}(\mathbf{a}_i^{\text{hash}}), C_i^{\text{verify}} = \text{Comm}(\mathbf{a}_i^{\text{verify}})\}_{i \in \{L, R, O\}}$
- $\mathcal{P}$ computes $\vec{\mathbf{w}}_{\text{verify}}$ and $\mathbf{w}_{\text{hash}}$; $\mathcal{P} \rightarrow \mathcal{V}$: $C_w^v \leftarrow \text{Comm}(\vec{\mathbf{w}}_{\text{verify}}); C_w^h \leftarrow \text{Comm}(\mathbf{w}_{\text{hash}})$;
 - $\mathcal{P}$ and $\mathcal{V}$ run Algorithm 2 with zero-knowledge transformation to reduce the verification into checking projections of $\vec{\pi}_{\text{reduce}}, \vec{\mathbf{w}}_{\text{verify}}, \mathbf{w}_{\text{hash}}, \mathbf{b}, \{\mathbf{a}_i^{\text{verify}}, \mathbf{a}_i^{\text{hash}}\}_{i \in \{L, R, O\}}$.
 - $\mathcal{P}$ runs $\text{Prove}_{\text{open}}$ of matrix commitment to let $\mathcal{V}$ verify the above projections.

Together, zkSNARK proofs for Eqs. (8)–(10) constitute the PoP layer. In this PoP layer, Evalyn proofs are employed for the matrix equations. To achieve zero knowledge, transcript elements are homomorphically committed using the zero-knowledge transformations [78,90].

Claim 4. *The PoP protocol is a zero-knowledge AoK for relation $\mathcal{R}_{\text{pop}}$ below:*

$$\left\{ \left(\begin{array}{l} C_b, C_\pi, C_v, \mathbf{x}_l, \mathbf{x}_r, \\ \{C_i^{\text{hash}}\}_{i \in \{L, R, O\}}, \\ \{C_i^{\text{verify}}\}_{i \in \{L, R, O\}} \end{array} : (\vec{\pi}_{\text{reduce}}, v) \right) \left| \begin{array}{l} \text{Verify}_{\text{reduce}}(\vec{\pi}_{\text{reduce}}, v, \mathbf{x}_l, \mathbf{x}_r) = \text{true} \wedge \\ C_\pi = \text{Comm}(\vec{\pi}_{\text{reduce}}) \wedge C_v = \text{Comm}(v) \wedge \\ \text{Fiat-Shamir challenge matches} \end{array} \right. \right\}.$$

(The proof is provided in Appendix F.9.)

The non-interactive PoP protocol has the following interface:

- $\text{Prove}((C_b, C_\pi, C_v, \mathbf{x}_l, \mathbf{x}_r, \{C_i^{\text{hash}}, C_i^{\text{verify}}\}_{i \in \{L, R, O\}}), (\vec{\pi}_{\text{reduce}}, v)) \rightarrow \pi_{\text{pop}}$: Taking the commitments and the prior proof $\vec{\pi}_{\text{reduce}}$, the prover runs this algorithm with the internal randomness used (*e.g.*, commitment opening) to produce a logarithmic-size proof π_{pop} .
- $\text{Verify}(\pi_{\text{pop}}, (C_\pi, C_v, \mathbf{x}_l, \mathbf{x}_r, \{C_i^{\text{hash}}, C_i^{\text{verify}}\}_{i \in \{L, R, O\}}, C_b)) \rightarrow \text{true/false}$: This verifies the proof without knowing $\vec{\pi}_{\text{reduce}}$ or v .

9 Putting Everything Together

Algorithm 3 constructs a zkSNARK for $\mathcal{R}_{\text{main}}$ with our developed techniques. The prover first converts Type-2 operations into Type-1 expressions and commits to the new leaf nodes $(\mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{k'})$ of the transformed DAG into C_{type2} . The commitment to $(\mathbf{a}_1 \parallel \dots \parallel \mathbf{a}_k \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{k'})$ is derived from $C_a = \text{Comm}(\mathbf{a}_1 \parallel \dots \parallel \mathbf{a}_k)$ and C_{type2} . The prover then traverses the DAG and reduces verification to one

Algorithm 3. zkSNARK for $f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}$

Input: Public input: $k, n \in \mathbb{N}; C_a, C_b, \{C_i^{\text{verify}}\}_{i \in \{L, R, O\}}, \{C_i^{\text{hash}}\}_{i \in \{L, R, O\}} \in \mathbb{G}$;
 $\mathcal{P}$'s private input: $f \in \mathcal{E}, \{\mathbf{a}_i\}_{i \in [k]} \in \mathbb{F}^{n \times n}$

Output: $\mathcal{P}$: $\pi := (C_{\text{type2}}, C_\pi, C_v, \pi_{\text{PoP}}, \pi_{\text{open}})$; $\mathcal{V}$: **true** or **false**

- 1 $\mathcal{P}$ transforms f into a Type-1 expression $f' \in \mathcal{E}_1$ and commits to k' new leaf nodes $(\mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{k'})$ into $C_{\text{type2}} \in \mathbb{G}$;
- 2 $\mathcal{P} \rightarrow \mathcal{V}$: $C_{\text{type2}} \in \mathbb{G}$;
- 3 $\mathcal{P}$ runs Algorithm 2 to produce $\tilde{\pi}_{\text{reduce}}$, the leaf-node projection value v , and projecting vectors $\vec{l} = \text{ts}(\mathbf{x}_l)$ and $\vec{r} = \text{ts}(\mathbf{x}_r)$ for $(\mathbf{a}_1 \parallel \dots \parallel \mathbf{a}_k \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{k'})$;
- 4 $\mathcal{P} \rightarrow \mathcal{V}$: $(C_\pi, C_v, \vec{x}_l, \vec{x}_r)$ after running $C_\pi \leftarrow \text{Comm}(\tilde{\pi}_{\text{reduce}})$; $C_v \leftarrow \text{Comm}(v)$;
- 5 $\mathcal{P}$ and $\mathcal{V}$ run the PoP protocol on public input $(C_b, C_\pi, C_v, \mathbf{x}_l, \mathbf{x}_r, \{C_i^{\text{hash}}, C_i^{\text{verify}}\}_{i \in \{L, R, O\}})$ and private input $(\tilde{\pi}_{\text{reduce}}, v)$;
- 6 $\mathcal{P}$ and $\mathcal{V}$ run the $\text{Prove}_{\text{open}}$ and $\text{Verify}_{\text{open}}$ algorithms of the matrix commitment scheme to assert proof π_{open} that $v = \vec{l}^\top (\mathbf{a}_1 \parallel \dots \parallel \mathbf{a}_k \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{k'}) \vec{r}$;
- 7 **return true** iff all of the above return **true**;

scalar projection v with projecting vectors $\vec{l} = \text{ts}(\mathbf{x}_l)$ and $\vec{r} = \text{ts}(\mathbf{x}_r)$. The commitment C_π to the proof reduction transcript π_{reduce} is sent to the verifier. Subsequently, the parties run the PoP protocol to validate $\tilde{\pi}_{\text{reduce}}$, v , $\mathbf{x}_l$, and $\mathbf{x}_r$. Finally, the $\text{Verify}_{\text{open}}$ algorithm of the matrix commitment scheme can verify the correctness of the projection v .

The non-interactive version of Algorithm 3 has the following interface:

- $\text{Prove}((k, n), (C_a, C_b, \{C_i^{\text{hash}}, C_i^{\text{verify}}\}_{i \in \{L, R, O\}}), (f, \{\mathbf{a}_i\}_{i \in [k]})) \rightarrow \pi$: Output a proof π for $f(\mathbf{a}_1, \dots, \mathbf{a}_k) = \mathbf{0}$, using private input $(f, \{\mathbf{a}_i\}_{i \in [k]})$.
- $\text{Verify}(\pi, (k, n), C_a, C_b, \{C_i^{\text{hash}}, C_i^{\text{verify}}\}_{i \in \{L, R, O\}})) \rightarrow \text{true/false}$ Verify the succinct proof π in zero knowledge, given both the PoP-circuit commitment and the leaf-node commitment.

Theorem 4. *The non-interactive version of Algorithm 3 is a zkSNARK for $\mathcal{R}_{\text{main}}$ in Eq. (1), with knowledge soundness error:*

$$\kappa^{\text{ks-fs}_{\text{overall}}} = (Q + 1) \cdot (\kappa_{\text{reduce}}^{\text{cs}} + 2\kappa_{\text{open}}^{\text{ks}} + \kappa_{\text{pop-reduce}}^{\text{cs}} + 2\kappa_{\text{pop-open}}^{\text{ks}}) + 3\kappa_{\text{com}}.$$

where $\kappa_{\text{reduce}}^{\text{cs}}$, $\kappa_{\text{open}}^{\text{ks}}$, $\kappa_{\text{pop-reduce}}^{\text{cs}}$, and $\kappa_{\text{pop-open}}^{\text{ks}}$, represent the computational soundness error of the proof reduction, the knowledge soundness error of the $\text{Prove}_{\text{open}}$ algorithm, the computational soundness error of the proof reduction in the PoP protocol, and the knowledge soundness error of the $\text{Prove}_{\text{open}}$ algorithm in the PoP protocol, respectively. κ_{com} represents the binding advantage of the commitment scheme. Q indicates the maximum number of queries the prover can make to the random oracle. (The proof is provided in Appendix F.8.)

The overall complexity of our approach depends on the matrix commitment. We adopt Dory [59] (see Appendix E.2), which suits large, low-bitwidth matrices and vectors. For an m -vector, its $\text{Prove}_{\text{open}}$ algorithm uses $O(\sqrt{m})$ pairings (a

dominating prover cost for typical values of m like $\sim 2^{30}$) and $m + O(\sqrt{m})$ field operations, while both proof size and verifier time are $O(\log m)$.

For a matrix expression involving M atomic operations and $k \leq 2M$ leaf nodes on $n \times n$ matrices, the proof reduction and the $\text{Prove}_{\text{open}}$ algorithm both take $O(Mn^2)$ prover time. The PoP prover time is $O(\log(Mn^2))$, so the total prover time is $O(Mn^2)$. Proof sizes for the $\text{Prove}_{\text{open}}$ algorithm and the PoP are $O(\log(Mn))$ and $O(\log(M \log n))$, respectively, culminating in a total proof size of $O(\log(Mn))$. Verifier time is similarly $O(\log(Mn))$.

One can replace Dory with different multilinear PCS schemes [91] for different tradeoffs. Using a multilinear PCS with constant proof size and verifier time, such as Samaritan [39], our scheme can reach $O(1)$ proof size and verifier time as well. However, Samaritan exhibits higher concrete prover time than Dory, despite both having the same asymptotic prover complexity.

Our empirical results (Appendix G) confirm the above complexities.

10 Conclusion

Shaping new directions in verifiable machine learning, we present *Evalyn*, a proof composition framework that gives a single matrix-native interface within a simple, modular computation model. Hiding the neural network architecture is achieved by our directed acyclic graph formulation, and the proof-of-proof layer supplies succinctness and zero knowledge without exposing internal topology. Efficiency follows with strictly linear prover time $O(Mn^2)$ for M operations on $n \times n$ matrices, and with proof size and verifier time $O(\log(Mn))$, making *Evalyn* the first system to achieve such efficiency while preserving architecture privacy. Reframing inner product arguments, *LiteBullet* provides a field-only inner product proof without invoking polynomial encodings. Mapping non-linear operations to matrix expressions is done by table lookups, placing non-linear layers such as activations on the same footing as linear layers. Across end-to-end pipelines, our modular design enables composition, batching, and proof reuse. New extensions to improve deployability include scalable SNARKs for cryptographic neural-network computation [67], specialized hardware co-design [66], model-compiler integration with *Evalyn*, and configurable per-layer bit widths.

Acknowledgement

The first two authors acknowledge the use of generative AI tools for language revision. No scientific material in this paper was originated from AI.

Sherman Chow is supported in part by the General Research Fund (CUHK 14210825) from the Research Grants Council, University Grants Committee, and by the Direct Grant (4055238), and the Strategic Impact Enhancement Fund (3135517) of The Chinese University of Hong Kong.

References

1. Abbaszadeh, K., Pappas, C., Katz, J., Papadopoulos, D.: Zero-knowledge proofs of training for deep neural networks. In: CCS. pp. 4316–4330 (2024)
2. Abe, M., Fuchsbauer, G., Groth, J., Haralambiev, K., Ohkubo, M.: Structure-preserving signatures and commitments to group elements. In: CRYPTO. pp. 209–236 (2010)
3. Ames, S., Hazay, C., Ishai, Y., Venkatasubramanian, M.: Liger: Lightweight sub-linear arguments without a trusted setup. *Des. Codes Cryptogr.* **91**(11), 3379–3424 (2023)
4. Attema, T., Cramer, R., Kohl, L.: A compressed Σ -protocol theory for lattices. In: CRYPTO Part II. pp. 549–579 (2021)
5. Attema, T., Fehr, S., Klooß, M.: Fiat-Shamir transformation of multi-round interactive proofs (extended version). *J. Cryptol.* **36**(4), 36 (2023)
6. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch.* 2018/046 (2018)
7. Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E., Virza, M.: SNARKs for C: Verifying program executions succinctly and in zero knowledge. In: CRYPTO. pp. 90–108 (2013)
8. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for R1CS. In: EUROCRYPT Part I. pp. 103–128 (2019)
9. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive oracle proofs. In: TCC (B2). pp. 31–60 (2016)
10. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. In: CRYPTO Part II. pp. 276–294 (2014)
11. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In: ITCS. pp. 326–349 (2012)
12. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: Recursive composition and bootstrapping for SNARKS and proof-carrying data. In: STOC. pp. 111–120 (2013)
13. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: EUROCRYPT Part II. pp. 327–357 (2016)
14. Bootle, J., Chiesa, A., Groth, J.: Linear-time arguments with sublinear verification from tensor codes. In: TCC Part II. pp. 19–46 (2020)
15. Bootle, J., Chiesa, A., Hu, Y., Orrù, M.: Gemini: Elastic SNARKs for diverse environments. In: EUROCRYPT Part II. pp. 427–457 (2022)
16. Bootle, J., Chiesa, A., Liu, S.: Zero-knowledge IOPs with linear-time prover and polylogarithmic-time verifier. In: EUROCRYPT Part II. pp. 275–304 (2022)
17. Bootle, J., Chiesa, A., Sotiraki, K.: Sumcheck arguments and their applications. In: CRYPTO Part I. pp. 742–773 (2021)
18. Bootle, J., Lyubashevsky, V., Nguyen, N.K., Seiler, G.: A non-PCP approach to succinct quantum-safe zero-knowledge. In: CRYPTO Part II. pp. 441–469 (2020)
19. Boschini, C., Camenisch, J., Ovsiankin, M., Spooner, N.: Efficient post-quantum SNARKs for RSIS and RLWE and their applications to privacy. In: PQCrypto. pp. 247–267 (2020)
20. Bowe, S., Grigg, J., Hopwood, D.: Halo: Recursive proof composition without a trusted setup. *IACR Cryptol. ePrint Arch.* 2019/1021 (2019)

21. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: S&P. pp. 315–334 (2018)
22. Bünz, B., Chiesa, A., Mishra, P., Spooner, N.: Proof-carrying data from accumulation schemes. IACR Cryptol. ePrint Arch. 2020/499 (2020)
23. Bünz, B., Fisch, B., Szepieniec, A.: Transparent SNARKs from DARK compilers. In: EUROCRYPT Part I. pp. 677–706 (2020)
24. Bünz, B., Maller, M., Mishra, P., Tyagi, N., Vesely, P.: Proofs for inner pairing products and applications. In: ASIACRYPT Part III. pp. 65–97 (2021)
25. Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Accumulation without homomorphism. IACR Cryptol. ePrint Arch. 2024/474 (2024)
26. Campanelli, M., Fiore, D., Querol, A.: LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In: CCS. pp. 2075–2092 (2019)
27. Canetti, R., Chen, Y., Holmgren, J., Lombardi, A., Rothblum, G.N., Rothblum, R.D.: Fiat-Shamir from simpler assumptions. IACR Cryptol. ePrint Arch. 2018/1004 (2018)
28. Canetti, R., Chen, Y., Holmgren, J., Lombardi, A., Rothblum, G.N., Rothblum, R.D., Wichs, D.: Fiat-Shamir: From practice to theory. In: STOC. pp. 1082–1090 (2019)
29. Chee, J., Cai, Y., Kuleshov, V., Sa, C.D.: QuIP: 2-bit quantization of large language models with guarantees. In: NeurIPS. pp. 4396–4429 (2023)
30. Chen, B., Waiwitlikhit, S., Stoica, I., Kang, D.: ZKML: An optimizing system for ML inference in zero-knowledge proofs. In: EuroSys. pp. 560–574 (2024)
31. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In: EUROCRYPT Part II. pp. 499–530 (2023)
32. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, P., Ward, N.P.: Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In: EUROCRYPT Part I. pp. 738–768 (2020)
33. Chiesa, A., Ojha, D., Spooner, N.: Fractal: Post-quantum and transparent recursive proofs from holography. In: EUROCRYPT Part I. pp. 769–793 (2020)
34. Chiesa, A., Tromer, E.: Proof-carrying data and hearsay arguments from signature cards. In: ICS. pp. 310–331 (2010)
35. Cormode, G., Mitzenmacher, M., Thaler, J.: Practical verified computation with streaming interactive proofs. In: ITCS. pp. 90–112 (2012)
36. Feng, B., Qin, L., Zhang, Z., Ding, Y., Chu, S.: ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences. IACR Cryptol. ePrint Arch. 2021/087 (2021)
37. Frantar, E., Alistarh, D.: QMoE: Sub-1-bit compression of trillion parameter models. In: MLSys (2024)
38. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. IACR Cryptol. ePrint Arch. 2019/953 (2019)
39. Ganesh, C., Patranabis, S., Singh, N.: Samaritan: Linear-time prover SNARK from new multilinear polynomial commitments. IACR Cryptol. ePrint Arch. 2025/419 (2025)
40. Garg, S., Jain, A., Jin, Z., Zhang, Y.: Succinct zero knowledge for floating point computations. In: CCS. pp. 1203–1216 (2022)
41. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct NIZKs without PCs. In: EUROCRYPT. pp. 626–645 (2013)
42. Gentry, C., Halevi, S., Lyubashevsky, V.: Practical non-interactive publicly verifiable secret sharing with thousands of parties. In: EUROCRYPT Part I. pp. 458–487 (2022)

43. Goldreich, O., Krawczyk, H.: On the composition of zero-knowledge proof systems. *SIAM J. Comput.* **25**(1), 169–192 (1996)
44. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating computation: Interactive proofs for muggles. *J. ACM* **62**(4), 1–64 (2015)
45. Golovnev, A., Lee, J., Setty, S.T.V., Thaler, J., Wahby, R.S.: Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In: *CRYPTO Part II*. pp. 193–226 (2023)
46. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: *Usenix Security*. pp. 519–535 (2021)
47. Groth, J.: Efficient zero-knowledge arguments from two-tiered homomorphic commitments. In: *ASIACRYPT*. pp. 431–448 (2011)
48. Groth, J.: On the size of pairing-based non-interactive arguments. In: *EUROCRYPT Part II*. pp. 305–326 (2016)
49. Kang, D., Hashimoto, T., Stoica, I., Sun, Y.: Scaling up trustless DNN inference with zero-knowledge proofs. *arXiv:2210.08674* (2022)
50. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: *ASIACRYPT*. pp. 177–194 (2010)
51. Kei, A.Y.L., Chow, S.S.M.: SHAFT: Secure, handy, accurate and fast transformer inference. In: *NDSS* (2025)
52. Kei, A.Y.L., Ng, L.K.L., Ma, J.P.K., Chow, S.S.M.: Sort, Sweep, Mirror: Batch private interval lookup with logarithmic cost. In: *S&P* (2026), to appear
53. Keuffer, J., Molva, R., Chabanne, H.: Efficient proof composition for verifiable computation. In: *ESORICS Part I*. pp. 152–171 (2018)
54. Kothapalli, A., Parno, B.: Algebraic reductions of knowledge. In: *CRYPTO Part IV*. pp. 669–701 (2023)
55. Kothapalli, A., Setty, S.T.V.: SuperNova: Proving universal machine executions without universal circuits. *IACR Cryptol. ePrint Arch.* 2022/1758 (2022)
56. Kothapalli, A., Setty, S.T.V.: HyperNova: Recursive arguments for customizable constraint systems. In: *CRYPTO Part X*. pp. 345–379 (2024)
57. Kothapalli, A., Setty, S.T.V., Tzialla, I.: Nova: Recursive zero-knowledge arguments from folding schemes. In: *CRYPTO Part IV*. pp. 359–388 (2022)
58. Lai, R.W.F., Malavolta, G., Ronge, V.: Succinct arguments for bilinear group arithmetic: Practical structure-preserving cryptography. In: *CCS*. pp. 2057–2074 (2019)
59. Lee, J.: Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In: *TCC Part II*. pp. 1–34 (2021)
60. Lee, S., Ko, H., Kim, J., Oh, H.: vCNN: Verifiable convolutional neural network based on zk-SNARKs. *IEEE TDSC* **21**(4), 4254–4270 (2023)
61. Li, W., Zhang, Z., Chow, S.S.M., Guo, Y., Gao, B., Song, X., Deng, Y., Liu, J.: Shred-to-shine metamorphosis in polynomial commitment evolution. *IACR Cryptol. ePrint Arch.* 2025/1354 (2025)
62. Li, W., Zhang, Z., Guo, Y., Chow, S.S.M., Wan, Z.: Succinct hash-based arbitrary-range proofs. *IEEE Trans. Inf. Forensics Secur.* **20**, 145–158 (2025)
63. Liu, T., Xie, X., Zhang, Y.: zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In: *CCS*. pp. 2968–2985 (2021)
64. Lund, C., Fortnow, L., Karloff, H.J., Nisan, N.: Algebraic methods for interactive proof systems. *J. ACM* **39**(4), 859–868 (1992)
65. Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., et al.: FP8 formats for deep learning. *arXiv:2209.05433* (2022)

66. Ng, L.K.L., Chow, S.S.M.: GForce: GPU-friendly oblivious and rapid neural network inference. In: Usenix Security. pp. 2147–2164 (2021)
67. Ng, L.K.L., Chow, S.S.M.: SoK: Cryptographic neural-network computation. In: S&P. pp. 497–514 (2023)
68. ONNX: Open neural network exchange. <https://github.com/onnx/onnx> (2018), accessed September 12, 2025
69. Papamanthou, C., Shi, E., Tamassia, R.: Signatures of correct computation. In: TCC. pp. 222–242 (2013)
70. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. Commun. ACM **59**(2), 103–112 (2016)
71. Setty, S.T.V.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: CRYPTO Part III. pp. 704–737 (2020)
72. Setty, S.T.V., Lee, J.: Quarks: Quadruple-efficient transparent zkSNARKs. IACR Cryptol. ePrint Arch. 2020/1275 (2020)
73. Setty, S.T.V., Thaler, J., Wahby, R.S.: Unlocking the lookup singularity with Lasso. In: EUROCRYPT Part VI. pp. 180–209 (2024)
74. Sun, H., Li, J., Zhang, H.: zkLLM: Zero knowledge proofs for large language models. In: CCS. pp. 4405–4419 (2024)
75. Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I.J., Fergus, R.: Intriguing properties of neural networks. In: ICLR (Poster) (2014)
76. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: CRYPTO Part II. pp. 71–89 (2013)
77. Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: TCC. pp. 1–18 (2008)
78. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zk-SNARKs without trusted setup. In: S&P. pp. 926–943 (2018)
79. Waiwitlikhit, S., Stoica, I., Sun, Y., Hashimoto, T., Kang, D.: Trustless audits without revealing data or models. In: ICML. pp. 49808–49821 (2024)
80. Wang, H., Hoang, T.: ezDPS: An efficient and zero-knowledge machine learning inference pipeline. Proc. Priv. Enhancing Technol. **2023**(2), 430–448 (2023)
81. Weng, C., Yang, K., Xie, X., Katz, J., Wang, X.: Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In: Usenix Security. pp. 501–518 (2021)
82. Weng, J., Weng, J., Tang, G., Yang, A., Li, M., Liu, J.N.: pvCNN: Privacy-preserving and verifiable convolutional neural network testing. IEEE Trans. Inf. Forensics Secur. **18**, 2218–2233 (2023)
83. Xiao, G., Lin, J., Sez nec, M., Wu, H., Demouth, J., Han, S.: SmoothQuant: Accurate and efficient post-training quantization for large language models. In: ICML. pp. 38087–38099 (2023)
84. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct zero-knowledge proofs with optimal prover computation. In: CRYPTO Part III. pp. 733–764 (2019)
85. Xie, T., Zhang, Y., Song, D.: Orion: Zero knowledge proof with linear prover time. In: CRYPTO. pp. 299–328 (2022)
86. Zafrir, O., Boudoukh, G., Izsak, P., Wasserblat, M.: Q8BERT: Quantized 8bit BERT. In: EMC2@NeurIPS. pp. 36–39 (2019)
87. Zcash: Halo2. <https://zcash.github.io/halo2>, accessed September 12, 2025
88. Zhang, J., Fang, Z., Zhang, Y., Song, D.: Zero knowledge proofs for decision tree predictions and accuracy. In: CCS. pp. 2039–2053. ACM (2020)
89. Zhang, J., Xie, T., Zhang, Y., Song, D.: Transparent polynomial delegation and its applications to zero knowledge proof. In: S&P. pp. 859–876 (2020)

90. Zhang, Y., Genkin, D., Katz, J., Papadopoulos, D., Papamanthou, C.: A zero-knowledge version of vSQL. IACR Cryptol. ePrint Arch. 2017/1146 (2017)
91. Zhang, Z., Li, W., Guo, Y., Shi, K., Chow, S.S.M., Liu, X., Dong, J.: Fast RS-IOP multivariate polynomial commitments and verifiable secret sharing. In: Usenix Security. pp. 3187–3204 (2024)

A Related Work

General-Purpose zkSNARKs proves knowledge of a witness that satisfies a given arithmetic circuit, with prover time scaling in the number of multiplication gates. Since Groth16 [48], designs have evolved from quasilinear prover using FFTs [3,6,8,23,33,32,38,41,70,89] to strictly-linear and FFT-free zkSNARKs [15,31,45,71,72,84]. They rely on polynomial intermediate representations, evolving from quadratic arithmetic programs (QAP) [41] to R1CS [7] and PLONK-style constraints [31,38]. Recent zkSNARKs typically follow a paradigm that combines a polynomial interactive oracle proof (PIOP) [9,23] with a polynomial commitment scheme [50]. PIOPs typically build on the sumcheck protocol [64], introduced to zkSNARKs by Cormode *et al.* [35]. PCS commits to a polynomial and then verifies its values at specific points. Elliptic-curve-based PCS include Kate–Zaverucha–Goldberg [50,69], Bulletproofs [21], Hyrax [78], and Dory [59]. Post-quantum PCS include BCG20 [14], BCL22 [16], Orion [45,85], and PolyFRIM [91]. Most PCS verify polynomial evaluations via vector-matrix-vector products. Thus, they also function as matrix commitment schemes.

Verifiable Machine Learning extends zkSNARKs to machine learning models, especially NNs. Early works transform NNs to Circ-SAT and apply general-purpose zkSNARKs [36,80,82], with scaling improvements through specialized protocols [49,60]. Later approaches [53,63,74] use variations of the GKR protocol [44] for improved prover efficiency. Since NNs rely heavily on matrix operations, a central problem in VML research is designing efficient protocols for these operations, *e.g.*, matrix multiplications, convolutions, non-linear functions.

Folding Techniques were first introduced by Bootle *et al.* [13] and later refined [21], enabling an inner-product argument, Bulletproofs, for Pedersen vector commitments. This technique has been extended to other cryptographic settings, such as bilinear groups [24,58,59], unknown-order groups [19], and lattices [18]. Bootle *et al.* [17] demonstrate that folding techniques can be interpreted as sumcheck arguments. This paper explores the converse implication, proposing an inner product proof based on folding techniques to replace the sumcheck.

A.1 Comparison with IVC/PCD

We work through a simple chain example that underlies Fig. 4 for highlighting: (i) per-step recursion/accumulation in IVC/PCD versus a single outer PoP in Evalyn, (ii) intermediate-state commitments versus scalar projections across the interface, and (iii) step order versus reverse topological order.

Table 2. Distinctions between Evalyn and IVC/PCD

Aspect	Evalyn	IVC/PCD
Proof-of-Proof use	Outer one-shot: a single PoP after backward reduction over the whole DAG.	Per-step: recursive verification and/or accumulation at each step, maintaining a proof throughout.
Intermediate-state commitments	Avoided between blocks: share only a logarithmic number of field elements by passing scalar projections and projecting vectors across the interface; the outer proof binds the reduction transcript and the topology commitment.	Typically present: per-step states are bound via commitments to intermediate states or accumulator digests.
Aggregation order	Backward reduction: reduce in reverse dependency/topological order of the DAG.	Forward recursion: compose proofs in the step/temporal order of the computation (chain) or along the message graph.

We consider a chain of computation on inputs $\mathbf{a}_{i \in [M]}$ as follows:

$$\mathbf{c}_0 = \mathbf{0}, \quad \mathbf{c}_i = \phi_i(\mathbf{c}_{i-1}, \mathbf{a}_i) \quad \forall i \in [M], \quad \text{with } \mathbf{c}_M = \mathbf{0}.$$

$\phi_{i \in [M]}$ are binary Type-1 operations. This corresponds to the matrix expression:

$$f(\mathbf{a}_1, \dots, \mathbf{a}_M) := \phi_M(\phi_{M-1}(\dots, \phi_1(\mathbf{0}, \mathbf{a}_1), \mathbf{a}_2), \dots, \mathbf{a}_M) = \mathbf{0}.$$

Both IVC/PCD and Evalyn can provide a zkSNARK proof for this computation, composing atomic proofs in *step order* or in *reverse topological order* (Fig. 4).

Forward recursion and per-step composition in IVC/PCD. IVC/PCD maintain one evolving proof. At step i , the prover $\mathcal{P}_i$ produces a proof π_i that certifies $\phi_i(\mathbf{c}_{i-1}, \mathbf{a}_i) = \mathbf{c}_i$ and that the previous proof verifies: $\mathcal{V}_{i-1}(\pi_{i-1}) = \text{true}$. Thus π_i attests both the current predicate and the validity of π_{i-1} .

In accumulation-based variants, the prover updates an accumulation A_i that aggregates per-step obligations; openings are deferred until the end. A decider $\mathcal{D}$ only checks that both π_M and A_M are valid, instead of replaying all intermediate checks, *e.g.*, the polynomial commitment openings of previous proofs. A VML example, Kaizen [1], uses IVC to prove NN training and attains prover time $O(Mn^2)$ per iteration with proof size and verifier time $O(\log M \cdot \log^2 n)$ using Orion PCS [85], where the $\log M$ factor arises from the depth reduction technique [25], which organizes the M predicates into a depth- $O(\log M)$ tree.

Backward reduction and one-shot PoP in Evalyn. Unlike IVC/PCD, Evalyn applies PoP only once to the proof generated from the backward proof reduction. Reduction starts from the last predicate $\phi_M(\mathbf{c}_{M-1}, \mathbf{a}_M) = \mathbf{c}_M$ and proceeds

toward $\phi_1(\mathbf{c}_0, \mathbf{a}_1) = \mathbf{c}_1$ in reverse step order. As shown in Fig. 4, at step M , $\mathcal{P}_M$ reduces ϕ_M to scalar projections of $\mathbf{c}_{M-1}$ and $\mathbf{a}_M$ at random vectors $\vec{\mathbf{l}}_{M-1}, \vec{\mathbf{r}}_{M-1}$ and $\vec{\mathbf{l}}'_M, \vec{\mathbf{r}}'_M$, producing scalars v_{M-1} and v'_M . It passes $(v_{M-1}, \vec{\mathbf{l}}_{M-1}, \vec{\mathbf{r}}_{M-1})$ to $\mathcal{P}_{M-1}$. Recursively, $\mathcal{P}_i$ reduces the projection v_i of $\mathbf{c}_i$ to projections v_{i-1} and v'_i of $\mathbf{c}_{i-1}$ and $\mathbf{a}_i$, until only leaf projections v'_i remain. Leaf projections are then merged into a single projection with an additional LiteBullet proof. Finally, a one-shot PoP verifies the M reduced predicates to achieve succinctness and zero knowledge. With Dory PCS [59], the proof size and verifier time are $O(\log(Mn))$.

Commitments vs. scalar projections for cross-step consistency. Composition must enforce that the output of ϕ_{i-1} equals the input of ϕ_i . IVC/PCD enforce this by binding intermediate states $\mathbf{c}_i$. These commitments are either verified via PoP across successive operations [10] or accumulated into the commitments of subsequent operations [22]. Both approaches are costly due to the relatively expensive group operations of group-based commitment schemes. In contrast, Evalyn links successive steps through scalar values v_i and random vectors $\vec{\mathbf{l}}_i, \vec{\mathbf{r}}_i$, avoiding per-step commitments between blocks while the outer PoP binds the full reduction transcript and the topology commitment.

Table 2 summarizes the distinctions between Evalyn and IVC/PCD.

A.2 Comparison with the Monolithic-Circuit Approach

A monolithic circuit approach compiles the entire NN computation into a single arithmetic circuit. In theory, this approach can achieve a prover complexity of $O(Mn^2)$, matching our scheme’s asymptotic efficiency, by combining:

- Specialized linear-size circuits for superlinear operations like matrix multiplication and convolution, and
- General-purpose zkSNARKs with strictly linear prover complexity and logarithmic verifier complexity.

However, the concrete prover times differ significantly, with the monolithic approach being at least an order of magnitude slower than our method, due to larger circuit sizes and substantial commitment overhead.

Consider our NN example with 2^{10} layers, each defined by:

$$\mathbf{X}_i = \sigma(\mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i), \quad i \in [2^{10}],$$

where each weight matrix $\mathbf{W}_i$ has a dimension $2^{10} \times 2^{10}$, and both the biases $\mathbf{B}_i$ and the layer inputs/outputs $\mathbf{X}_i$ are vectors of length 2^{10} .

Concrete Cost of the Monolithic-Circuit Approach. Compiling this NN into a monolithic circuit yields $\sim 2^{30}$ input/output (I/O) gates and $\sim 2^{30}$ multiplication gates, consisting of:

- I/O gates for $\mathbf{X}_i, \mathbf{W}_i, \mathbf{B}_i, \mathbf{c}_i := \mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i$, and the auxiliary inputs required by lookup proofs ($4 \cdot 2^{20} + 2 \cdot 2^{25} + 3 \cdot 2^{18}$ (respectively $4 \cdot 2^{20} + 5 \cdot 2^{13}$) scalar values for 16-bit (respectively 8-bit) quantization), totaling $\sim 2^{30} + 2^{26}$ scalar values (for 16-bit quantization; the following calculations use this case).

- Multiplication gates used to verify the computation $\mathbf{c}_i = \mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i$ via $\vec{\mathbf{l}}^\top (\mathbf{c}_i - \mathbf{B}_i) \vec{\mathbf{r}} = \vec{\mathbf{l}}^\top \mathbf{W}_i \mathbf{X}_{i-1} \vec{\mathbf{r}}$ with random vectors $\vec{\mathbf{l}}, \vec{\mathbf{r}} \in \mathbb{F}^{2^{10}}$. This requires $\sim 2^{30}$ multiplication gates across all layers.
- Multiplication gates involved in the grand product computations for lookup proofs, totaling $\sim 8 \cdot 2^{20} + 4 \cdot 2^{25} + 6 \cdot 2^{18} \simeq 2^{27}$ scalar multiplications.

Overhead of Large, Sparse R1CS Matrices. In a monolithic approach, the Circ-SAT problem is transformed into a constraint system, typically the rank-1 constraint system (R1CS, Appendix E.4), characterized by extremely large and sparse matrices $\{\mathbf{a}_i^{\text{mono}}\}_{i \in \{L, R, O\}}$ that satisfy:

$$(\mathbf{a}_L^{\text{mono}} \vec{\mathbf{w}}) \circ (\mathbf{a}_R^{\text{mono}} \vec{\mathbf{w}}) = \mathbf{a}_O^{\text{mono}} \vec{\mathbf{w}}.$$

Here, $\vec{\mathbf{w}}$ contains $\sim 2 \cdot 2^{30}$ entries representing all I/O and multiplication gates. Each R1CS matrix $\sim \mathbf{a}_i^{\text{mono}}$ thus has dimensions approximately $\sim (2^{30}) \times (2 \cdot 2^{30})$, each containing $\sim 2^{30}$ non-zero entries.

Managing this sparsity—even with optimized methods like Spartan [71], which uses lookup proofs to verify sparse matrices—results in significant overhead. Specifically, Spartan utilizes lookup proofs to verify sparse matrices. Following Lasso [73], these lookup proofs require commitments to 6 vectors of size 2^{30} and 3 vectors of size $2 \cdot 2^{30}$, totaling $\sim 12 \cdot 2^{30}$ scalar values.

Consequently, the monolithic-circuit approach demands in the proving phase commitments to a total of $\sim 13 \cdot 2^{30}$ scalar values, making it practically prohibitive compared to the much lower commitment overhead of our Evalyn. Worse still, storing these scalars as 256-bit field elements would exceed practical RAM limits.

Efficiency Gains by Evalyn. A core design principle of Evalyn is to avoid commitments whenever possible, thus reducing prover time compared to the monolithic-circuit approach. Quantitatively, Evalyn reduces from committing to $\sim 13 \cdot 2^{30}$ scalar values in the monolithic-circuit approach to $\sim 2^{27}$ scalar values. Since commitment overhead is proportional to the number of scalar values committed, this $\sim 104\times$ reduction in commitment overhead translates into significant improvement in prover efficiency relative to the monolithic-circuit approach.

Smaller Circuits. Evalyn combines the proof reduction technique in Section 7 and the PoP method in Section 8.2 to replace the large monolithic circuit with a smaller PoP circuit and batched hash verification. Intuitively, the PoP method transforms the original DAG—consisting of large-scale matrices and matrix operations—into a topologically equivalent DAG composed of scalar operations and smaller, logarithmic-size atomic verifier circuits, as illustrated in Fig. 2. Since the PoP circuit is much smaller than the monolithic circuit yet preserves the original NN’s topological structure, committing to this PoP circuit effectively hides the NN architecture while greatly minimizing commitment overhead.

For an $n \times n$ matrix product, such as multiplication, Hadamard product, and convolution, a monolithic circuit uses n^2 scalar multiplications, whereas our PoP circuit needs at most $4 \log_2 n$ multiplication gates in PoP circuit and $\eta \cdot 2 \log_2 n$ scalar multiplications within the associated hashes, where η denotes the scalar multiplication count per hash. Thus, when $\eta \leq n$ (*e.g.*, Poseidon hash, $\eta \simeq 2^8$), this PoP approach enhances efficiency. Consider our example NN:

- The proof reduction involves $\sim 2^{17}$ hashes, with $\sim 9 \cdot \log_2^2(2^{20}) + 6 \cdot \log_2^2(2^{25}) + 3 \log_2^2(2^{18}) \simeq 2^{13}$ for the lookup proofs, $\sim 2^{10} \cdot 6 \log_2(2^{10}) = 60 \cdot 2^{10}$ for matrix multiplications, and $\sim 2^{10} \cdot 6 \log_2(2^{10}) = 60 \cdot 2^{10}$ for batch projections of $\mathbf{X}_i$ resulting from the lookup proof and the matrix multiplication proofs.
- Poseidon hashes (each requiring $\sim 2^8$ scalar multiplications) lead to $\sim 2^{25}$ multiplication gates for all hash operations.
- The PoP circuit involves $\sim 2^{18}$ scalar multiplication gates, approximately equivalent to twice the number of hashes.

The prover time of our scheme thus includes:

- Committing to $\mathbf{X}_i$ and auxiliary matrices in lookup proofs, containing in total $\sim 2^{26}$ scalars.
- The prover time for the PoP circuit with approximately 2^{16} gates.
- The prover time for the $\sim 2^{16}$ hashes, dominated by committing to the $\sim 2^{25}$ internal multiplication gates of hash computations.

In total, $\sim 2^{26} + 2^{25} < 2^{27}$ scalar values are committed in Evalyn, substantially lowering the commitment overhead compared to the monolithic-circuit approach.

A.3 Comparison between Vector and Polynomial Representations

The vector/matrix representation and the polynomial representation are interchangeable approaches for encoding computational problems.

Equivalence Between Polynomial Evaluation and Inner Product. Polynomial evaluation can equivalently be represented as an inner product or a vector-matrix-vector product (also referred to as matrix projection). This relationship is fundamental in various PCS, *e.g.*, [14,45,59,78].

Consider a vector $\vec{a} \in \mathbb{F}^m$. It can be uniquely represented as a multilinear polynomial $\psi_a(\mathbf{x})$ in $\log_2 m$ variables, where each vector element corresponds to a polynomial term. For example, a vector $\vec{a} = (1, 2, 3, 4)$ can be represented by:

$$\psi_a(x_1, x_2) = 1 + 2x_2 + 3x_1 + 4x_1x_2.$$

Evaluating ψ_a at a point (x_1, x_2) can equivalently be computed as an inner product or matrix projection:

$$\psi_a(x_1, x_2) = (1, 2, 3, 4) \bullet [(1, x_1) \otimes (1, x_2)] = (1 \ x_1) \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ x_2 \end{pmatrix}.$$

A PCS commits to a polynomial ψ_a and later verifies its evaluation at a chosen point $\mathbf{x}$. Equivalently, a vector commitment scheme commits to a vector $\vec{a}$ and later verifies its inner product with a tensor-structured vector, or a matrix commitment scheme commits to a matrix and later verifies its projection onto tensor-structured vectors. This equivalence is widely utilized in PCS methods, *e.g.*, Hyrax [78], Dory [59], BCG20 [14], and Brakedown [45], where inner product arguments (IPAs) verify polynomial evaluations.

Simplified Pipeline using Vector Representation. Traditional zkSNARK pipelines transform matrix-based constraint systems (*e.g.*, R1CS) into polynomial equations. These polynomial equations are either univariate (leveraging FFTs) or low-degree multivariate (handled via sumcheck). Sumcheck reduces multivariate polynomial equations to multilinear polynomial evaluations, which are verified using PCS typically based on IPAs.

Hence, the classical zkSNARK pipeline is:

Matrix equation $\rightarrow$ *Multilinear polynomial equation* $\rightarrow$ *Sumcheck*
 $\rightarrow$ *Polynomial evaluation* $\rightarrow$ *PCS* $\rightarrow$ *Inner product with tensor-structured vectors*
 $\rightarrow$ *IPA* $\rightarrow$ *True/False*

Adopting a vector representation significantly simplifies this pipeline:

Matrix equation $\rightarrow$ *LiteBullet*
 $\rightarrow$ *Inner product with tensor-structured vectors*
 $\rightarrow$ *IPA* $\rightarrow$ *True/False*

This streamlined pipeline simplifies both the protocol design and implementation. Thus, we prefer the matrix/vector representation.

Advantages of Polynomial-Free Representation for VML. Representing NNs with matrices and vectors is a natural fit, as NNs are inherently sequences of matrix operations. Polynomial-based encodings, developed mostly for cryptography, introduce a foreign abstraction layer for ML practitioners that complicates implementation and limits VML’s accessibility. Our polynomial-free approach avoids this overhead, integrating seamlessly with existing ML frameworks and making verifiable machine learning more practical for widespread adoption.

A.4 Comparison between LiteBullet and Sumcheck

We briefly recall the standard identity relating multilinear extensions (MLEs) on the Boolean hypercube to inner products, and then explain how LiteBullet implements the same algebra via folding, without polynomials. Let $m = 2^\mu$ and let $\vec{a}, \vec{b} \in \mathbb{F}^m$ (with $\mathbb{F}$ the base field). Define the MLEs $\psi_a, \psi_b: \mathbb{F}^\mu \rightarrow \mathbb{F}$ in the standard way:

$$\psi_a(\mathbf{x}) = \sum_{i=1}^m a_i \omega_i(\mathbf{x}), \quad \text{where} \quad \omega_i(\mathbf{x}) := \prod_{\tau=1}^{\mu} (i_\tau x_\tau + (1 - i_\tau)(1 - x_\tau)),$$

where i_τ represents the τ -th bit of i in little-endian format. and similarly for $\psi_b(\mathbf{x})$. Then we have the canonical identity:

$$\vec{a} \bullet \vec{b} = \sum_{\mathbf{x} \in \{0,1\}^\mu} \psi_a(\mathbf{x}) \cdot \psi_b(\mathbf{x}). \quad (11)$$

This is immediate by expanding both ψ_a and ψ_b in the ω_i basis: the orthogonality relation $\sum_{\mathbf{x} \in \{0,1\}^\mu} \omega_i(\mathbf{x}) \omega_j(\mathbf{x}) = 1$ if $i = j$ and 0 otherwise kills all cross terms and leaves $\sum_{i=1}^m a_i b_i$.

Sumcheck of Inner Product. Eq. (11) is exactly the form required by sumcheck: the prover runs sumcheck on the polynomial $\psi(\mathbf{x}) = \psi_a(\mathbf{x})\psi_b(\mathbf{x})$ to prove

that $\sum_{\mathbf{x} \in \{0,1\}^\mu} \psi(\mathbf{x}) = c$. After μ rounds, sumcheck reduces verification to checking evaluations of ψ_a and ψ_b at a random point $\mathbf{x} \in \mathbb{F}^\mu$, which one typically proves using a PCS.

LiteBullet via Folding. LiteBullet does not pass to polynomials. Instead, it folds the vectors $(\vec{a}, \vec{b})$ round by round using a random scalar $x \in \mathbb{F}^*$, splitting each vector into left and right halves $(\vec{a}_{\text{LH}}, \vec{a}_{\text{RH}})$ and $(\vec{b}_{\text{LH}}, \vec{b}_{\text{RH}})$ with zero padding if needed so the halves have equal length:

$$(\vec{a}, \vec{b}) \leftarrow (\vec{a}_{\text{LH}} + x^{-1}\vec{a}_{\text{RH}}, \vec{b}_{\text{LH}} + x\vec{b}_{\text{RH}}).$$

By bilinearity, $(\vec{a}_{\text{LH}} + x^{-1}\vec{a}_{\text{RH}}) \bullet (\vec{b}_{\text{LH}} + x\vec{b}_{\text{RH}}) = (\vec{a}_{\text{LH}} \bullet \vec{b}_{\text{LH}}) + (\vec{a}_{\text{RH}} \bullet \vec{b}_{\text{RH}}) + x\iota + x^{-1}\rho$, where $\iota = \vec{a}_{\text{LH}} \bullet \vec{b}_{\text{RH}}$ and $\rho = \vec{a}_{\text{RH}} \bullet \vec{b}_{\text{LH}}$. Hence the fold keeps the base terms and isolates the cross terms with known coefficients x and x^{-1} , and the verifier maintains the invariant $\vec{a} \bullet \vec{b} = ((\vec{a}_{\text{LH}} + x^{-1}\vec{a}_{\text{RH}}) \bullet (\vec{b}_{\text{LH}} + x\vec{b}_{\text{RH}})) - x\iota - x^{-1}\rho$ for the current round. Thus, LiteBullet mirrors sumcheck's recurrence. This fold matches the first round of sumcheck for $\sum_{\mathbf{x} \in \{0,1\}^{\log_2 m}} \psi_a(\mathbf{x})\psi_b(\mathbf{x})$: with $\mathbf{x} = (x_1, \mathbf{x}_{-1})$ and $h^{(1)}(x) = \sum_{\mathbf{x}_{-1} \in \{0,1\}^{m-1}} \psi_a(x, \mathbf{x}_{-1})\psi_b(x, \mathbf{x}_{-1})$, the verifier checks $h^{(1)}(0) + h^{(1)}(1) = c$, samples $x \in \mathbb{F}^*$, and binds the claim to $h^{(1)}(x)$, which is equivalent to replacing $(\vec{a}, \vec{b})$ by $(\vec{a}_{\text{LH}} + x^{-1}\vec{a}_{\text{RH}}, \vec{b}_{\text{LH}} + x\vec{b}_{\text{RH}})$ and carrying the cross terms with coefficients x and x^{-1} . In LiteBullet, we stay in the vector space and carry ι and ρ forward explicitly. After $\log_2 m$ rounds for vectors of length m , the only remaining checks are two projections onto tensor-structured vectors.

To illustrate, in the simplest case $m = 2$ ($\mu = 1$), the standard MLEs over $\{0,1\}$ are $\psi_a(x) = xa_1 + (1-x)a_2$ and $\psi_b(x) = xb_1 + (1-x)b_2$. We then have $\sum_{x \in \{0,1\}} \psi_a(x)\psi_b(x) = a_1b_1 + a_2b_2$, which is exactly the base case of Eq. (11). Accordingly, the inner product relation $c = \vec{a} \bullet \vec{b}$ can be expressed as a sumcheck relation on $\psi_a(x)$ and $\psi_b(x)$:

$$\begin{aligned} c &= \sum_{x \in \{0,1\}} [xa_1 + (1-x)a_2] \cdot [xb_1 + (1-x)b_2] \\ &= \sum_{x \in \{0,1\}} [a_1b_1x^2 + a_2b_2(1-x)^2 + (a_1b_2 + a_2b_1)x(1-x)]. \end{aligned}$$

In the sumcheck protocol, the prover computes the coefficients of the polynomial $h^{(1)}(x) = a_1b_1x^2 + a_2b_2(1-x)^2 + (a_1b_2 + a_2b_1)x(1-x)$ (Line 3, Algorithm 5, Appendix E.3). Here, the coefficient of $x(1-x)$ consists of two terms, a_1b_2 and a_2b_1 , which corresponds to $\iota_{(1)}$ and $\rho_{(1)}$ in LiteBullet (Line 4, Algorithm 1). Thus, LiteBullet can be viewed as the counterpart of sumcheck.

Beyond Coefficient-Form Sumcheck. One might suspect that expanding the sumcheck-based inner product (*e.g.*, BCG20 [14]) into coefficients yields LiteBullet. They are closely related but not identical:

- **Efficiency:** the sumcheck-based inner product in BCG20 performs about $6m$ field multiplications [14, p.38], while LiteBullet needs about $4m$ (for the same m -inner product).

- **Conceptual:** sumcheck encodes vectors as polynomials and proves an identity over $\{0, 1\}^\mu$, whereas LiteBullet performs the same folding algebra directly on vectors, avoiding polynomial encoding.

B Additional Definitions

B.1 Concepts of Soundness

Definition 6 (Proof/Argument/Argument of Knowledge). A proof, an argument, and an argument of knowledge (AoK), defined by the $(\text{Setup}, \mathcal{P}, \mathcal{V})$ tuple below, should satisfy soundness, computational soundness, and knowledge soundness, respectively. They all possess perfect completeness.

- **Setup:** Given a security parameter 1^λ , outputs public parameters pp .
- **Prover $\mathcal{P}$:** Given the public parameters pp , the statement u , outputs a proof transcript π for a language $\mathcal{L}$. Alternatively, given pp , a statement u , and a private witness w , the prover outputs a transcript π for a relation $\mathcal{R}$.
- **Verifier $\mathcal{V}$:** Given pp , statement u , and transcript π , outputs true or false.

Let $(\mathcal{P} \rightleftharpoons \mathcal{V})$ denote the interaction between $\mathcal{P}$ and $\mathcal{V}$.

Definition 7 (Perfect Completeness). $(\text{Setup}, \mathcal{P}, \mathcal{V})$ for a language $\mathcal{L}$ / a relation $\mathcal{R}$ has perfect completeness if for any non-uniform PPT adversary $\mathcal{A}$:

$$\Pr \left(\begin{array}{l} \{u \in \mathcal{L} / \mathcal{R}\} \\ \vee \mathcal{V}(\text{pp}, u, \pi) = \text{true} \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ u / (u, w) \leftarrow \mathcal{A}(\text{pp}); \\ \pi \leftarrow \mathcal{P}(\text{pp}, u) / \mathcal{P}(\text{pp}, u, w) \end{array} \right) = 1,$$

where λ, pp, u, w , and π stand for the security parameter, public structured random strings, the input, the witness, and the transcript, respectively.

Definition 8 (Soundness [27]). $(\text{Setup}, \mathcal{P}, \mathcal{V})$ for a language $\mathcal{L}$ / a relation $\mathcal{R}$ has a soundness error $\kappa^s(u) : \mathbb{N} \mapsto [0, 1]$ if for any unbounded interactive $\mathcal{P}^*$, the probability below is bounded by $\kappa^s(u)$. If $\kappa^s(u)$ is negligible, the protocol is sound.

$$\Pr (\mathcal{V}(\text{pp}, u, \pi) = \text{true} \mid u \notin \mathcal{L} / \mathcal{R}; \pi \leftarrow (\mathcal{P}^* \rightleftharpoons \mathcal{V})(\text{pp}, u)) \leq \kappa^s(u).$$

An argument possesses computational soundness, weaker than soundness:

Definition 9 (Computational Soundness). A protocol $(\text{Setup}, \mathcal{P}, \mathcal{V})$ for the language $\mathcal{L}$ / a relation $\mathcal{R}$ has a computational soundness error $\kappa^{\text{cs}}(u) : \mathbb{N} \mapsto [0, 1]$ if for any PPT adversary $\mathcal{P}^*$, the probability below is bounded by $\kappa^{\text{cs}}(u)$. If $\kappa^{\text{cs}}(u)$ is negligible, the protocol is computationally sound.

$$\Pr (\mathcal{V}(\text{pp}, u, \pi) = \text{true} \mid u \notin \mathcal{L} / \mathcal{R}; \pi \leftarrow (\mathcal{P}^* \rightleftharpoons \mathcal{V})(\text{pp}, u)) \leq \kappa^{\text{cs}}(u).$$

In interactive proofs, by definition, $\kappa^{\text{cs}}(u) \leq \kappa^s(u)$.

An AoK requires knowledge soundness [4]:

Definition 10 (Knowledge Soundness). A protocol $(\text{Setup}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}$ is knowledge sound with a knowledge soundness error of $\kappa^{\text{ks}}(u): \mathbb{N} \mapsto [0, 1]$ if there exists an algorithm Ext , called a knowledge extractor, with the following characteristics: Given input u and black-box oracle access to a (potentially dishonest) prover $\mathcal{P}^*$, the extractor Ext runs in an expected number of steps that is polynomial in $|u|$ (where queries to $\mathcal{P}^*$ count as a single step) and outputs a witness $(u, w) \in \mathcal{R}$ with probability:

$$\Pr((u, \text{Ext}(\mathcal{P}^*, u)) \in \mathcal{R}) \geq \epsilon(\mathcal{P}^*, u) - \kappa^{\text{ks}}(u),$$

where $\epsilon(\mathcal{P}^*, u)$ represents the probability that $\mathcal{P}^*$ generates an accepting transcript. The protocol is considered knowledge sound if $\kappa^{\text{ks}}(u)$ is negligible.

If a protocol is knowledge-sound with knowledge-soundness error $\kappa^{\text{ks}}(u)$, then it must also be sound with soundness error $\kappa^{\text{s}}(u) \leq \kappa^{\text{ks}}(u)$. Otherwise, for some input $u \notin \mathcal{R}$ and a PPT adversary $\mathcal{P}^*$, we would have:

$$\Pr(\mathcal{V}(\text{pp}, u, \pi) = \text{true} \mid u \notin \mathcal{R}; \pi \leftarrow (\mathcal{P}^* \Rightarrow \mathcal{V})(\text{pp}, u)) > \kappa^{\text{ks}}(u).$$

In this case, for such an input u , the extractor Ext must be able to extract a witness w with positive probability such that $(u, w) \in \mathcal{R}$, i.e., $\exists w$ s.t. $(u, w) \in \mathcal{R}$. This contradicts the assumption that $u \notin \mathcal{R}$, as defined in Section 3.2.

B.2 Reduction of Knowledge

RoK is a generalization of AoK. An RoK reduces checking knowledge of a witness in one relation to checking knowledge of a witness in another (simpler) relation.

Definition 11 (Reduction of Knowledge [54]). For relations $\mathcal{R}_1$ and $\mathcal{R}_2$ consisting of public parameters-input-witness tuples, a reduction of knowledge from $\mathcal{R}_1$ to $\mathcal{R}_2$ is defined by PPT algorithms $(\text{Setup}, \mathcal{P}, \mathcal{V})$ denoting the setup algorithm, the prover, and the verifier respectively with the following interface:

- $\text{Setup}(\lambda) \rightarrow \text{pp}$: Takes security parameter λ . Outputs public parameters pp .
- $\mathcal{P}(\text{pp}, u_1, w_1) \rightarrow (u_2, w_2)$: Takes as input public parameters pp , and input-witness pair (u_1, w_1) . Interactively reduces the statement $(\text{pp}, u_1, w_1) \in \mathcal{R}_1$ to a new statement $(\text{pp}, u_2, w_2) \in \mathcal{R}_2$.
- $\mathcal{V}(\text{pp}, u_1) \leftarrow u_2$: Takes as input public parameters pp , and a statement u_1 . Interactively reduces the task of checking the relation $u_1 \in \mathcal{R}_1$ to the task of checking a new relation $u_2 \in \mathcal{R}_2$.

Here, we treat $(\mathcal{P} \Rightarrow \mathcal{V})$ as a function that takes as input (pp, u_1, w_1) and runs the interaction on prover input (pp, u_1, w_1) and verifier input (pp, u_1) . At the end of the interaction, $(\mathcal{P} \Rightarrow \mathcal{V})$ outputs the verifier's statement u_2 and the prover's witness w_2 . An RoK $(\text{Setup}, \mathcal{P}, \mathcal{V})$ satisfies the following conditions:

- **Completeness:** For any PPT adversary $\mathcal{A}$, given $\text{pp} \leftarrow \text{Setup}(\lambda)$ and $(u_1, w_1) \leftarrow \mathcal{A}(\text{pp})$ such that $(\text{pp}, u_1, w_1) \in \mathcal{R}_1$, the prover's output statement is equal to the verifier's output statement and

$$(\mathcal{P} \Rightarrow \mathcal{V})(\text{pp}, u_1, w_1) \in \mathcal{R}_2.$$

- *Knowledge soundness:* For any expected polynomial-time adversaries $\mathcal{A}$ and $\mathcal{P}^*$, there exists an expected polynomial-time extractor Ext such that given $\text{pp} \leftarrow \text{Setup}(\lambda)$ and $(u_1, w_1^*) \leftarrow \mathcal{A}(\text{pp})$,

$$|\Pr[(u_1, \text{Ext}(\text{pp}, u_1, w_1^*)) \in \mathcal{R}_1] - \Pr[(u_2, \text{Ext}(\text{pp}, u_2, w_2^*)) \in \mathcal{R}_2]| \leq \kappa,$$

where κ is negligible.

- *Public reducibility:* There exists a deterministic polynomial-time function Reduce such that for any PPT adversary $\mathcal{A}$ and expected polynomial-time adversary $\mathcal{P}^*$, given $\text{pp} \leftarrow \text{Setup}(\lambda)$, $(u_1, w_1^*) \leftarrow \mathcal{A}(\text{pp})$, and $(u_2, w_2) \leftarrow (\mathcal{P}^* \rightleftharpoons \mathcal{V})(\text{pp}, u_1, w_1^*)$ with interaction transcript π , we have $\text{Reduce}(\text{pp}, u_1, \pi) = u_2$.

Lemma 1 (Sequential Composition of RoK [54]). Consider ternary relations $\mathcal{R}_1, \mathcal{R}_2$, and $\mathcal{R}_3$. For RoKs $\Pi_1 = (\text{Setup}_1, \mathcal{P}_1, \mathcal{V}_1): \mathcal{R}_1 \rightarrow \mathcal{R}_2$ and $\Pi_2 = (\text{Setup}_2, \mathcal{P}_2, \mathcal{V}_2): \mathcal{R}_2 \rightarrow \mathcal{R}_3$, the sequential composition of Π_1 and Π_2 , denoted by $(\text{Setup}, \mathcal{P}, \mathcal{V})$, is an RoK from $\mathcal{R}_1$ to $\mathcal{R}_3$. Its algorithms are given by:

$$\mathcal{P}(\text{pp}, u_1, w_1) = \mathcal{P}_2(\text{pp}, \mathcal{P}_1(\text{pp}, u_1, w_1)), \quad \mathcal{V}(\text{pp}, u_1) = \mathcal{V}_2(\text{pp}, \mathcal{V}_1(\text{pp}, u_1)).$$

Lemma 2 (Parallel Composition of RoK [54]). Consider ternary relations $\mathcal{R}_1, \mathcal{R}_2, \mathcal{R}_3$, and $\mathcal{R}_4$. We define the relation $\mathcal{R}_1 \times \mathcal{R}_2$ as follows:

$$\mathcal{R}_1 \times \mathcal{R}_2 := \{((u_1, u_2), (w_1, w_2)) \mid (u_1, w_1) \in \mathcal{R}_1 \wedge (u_2, w_2) \in \mathcal{R}_2\},$$

and similarly for $\mathcal{R}_3 \times \mathcal{R}_4$. For RoKs $\Pi_1 = (\text{Setup}_1, \mathcal{P}_1, \mathcal{V}_1): \mathcal{R}_1 \rightarrow \mathcal{R}_2$ and $\Pi_2 = (\text{Setup}_2, \mathcal{P}_2, \mathcal{V}_2): \mathcal{R}_3 \rightarrow \mathcal{R}_4$, the parallel composition of Π_1 and Π_2 is an RoK from $\mathcal{R}_1 \times \mathcal{R}_3$ to $\mathcal{R}_2 \times \mathcal{R}_4$, with $(\mathcal{P}, \mathcal{V})$ defined as follows:

$$\begin{aligned} \mathcal{P}(\text{pp}, (u_1, u_3), (w_1, w_3)) &= (\mathcal{P}_1(\text{pp}, u_1, w_1), \mathcal{P}_2(\text{pp}, u_3, w_3)), \\ \mathcal{V}(\text{pp}, (u_1, u_3)) &= (\mathcal{V}_1(\text{pp}, u_1, w_1), \mathcal{V}_2(\text{pp}, u_3, w_3)). \end{aligned}$$

We extend Lemmas 1 and 2 to the following lemmas.

Lemma 3 (Decoupling of RoK). Consider relations $\mathcal{R}_1, \mathcal{R}_2, \mathcal{R}_3, \mathcal{R}_4$, and $\mathcal{R}_5$. We define the relation $\mathcal{R}_2 \times \mathcal{R}_3$ as follows:

$$\mathcal{R}_2 \times \mathcal{R}_3 := \{((u_2, u_3), (w_2, w_3)) \mid (u_2, w_2) \in \mathcal{R}_1 \wedge (u_3, w_3) \in \mathcal{R}_2\},$$

For RoKs $\Pi_1 = (\text{Setup}_1, \mathcal{P}_1, \mathcal{V}_1): \mathcal{R}_1 \rightarrow \mathcal{R}_2 \times \mathcal{R}_3$ (referred to as decoupling of RoK), $\Pi_2 = (\text{Setup}_2, \mathcal{P}_2, \mathcal{V}_2): \mathcal{R}_2 \rightarrow \mathcal{R}_4$, and $\Pi_3 = (\text{Setup}_3, \mathcal{P}_3, \mathcal{V}_3): \mathcal{R}_3 \rightarrow \mathcal{R}_5$, the composition of Π_1, Π_2 , and Π_3 , denoted by $(\text{Setup}, \mathcal{P}, \mathcal{V})$, is an RoK from $\mathcal{R}_1$ to $\mathcal{R}_4 \times \mathcal{R}_5$. Its algorithms are given by:

$$\begin{aligned} \mathcal{P}(\text{pp}, u_1, w_1) &= (\mathcal{P}_2(\text{pp}, [\mathcal{P}_1(\text{pp}, u_1, w_1)]_1), \mathcal{P}_3(\text{pp}, [\mathcal{P}_1(\text{pp}, u_1, w_1)]_2)), \\ \mathcal{V}(\text{pp}, u_1) &= (\mathcal{V}_2(\text{pp}, [\mathcal{V}_1(\text{pp}, u_1)]_1), \mathcal{V}_3(\text{pp}, [\mathcal{V}_1(\text{pp}, u_1)]_2)), \end{aligned}$$

where $[\cdot]_i$ denotes the first ($i = 1$) or the second ($i = 2$) component of a pair.

Lemma 4 (Recomposition of RoK). *Let $\mathcal{R}_1$, $\mathcal{R}_2$, and $\{\mathcal{R}'_i\}_{i \in [k]}$ be relations. For RoKs $\Pi_1 = (\text{Setup}_1, \mathcal{P}_1, \mathcal{V}_1): \mathcal{R}_1 \rightarrow \mathcal{R}'_1 \times \dots \times \mathcal{R}'_k$, which decomposes $\mathcal{R}_1$ into simpler relations $\mathcal{R}'_1, \dots, \mathcal{R}'_k$, and $\Pi_2 = (\text{Setup}_2, \mathcal{P}_2, \mathcal{V}_2): \mathcal{R}'_1 \times \dots \times \mathcal{R}'_k \rightarrow \mathcal{R}_2$, which recomposes $\mathcal{R}'_1, \dots, \mathcal{R}'_k$ into a single relation $\mathcal{R}_2$. Then the sequential composition of Π_1 and Π_2 , denoted $(\text{Setup}, \mathcal{P}, \mathcal{V})$, is an RoK from $\mathcal{R}_1$ to $\mathcal{R}_2$:*

$$\begin{aligned}\mathcal{P}(\text{pp}, u_1, w_1) &= \mathcal{P}_2(\text{pp}, \mathcal{P}_1(\text{pp}, u_1, w_1)), \\ \mathcal{V}(\text{pp}, u_1) &= \mathcal{V}_2(\text{pp}, \mathcal{V}_1(\text{pp}, u_1)).\end{aligned}$$

Recursively using decoupling and recomposition, we propose the following:

Theorem 5 (DAG of RoK). *Given a DAG of relations with a single root, with each node representing a relation: for a node with a single parent, there exists a decoupling of RoK from its parent; for a node with multiple parents, a recomposition of RoK exists from its parents to this node. The composition of these RoKs functions as an RoK from the root-node relation to all leaf-node relations.*

Proof. For any edge $\mathcal{R}_1 \rightarrow \mathcal{R}_2$ in the DAG, a trivial RoK $\mathcal{R}_1 \rightarrow \mathcal{R}_1$ exists and can be added to expand the edge into a chain $\mathcal{R}_1 \rightarrow \mathcal{R}_1 \rightarrow \mathcal{R}_2$. Introducing trivial RoKs into the DAG creates interleaved layers of decoupling and recomposition.

For the first layer, a decoupling RoK exists from the root node to its children, so the result holds. Assume for mathematical induction that for the $(i-1)$ -th layer, an RoK exists from the root node relation to all relations in the $(i-1)$ -th layer; there is an RoK from the root relation to all relations in the i -th layer:

- If the i -th layer is a decoupling layer, Lemma 3 ensures that there is an RoK from the root relation to all i -th layer relations.
- If the i -th layer is a recomposition layer formed by multiple RoK recompositions, Lemma 2 guarantees that the parallel composition of these recompositions is an RoK. Furthermore, by applying Lemma 4, we confirm that there is an RoK from the root relation to all relations in the i -th layer.

So, there is an RoK from the root node relation to all leaf node relations. $\square$

This theorem establishes a general result, of which the proof system in this paper is a specific instance. However, rather than applying it to our scheme, we provide tailored security proofs for our construction for clarity in the main text. Namely, Theorem 2 is a special case of Theorem 5.

B.3 Schwartz–Zippel Lemma

Lemma 5 (Schwartz–Zippel Lemma). *Let $\phi(\mathbf{x}) \in \mathbb{F}[X^\mu]$ be a non-zero polynomial with degree $d > 0$. Let $\mathbb{F}' \subseteq \mathbb{F}$ and let $x_1, \dots, x_\mu \in \mathbb{F}'$ be selected independently and uniformly at random from $\mathbb{F}'$. $\Pr[\phi(x_1, \dots, x_\mu) = 0] \leq \frac{d}{|\mathbb{F}'|}$.*

Corollary 2. *Given $\mathbf{a} \neq \mathbf{0} \in \mathbb{F}^{m \times n}$,*

$$\Pr(\vec{l}^\top \mathbf{a} \vec{r} = 0 \mid \mathbf{x}_l \xleftarrow{\$} \mathbb{F}^{\log_2 m}, \mathbf{x}_r \xleftarrow{\$} \mathbb{F}^{\log_2 n}, \vec{l} = \text{ts}(\mathbf{x}_l), \vec{r} = \text{ts}(\mathbf{x}_r)) \leq 1/|\mathbb{F}|.$$

Proof. The $\vec{l}^\top \mathbf{a} \vec{r}$ as above equals a multilinear polynomial in $(\mathbf{x}_l, \mathbf{x}_r)$. $\square$

Corollary 3. Given $\mathbf{a} \neq \mathbf{0} \in \mathbb{F}^{n \times n}$,

$$\Pr(\vec{l}^\top \mathbf{a} \vec{r} = 0 \mid y, y' \xleftarrow{\$} \mathbb{F}, \vec{l} = (1, y, \dots, y^{n-1}), \vec{r} = (1, y', \dots, y'^{n-1})) \leq n/|\mathbb{F}|.$$

Proof. The $\vec{l}^\top \mathbf{a} \vec{r}$ as above is a degree- n bivariate polynomial in (y, y') . $\square$

B.4 zkSNARK

An argument of knowledge allows a computationally bounded prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that he knows a witness w for a statement u .

Definition 12 (AoK). (Setup, $\mathcal{P}$, $\mathcal{V}$) is called AoK for the relation $\mathcal{R}$ if it satisfies perfect completeness and knowledge soundness (Definitions 7 and 10).

Definition 13 (SNARK). A non-interactive argument of knowledge (Setup, $\mathcal{P}$, $\mathcal{V}$) is a succinct non-interactive argument of knowledge (SNARK) if:

- The setup time is linear in witness size.
- The prover time is linear in witness size.
- The proof length is polylogarithmic in witness size.
- The verifier time is polylogarithmic in witness size.

Definition 14 (zkSNARK). A zero-knowledge non-interactive argument of knowledge (zkSNARK) is a SNARK with the property of perfect zero knowledge.

Definition 15 (Perfect Zero Knowledge of AoK). A non-interactive AoK (Setup, $\mathcal{P}$, $\mathcal{V}$) is perfect zero knowledge if there exists a PPT simulator $\mathcal{S}$ and a trapdoor td such that for all stateful unbounded distinguishers $\mathcal{D}$:

$$\begin{aligned} & \Pr \left(\begin{array}{l} (u, w) \in \mathcal{R} \\ \wedge \mathcal{D}(\text{pp}, u, \pi) = \text{true} \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); (u, w) \leftarrow \mathcal{A}; \\ \pi \leftarrow \mathcal{P}(\text{pp}, u, w) \end{array} \right) \\ &= \Pr \left(\begin{array}{l} (u, w) \in \mathcal{R} \\ \wedge \mathcal{D}(\text{pp}, u, \pi^*) = \text{true} \end{array} \middle| \begin{array}{l} (\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda); (u, w) \leftarrow \mathcal{A}; \\ \pi^* \leftarrow \mathcal{S}(\text{pp}, u, \text{td}) \end{array} \right). \end{aligned}$$

B.5 Commitment

Definition 16 (Commitment). A non-interactive commitment scheme consists of a pair of PPT algorithms (Setup, Comm). The setup algorithm $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ generates public parameters pp for the scheme, provided with the security parameter λ . The commitment algorithm Comm defines a function $\mathbb{M} \times \mathbb{R} \mapsto \mathbb{C}$ for the message space $\mathbb{M}$, the randomness space $\mathbb{R}$, and the commitment space $\mathbb{C}$. For a message $a \in \mathbb{M}$, the algorithm draws $\tilde{a} \xleftarrow{\$} \mathbb{R}$ uniformly at random, and computes the commitment $C_a \leftarrow \text{Comm}(a; \tilde{a})$.

Non-hiding commitments omit randomness, so $\mathbb{R} = \emptyset$ and $C_a \leftarrow \text{Comm}(a)$.

Definition 17 (Computational Binding). A commitment scheme Comm is computational binding with binding advantage κ_{com} if κ_{com} is negligible in λ where

$$\Pr \left(\begin{array}{l} a_0 \neq a_1 \\ \wedge \text{Comm}(a_0; \tilde{a}_0) = \text{Comm}(a_1; \tilde{a}_1) \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ a_0, \tilde{a}_0, a_1, \tilde{a}_1 \leftarrow \mathcal{A}(\text{pp}) \end{array} \right) \leq \kappa_{\text{com}}$$

for any PPT adversary $\mathcal{A}$, over the randomness of Setup and $\mathcal{A}$.

Definition 18 (Computational Hiding). A commitment scheme is said to be hiding if there exists a negligible error κ_{hiding} such that, for any PPT adversary $\mathcal{A}$,

$$\left| \Pr \left(\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (a_0, a_1) \in \mathbb{M}, i \xleftarrow{\$} \{0, 1\}, \tilde{a} \xleftarrow{\$} \mathbb{R}, \\ C_a \leftarrow \text{Comm}(a_i; \tilde{a}); i' \leftarrow \mathcal{A}(\text{pp}, C_a) \end{array} \right) - \frac{1}{2} \right| \leq \kappa_{\text{hiding}}.$$

B.6 Round-by-Round Soundness

Definition 19 (Round-by-Round Soundness [27]). Let $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ be a $(2\mu + 1)$ -message public coin interactive proof system (Definition 20, Appendix B.4) for a language $\mathcal{L}$. For any statement $u \in \{0, 1\}^*$, and any prefix π_j of a protocol transcript. Let $\mathbf{H}(u, \pi_j)$ denote the distribution of the next message of $\mathcal{V}$ when the transcript so far is $\pi_j = (\alpha_0, x_1, \alpha_1, \dots, \alpha_{j-1}, x_j, \alpha_j)$, $j \in [0, \mu]$ contains the challenges and responses of the in the initial $2j + 1$ moves. Π has round-by-round soundness error $\{\kappa_j^{\text{rbr}}(u)\}_{j \in [0, \mu]}$ if there exists a series of deterministic (not necessarily efficiently computable) functions $\{\text{State}_j\}_{j \in [0, \mu]}$ that takes as input a statement u and a transcript prefix π_j and outputs either **accept** or **reject** such that the following properties hold:

- If $u \notin \mathcal{L}$, then $\text{State}_0(u, \emptyset) = \text{reject}$, where $\emptyset$ denotes the empty transcript.
- If $\text{State}_{j-1}(u, \pi_{j-1}) = \text{reject}$ for a transcript prefix π_{j-1} , then for every potential prover's message α_j and verifier's challenge x_j , it holds that:

$$\Pr \left(\text{State}_j(u, \pi_j) = \text{accept} \middle| \begin{array}{l} \text{State}_{j-1}(u, \pi_{j-1}) = \text{reject}; \\ x_j \leftarrow \mathbf{H}(u, \pi_{j-1}); \pi_j \leftarrow (\pi_{j-1} \| x_j \| \alpha_j) \end{array} \right) \leq \kappa_j^{\text{rbr}}(u).$$

- For any full transcript π for which the verifier halts, if $\text{State}_\mu(u, \pi) = \text{reject}$ then $\mathcal{V}(u, \pi) = \text{false}$.

Below lemma connects the round-by-round and overall soundness errors.

Lemma 6 (Round-by-Round Soundness Implies Soundness [27, Proposition 5.4]). Let Π be a $(2\mu + 1)$ -move interactive proof with round-by-round soundness error $\kappa_j^{\text{rbr}}(u)$, $j \in [\mu]$. Then, Π has a soundness error $\sum_{j=1}^{\mu} \kappa_j^{\text{rbr}}(u)$.

B.7 Special Soundness

As demonstrated [4], special soundness implies knowledge soundness.

Definition 20 (Public Coin). A $(2\mu+1)$ -move public-coin interactive protocol is an argument of knowledge $(\text{Setup}, \mathcal{P}, \mathcal{V})$ where in the $2i$ -th move, $\mathcal{V}$ sends the i th challenge $x_i \xleftarrow{\$} \mathbb{R}_i$ from the randomness space $\mathbb{R}_i$. All challenges $x_1, \dots, x_\mu$ sent from the verifier to the prover are chosen uniformly at random and independently of the prover's messages.

Definition 21 (Tree of Transcripts [4]). An $(t_1, \dots, t_\mu)$ -tree of transcripts for a $(2\mu+1)$ -move public-coin protocol is a set of $T = \prod_{i=1}^\mu t_i$ transcripts arranged in a tree. Nodes in this tree represent the prover's messages, and edges correspond to the verifier's challenges. Each node at depth i has precisely t_i children, corresponding to t_i pairwise distinct challenges. Every transcript corresponds to a unique path from the root node to a leaf node.

Definition 22 $((t_1, \dots, t_\mu)$ -Special Soundness [4]). A $(2\mu+1)$ -move public-coin protocol is $(t_1, \dots, t_\mu)$ -special sound if there exists a PPT algorithm Ext^s that on input an $(t_1, \dots, t_\mu)$ -tree of accepting transcripts outputs a witness w for a relation $\mathcal{R}$.

Special soundness implies standard knowledge soundness (Definition 10, Appendix B.4), known as the general forking lemma [4, Theorem 1].

Lemma 7 (Tree-Finding Algorithm [4]). Let $\mu, t_1, \dots, t_\mu \in \mathbb{N}$ be such that $T = \prod_{i=1}^\mu t_i$ can be upper bounded by a polynomial of $|u|$. Let $(\text{Setup}, \mathcal{P}, \mathcal{V})$ be a $(t_1, \dots, t_\mu)$ -special sound $(2\mu+1)$ -move interactive protocol, where $\mathcal{V}$ samples each challenge uniformly at random from a challenge set of size $p = O(2^\lambda) \geq \max_i(t_i)$. Given an adversary that produces an accepting transcript with probability ϵ , there exists a $(t_1, \dots, t_\mu)$ -tree finding algorithm **TREE** that makes an expected number of $T = \prod_{i=1}^\mu t_i$ queries to $\mathcal{P}$ and successfully outputs a $(t_1, \dots, t_\mu)$ -tree of accepting transcript with probability at least:

$$\epsilon - \frac{p^\mu - \prod_{i=1}^\mu (p - t_i + 1)}{p^\mu} \geq \epsilon - \frac{\sum_{i=1}^\mu (t_i - 1)}{p}.$$

Lemma 8 (General Forking Lemma [4]). Let $\mu, t_1, \dots, t_\mu \in \mathbb{N}$ be such that $T = \prod_{i=1}^\mu t_i$ can be upper bounded by a polynomial of $|u|$. Let $(\text{Setup}, \mathcal{P}, \mathcal{V})$ be a $(t_1, \dots, t_\mu)$ -special sound $(2\mu+1)$ -move interactive protocol for the relation $\mathcal{R}$, where $\mathcal{V}$ samples each challenge uniformly at random from a challenge set of size $p = O(2^\lambda) \geq \max_i(t_i)$. Then $(\text{Setup}, \mathcal{P}, \mathcal{V})$ is knowledge sound with knowledge soundness error:

$$\kappa_\lambda^{\text{ks}}(|u|) = \frac{p^\mu - \prod_{i=1}^\mu (p - t_i + 1)}{p^\mu} \leq \frac{\sum_{i=1}^\mu (t_i - 1)}{p}.$$

Here, $\kappa_\lambda^{\text{ks}}(|u|)$ is negligible in λ .

B.8 Fiat–Shamir Security

Definition 23 (Adaptive Fiat–Shamir Transformation). *Adaptive Fiat–Shamir transformation on an interactive argument of knowledge $(\text{Setup}, \mathcal{P}, \mathcal{V})$, denoted by $(\text{Setup}, \mathcal{P}_{\text{FS}}, \mathcal{V}_{\text{FS}})$ for statement u and witness w , is a two-round AoK where $\mathcal{P}_{\text{FS}}(\text{pp}, u, w)$ runs $\mathcal{P}(\text{pp}, u, w)$ but computes, using a hash function Hash:*

$$x_1 = \text{Hash}(u, \alpha_0), \quad x_i = \text{Hash}(x_{i-1}, \alpha_{i-1}), \quad i \in [2, \mu],$$

instead of querying the verifier. The output is transcript $\text{tr} = (\alpha_0, \alpha_1, \dots, \alpha_\mu)$. On input of statement u and proving transcript $\text{tr} = (\alpha_0, \alpha_1, \dots, \alpha_\mu)$, $\mathcal{V}_{\text{FS}}$ accepts if, for x_i as above $\mathcal{V}$ accepts the transcript $\text{tr} = (\alpha_0, \alpha_1, \dots, \alpha_\mu)$ on input u .

Lemma 9 (Fiat–Shamir Transformation of Special-Sound Σ -Protocol). *The Fiat–Shamir transformation of a special-sound Σ -protocol [5] Π is an AoK with knowledge soundness error:*

$$\kappa^{\text{ks-fs}} = (Q + 1) \cdot \kappa^{\text{ks}},$$

where κ^{ks} is the knowledge soundness error of the interactive protocol, and Q is the maximum queries an adversary can make to attack the hash function.

Lemma 10 (Fiat–Shamir Transformation of Round-by-Round Sound Interactive Protocol). *The Fiat–Shamir transformation using a correlation-intractable hash function of a round-by-round sound interactive protocol Π is an argument with computational soundness error:*

$$\kappa^{\text{cs-fs}} = Q \cdot \kappa^{\text{s}},$$

where κ^{s} is the soundness error of the interactive protocol, and Q is the maximum queries an adversary can make to attack the hash function.

C Additional Proofs for Atomic Operations

C.1 Addition, Scalar Multiplication, and Transpose

For matrix addition, we have the following equivalence:

$$\left\{ v = \vec{l}^\top (\mathbf{a} + \mathbf{b}) \vec{r} \right\} \iff \left\{ v = \vec{l}^\top \mathbf{a} \vec{r} + \vec{l}^\top \mathbf{b} \vec{r} \right\}.$$

Hence, the algorithm tuple $(+. \text{Prove}, +. \text{Eval}, +. \text{Link}, +. \text{Verify})$ is:

- $+. \text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b}) \rightarrow \pi$. The prover computes $v_L \leftarrow \vec{l}^\top \mathbf{a} \vec{r}$ and $v_R \leftarrow \vec{l}^\top \mathbf{b} \vec{r}$. It outputs $\pi \leftarrow (v_L, v_R)$.
- $+. \text{Eval}(\pi) \rightarrow (v_L, v_R)$. It outputs the two elements of π .
- $+. \text{Link}(\vec{l}, \vec{r}, \pi) \rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$. It assigns $\vec{l}_L \leftarrow \vec{l}$ and $\vec{r}_L \leftarrow \vec{r}$ to the left child, and $\vec{l}_R \leftarrow \vec{l}$ and $\vec{r}_R \leftarrow \vec{r}$ to the right child.
- $+. \text{Verify}(v, \pi, v_L, v_R) \rightarrow \text{true/false}$. It checks if $v \stackrel{?}{=} v_L + v_R$ and $\pi = (v_L, v_R)$.

For scalar multiplication, we know:

$$\left\{ v = \vec{l}^\top (b \cdot \mathbf{a}) \vec{r} \right\} \iff \left\{ v = b \cdot (\vec{l}^\top \mathbf{a} \vec{r}) \right\}.$$

The Evalyn algorithm tuple for scalar multiplication is designed similarly to addition.

For the transpose operation,

$$\left\{ v = \vec{l}^\top (\mathbf{a}^\top) \vec{r} \right\} \iff \left\{ v = \vec{r}^\top \mathbf{a} \vec{l} \right\}.$$

Hence, the algorithm tuple $((\cdot)^\top.\text{Prove}, (\cdot)^\top.\text{Eval}, (\cdot)^\top.\text{Link}, (\cdot)^\top.\text{Verify})$ is:

- $(\cdot)^\top.\text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b}) \rightarrow \pi$. It outputs $\pi \leftarrow (v)$.
- $(\cdot)^\top.\text{Eval}(\pi) \rightarrow (v_L)$. It returns the element of π .
- $(\cdot)^\top.\text{Link}(\vec{l}, \vec{r}, \pi) \rightarrow (\vec{l}_L, \vec{r}_L)$. It assigns $\vec{l}_L \leftarrow \vec{r}$ and $\vec{r}_L \leftarrow \vec{l}$.
- $(\cdot)^\top.\text{Verify}(v, \pi, v_L) \rightarrow \text{true/false}$. It checks if $v \stackrel{?}{=} v_L$ and $\pi = (v_L)$.

C.2 Concatenation and Truncation

For the matrix concatenation $(\mathbf{a} \parallel \mathbf{b})$, verification is performed using:

$$\begin{aligned} & \left\{ v = \vec{l}^\top (\mathbf{a} \parallel \mathbf{b}) \vec{r} \text{ where } \mathbf{a} \in \mathbb{F}^{m \times n_L}, \mathbf{b} \in \mathbb{F}^{m \times n_R} \right\} \\ & \iff \left\{ v = (\vec{l}^\top \mathbf{a} \vec{r}_{[0:n_L]}) + (\vec{l}^\top \mathbf{b} \vec{r}_{[n_L:n_L+n_R]}) \right\}. \end{aligned}$$

The Evalyn tuple $(\|\cdot\|.\text{Prove}, \|\cdot\|.\text{Eval}, \|\cdot\|.\text{Link}, \|\cdot\|.\text{Verify})$ is:

- $\|\cdot\|.\text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b}) \rightarrow \pi$. The prover computes $v_L \leftarrow \vec{l}^\top \mathbf{a} \vec{r}_{[0:n_L]}$ and $v_R \leftarrow \vec{l}^\top \mathbf{b} \vec{r}_{[n_L:n_L+n_R]}$. It outputs $\pi \leftarrow (v_L, v_R)$.
- $\|\cdot\|.\text{Eval}(\pi) \rightarrow (v_L, v_R)$. It outputs the two elements of π .
- $\|\cdot\|.\text{Link}(\vec{l}, \vec{r}, \pi) \rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$. It assigns: $\vec{l}_L \leftarrow \vec{l}$ and $\vec{r}_L \leftarrow \vec{r}_{[0:n_L]}$ to the left child and $\vec{l}_R \leftarrow \vec{l}$ and $\vec{r}_R \leftarrow \vec{r}_{[n_L:n_L+n_R]}$ to the right child.
- $\|\cdot\|.\text{Verify}(v, \pi, v_L, v_R) \rightarrow \text{true/false}$. It checks if $v \stackrel{?}{=} v_L + v_R$ and $\pi = (v_L, v_R)$.

Truncation $\mathbf{a}_{[0:n]}$ is verified through:

$$\left\{ v = \vec{l}^\top \mathbf{a}_{[0:n]} \vec{r} \right\} \iff \left\{ v = \vec{l}^\top \mathbf{a} (\vec{r} \parallel \vec{0}) \right\}.$$

The Evalyn tuple is thus similarly constructed.

C.3 Frobenius Inner Product

The Frobenius inner product for matrices generalizes the vector inner product:

$$\mathbf{a} \bullet \mathbf{b} := \text{vec}(\mathbf{a}) \bullet \text{vec}(\mathbf{b}),$$

where $\text{vec}(\mathbf{a})$ and $\text{vec}(\mathbf{b})$ are the vectorizations of matrices $\mathbf{a}$ and $\mathbf{b}$.

Consequently, Frobenius product proof is achieved through LiteBullet on $\text{vec}(\mathbf{a})$ and $\text{vec}(\mathbf{b})$. The algorithm tuple $(\bullet.\text{Prove}, \bullet.\text{Eval}, \bullet.\text{Link}, \bullet.\text{Verify})$ wraps the respective subalgorithms of LiteBullet as follows:

- $\bullet.\text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b}) \rightarrow \pi$. It calls $\pi \leftarrow \text{LiteBullet.Prove}(v, \text{vec}(\mathbf{a}), \text{vec}(\mathbf{b}))$ and outputs π .
- $\bullet.\text{Eval}(\pi) \rightarrow (v_L, v_R)$. It outputs the last two elements of π .
- $\bullet.\text{Link}(\vec{l}, \vec{r}, \pi) \rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$. Using the verifier's challenges $\mathbf{x}$ derived from π , it computes $\vec{\xi}_l \leftarrow \text{ts}(\mathbf{x}_l)$ and $\vec{\xi}_r \leftarrow \text{ts}(\mathbf{x}_r)$ from the left and right halves of $\mathbf{x}$. It outputs $\vec{l}_L \leftarrow \vec{\xi}_l^{-1}$, $\vec{r}_L \leftarrow \vec{\xi}_r^{-1}$, $\vec{l}_R \leftarrow \vec{\xi}_l$, and $\vec{r}_R \leftarrow \vec{\xi}_r$.
- $\bullet.\text{Verify}(v, \pi, v_L, v_R) \rightarrow \text{true/false}$. It returns **true** if v_L and v_R are the final two elements of π , and $\text{LiteBullet.Verify}(v, \pi)$ returns **true**.

C.4 Hadamard Product

The Hadamard product is verified via Frobenius inner product due to:

$$\begin{aligned} \left\{ v = \vec{l}^\top (\mathbf{a} \circ \mathbf{b}) \vec{r} \right\} &\iff \left\{ v = \mathbf{a} \bullet (\mathbf{b} \circ (\vec{l} \otimes \vec{r}^\top)) \right\} \\ &\iff \left\{ \exists \mathbf{d} \in \mathbb{F}^{n \times n}, \text{ s.t. } v = \mathbf{a} \bullet \mathbf{d} \wedge \mathbf{d} = \mathbf{b} \circ (\vec{l} \otimes \vec{r}^\top) \right\}. \end{aligned}$$

Here, an inner product proof for $v = \text{vec}(\mathbf{a}) \bullet \text{vec}(\mathbf{d})$ reduces the verification to checking $\hat{a} = \text{vec}(\mathbf{a}) \bullet \vec{\xi}^{-1}$ and $\hat{b} = \text{vec}(\mathbf{d}) \bullet \vec{\xi}$. As $\vec{\xi}$ is tensor-structured, we can decompose it into $\vec{\xi} = \vec{\xi}_l \otimes \vec{\xi}_r$. Thus, $\hat{a} = \text{vec}(\mathbf{a}) \bullet \vec{\xi}^{-1}$ is transformed into $\hat{a} = (\vec{\xi}_l^{-1})^\top \mathbf{a} \vec{\xi}_r^{-1}$. Similarly, $\hat{b} = \text{vec}(\mathbf{d}) \bullet \vec{\xi}$ is transformed into:

$$\hat{b} = \mathbf{d} \bullet (\vec{\xi}_l \otimes \vec{\xi}_r^\top) = [\mathbf{b} \circ (\vec{l} \otimes \vec{r}^\top)] \bullet [\vec{\xi}_l \otimes \vec{\xi}_r^\top] = (\vec{l} \circ \vec{\xi}_l)^\top \mathbf{b} (\vec{r} \circ \vec{\xi}_r).$$

Due to this equivalence, the Evalyn proof for Hadamard product is as follows:

- $\circ.\text{Prove}(v, \vec{l}, \vec{r}; \mathbf{a}, \mathbf{b}) \rightarrow \pi$. The prover computes $\mathbf{d} \leftarrow \mathbf{b} \circ (\vec{l} \otimes \vec{r}^\top)$. Then, it invokes $\pi \leftarrow \text{LiteBullet.Prove}(v, \text{vec}(\mathbf{a}), \text{vec}(\mathbf{d}))$ and outputs π .
- $\circ.\text{Eval}(\pi) \rightarrow (v_L, v_R)$. It outputs the last two elements of π .
- $\circ.\text{Link}(\vec{l}, \vec{r}, \pi) \rightarrow (\vec{l}_L, \vec{r}_L, \vec{l}_R, \vec{r}_R)$. Using the verifier's challenges $\mathbf{x}$ derived from π , it computes $\vec{\xi}_l \leftarrow \text{ts}(\mathbf{x}_l)$ and $\vec{\xi}_r \leftarrow \text{ts}(\mathbf{x}_r)$ from the left and right halves of $\mathbf{x}$. It outputs $\vec{l}_L \leftarrow \vec{\xi}_l^{-1}$, $\vec{r}_L \leftarrow \vec{\xi}_r^{-1}$, $\vec{l}_R \leftarrow \vec{l} \circ \vec{\xi}_l$, and $\vec{r}_R \leftarrow \vec{r} \circ \vec{\xi}_r$.
- $\circ.\text{Verify}(v, \pi, v_L, v_R) \rightarrow \text{true/false}$. It returns **true** if v_L and v_R are the last two elements of π , and $\text{LiteBullet.Verify}(v, \pi)$ returns **true**.

C.5 Inversion and Element-Wise Inversion

Matrix inversion is verified through matrix multiplication:

$$\{ \mathbf{c} = \mathbf{a}^{-1} \} \iff \{ \mathbf{I} = \mathbf{a} \mathbf{c} \}.$$

Element-wise inversion is verified through Hadamard product:

$$\left\{ \mathbf{c} = \frac{1}{\mathbf{a}} \right\} \iff \{ \mathbf{1} = \mathbf{a} \circ \mathbf{c} \}.$$

Given the commitment to $\mathbf{c}$, matrix inversion and element-wise inversion can be proved using Type-1 expressions.

C.6 Convolution

Convolution is a fundamental operation widely used in signal processing, image processing, and CNNs, defined as the sum of element-wise products of matrices shifted relative to each other. Formally, with zero-padding, the convolution of two matrices $\mathbf{a}, \mathbf{b} \in \mathbb{F}^{n \times n}$ is defined as

$$\text{Conv}(\mathbf{a}, \mathbf{b}) := \left(\sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} a_{ij} b_{(I-i), (J-j)} \right)_{I=1, J=1}^{2n-1, 2n-1} \in \mathbb{F}^{(2n-1) \times (2n-1)}.$$

To provide an $O(n^2)$ -prover proof for convolution, we exploit a key property connecting convolution to the FFT. Specifically, for $\forall y, y' \in \mathbb{F}^*$ and $\vec{\mathbf{l}} = (1, y, y^2, \dots, y^{2n-1})$, $\vec{\mathbf{r}} = (1, y', y'^2, \dots, y'^{2n-1})$, we have:

$$\vec{\mathbf{l}}^\top \text{Conv}(\mathbf{a}, \mathbf{b}) \vec{\mathbf{r}} = (\vec{\mathbf{l}}_{[0:n]}^\top \mathbf{a} \vec{\mathbf{r}}_{[0:n]}) \cdot (\vec{\mathbf{l}}_{[0:n]}^\top \mathbf{b} \vec{\mathbf{r}}_{[0:n]}) \quad (12)$$

This equivalence is derived as follows:

$$\begin{aligned} \vec{\mathbf{l}}^\top \text{Conv}(\mathbf{a}, \mathbf{b}) \vec{\mathbf{r}} &= \sum_{I=-\infty}^{\infty} \sum_{J=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} a_{ij} b_{(i-I), (j-J)} y^{-I} y'^{-J} \\ &= \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} a_{ij} y^{-i} y'^{-j} \sum_{I=-\infty}^{\infty} \sum_{J=-\infty}^{\infty} b_{(i-I), (j-J)} y^{-(i-I)} y'^{-(j-J)} \\ &= \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} a_{ij} y^{-i} y'^{-j} \sum_{I=-\infty}^{\infty} \sum_{J=-\infty}^{\infty} b_{IJ} y^{-I} y'^{-J} = (\vec{\mathbf{l}}_{[1:n]}^\top \mathbf{a} \vec{\mathbf{r}}_{[1:n]}) \cdot (\vec{\mathbf{l}}_{[1:n]}^\top \mathbf{b} \vec{\mathbf{r}}_{[1:n]}). \end{aligned}$$

Provided with an additional commitment to $\mathbf{c} = \text{Conv}(\mathbf{a}, \mathbf{b})$, Eq. (12) is proven as a Type-1 expression with the leaf nodes $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$.

C.7 Max Pooling

Given a matrix $\mathbf{a} \in \mathbb{F}^{n \times n}$ and a stride s , its max pooling produces a matrix $\mathbf{c} \in \mathbb{F}^{(n/s) \times (n/s)}$, where each entry in $\mathbf{c}$ is the maximum from an $s \times s$ block in $\mathbf{a}$. A boolean matrix $\mathbf{b} \in \{0, 1\}^{n \times n}$ identifies the location of each maximum in $\mathbf{a}$.

Verification of $\mathbf{c}$ as the max pooling of $\mathbf{a}$ involves using random challenges $y, y' \in \mathbb{F}^*$ and vectors $\vec{\mathbf{l}} := (1, y, \dots, y^{n/s-1})$ and $\vec{\mathbf{r}} := (1, y', \dots, y'^{n/s-1})$. The verification checks the following equations/inequalities:

$$\mathbf{a} - \mathbf{c} \otimes \mathbf{1}_s \leq \mathbf{0}, \quad \vec{\mathbf{l}}^\top \mathbf{c} \vec{\mathbf{r}} = (\vec{\mathbf{l}} \otimes \vec{\mathbf{1}}_s)^\top (\mathbf{a} \circ \mathbf{b}) (\vec{\mathbf{r}} \otimes \vec{\mathbf{1}}_s), \quad \mathbf{b} \circ \mathbf{b} = \mathbf{b},$$

where $\mathbf{1}_s$ is an $s \times s$ matrix of ones, and $\vec{\mathbf{1}}_s$ is an s -vector of ones.

In the above, the inequality is proven using lookup proofs.

C.8 Normalization

The normalization $\mathbf{c}^f \in \mathbb{R}^{n \times n}$ of a floating-point matrix $\mathbf{a}^f \in \mathbb{R}^{n \times n}$ is defined as:

$$\mathbf{c}^f = (\mathbf{a}^f - \bar{a}^f \cdot \mathbf{1}) / \sqrt{\frac{1}{n^2} \cdot \sum_{i=1, j=1}^{n, n} (a_{ij}^f - \bar{a}^f)^2},$$

where $\bar{a}^f$ is the average of $\mathbf{a}^f$.

The quantization method (Appendix D) represents a floating-point matrix $\mathbf{a}^f$ by an integer matrix $\mathbf{a} \in \mathbb{F}^{n \times n}$ and a scaling factor s_a such that:

$$\mathbf{a}^f = s_a \cdot \mathbf{a}.$$

In this context, the normalized matrix $\mathbf{c}$ is defined as:

$$\mathbf{c} = \lfloor (n^2 \mathbf{a} - \sum_{i=1, j=1}^{n, n} a_{ij}) / \sqrt{n^2 \cdot (\sum_{i=1, j=1}^{n, n} a_{ij}^2) - (\sum_{i=1, j=1}^{n, n} a_{ij})^2} \rfloor,$$

where “/” represents floating-point division and “ $\lfloor \cdot \rfloor$ ” denotes the operation of truncating the fractional part of the resultant floating-point matrix. This equation is transformed into:

$$\mathbf{c} = \lfloor \sqrt{[(n^2 \mathbf{a} - \mathbf{a} \bullet \mathbf{1}) \circ (n^2 \mathbf{a} - \mathbf{a} \bullet \mathbf{1})] / [n^2(\mathbf{a} \bullet \mathbf{a}) - (\mathbf{a} \bullet \mathbf{1})^2]} \rfloor.$$

Let $\mathbf{q} = \lfloor [(n^2 \mathbf{a} - \mathbf{a} \bullet \mathbf{1}) \circ (n^2 \mathbf{a} - \mathbf{a} \bullet \mathbf{1})] / [n^2(\mathbf{a} \bullet \mathbf{a}) - (\mathbf{a} \bullet \mathbf{1})^2] \rfloor$. Using division with remainders, this equation is verified through:

$$\begin{aligned} \mathbf{q} \cdot [n^2(\mathbf{a} \bullet \mathbf{a}) - (\mathbf{a} \bullet \mathbf{1})^2] + \mathbf{r} &= [(n^2 \mathbf{a} - \mathbf{a} \bullet \mathbf{1}) \circ (n^2 \mathbf{a} - \mathbf{a} \bullet \mathbf{1})], \\ \mathbf{0} \leq \mathbf{r} &< [n^2(\mathbf{a} \bullet \mathbf{a}) - (\mathbf{a} \bullet \mathbf{1})^2], \quad (\mathbf{c} - \mathbf{1}) \circ (\mathbf{c} - \mathbf{1}) < \mathbf{q} \leq \mathbf{c} \circ \mathbf{c}. \end{aligned}$$

Here, the inequalities are proven using lookup proofs.

C.9 Other NN Operations

Although this appendix does not exhaustively cover all NN operations, we have reviewed every operation defined in the open neural network exchange (ONNX) format [68]. Apart from the control-flow operation **Loop**, all NN operations can be represented as Type-1 expressions via linear algebra transformations. The **Loop** operation can itself be accommodated by unrolling the loops into the DAG, ensuring that all NN computations ultimately expand into a DAG consisting exclusively of Type-1 operations.

D Rounding in Quantized Neural Networks

Handling floating-point arithmetic in zkSNARKs is challenging. General-purpose floating-point proofs for arithmetic circuits [40] incur significant overhead, as they require explicit floating-point conversion for each scalar gate.

zkLLM [74] permits small quantization errors, introducing vulnerabilities under adversarial attacks on NNs [75], where minor perturbations can drastically alter predictions or introduce backdoors into proofs.

Therefore, we adopt quantized NNs [29,37,82,83,86], which convert floating-point matrix computations into integer ones with scaling factors while preserving satisfactory model performance. Quantization ensures deterministic inference via integer computations, avoiding approximations in proofs.¹¹

Quantized NNs fit Evalyn. Quantization aligns the proved statement with the deployed model and is widely supported by production toolchains (*e.g.*, NVIDIA TensorRT) that convert floating-point models to INT8 with per-tensor or per-channel scales. As noted in a joint NVIDIA–Arm–Intel whitepaper [65, Section 1], “*For inference fixed-point INT8 representation is a popular option. In some cases even INT8 inference can encounter challenges in maintaining the accuracy required for application deployment.*” Moreover, quantization-aware training methods (*e.g.*, [86]) can recover accuracy while retaining efficiency.

In quantized NNs, each floating-point matrix is represented as an integer matrix combined with a scaling factor. Rounding is performed by constraining matrix values within a predefined numerical range. For two $n \times n$ matrices of 8-bit integers, $\mathbf{a}$ and $\mathbf{b}$, with an 8-bit scale s , the resulting matrix $\mathbf{c}$ is computed as

$$\mathbf{c} = \min\left(\max(\lfloor 2^{-7} \cdot s \cdot \mathbf{ab} \rfloor, -127 \cdot \mathbf{1}), 127 \cdot \mathbf{1}\right)$$

where $\mathbf{1}$ is an $n \times n$ all-ones matrix. The operator $\lfloor \cdot \rfloor$ applies element-wise floor, and min and max act element-wise.

To verify the correctness of this rounding operation, we introduce error matrices $\mathbf{e}_1$ and $\mathbf{e}_2$ that capture the overflow digits and the low-order digits, respectively, and a sign matrix $\boldsymbol{\iota} \in \{-1, 1\}^{n \times n}$ arising from a base 2^7 decomposition of $s \cdot \mathbf{ab}$. Verification is conducted using the following constraints:

$$2^{14} \cdot \mathbf{e}_1 + 2^7 \cdot (\boldsymbol{\iota} \circ \mathbf{c}) + \mathbf{e}_2 = \boldsymbol{\iota} \circ (\mathbf{ab}) \wedge \boldsymbol{\iota} \circ \boldsymbol{\iota} = \mathbf{1} \wedge \mathbf{0} \leq \mathbf{e}_1, \mathbf{e}_2, (\boldsymbol{\iota} \circ \mathbf{c}) < 2^7 \cdot \mathbf{1}.$$

The required inequalities in these Type-1 expressions are ensured using lookup proofs, verifying that entries of $\mathbf{e}_1$, $\mathbf{e}_2$, and $(\boldsymbol{\iota} \circ \mathbf{c})$ lie in $\{0, 1, \dots, 127\}$.

These Type-1 expressions and lookup proofs fit naturally into Evalyn.

¹¹ zkLLM [74, Appendix B] claims that quantized NNs are not applicable to VML due to an alleged incompatibility between finite-field computations and floating-point division. This assertion is inaccurate: floating-point division can be represented using integer division with quotient and remainder, together with appropriate range proofs.

Algorithm 4. Bulletproofs Inner-Product Protocol

Input: Public input: $C \in \mathbb{G}$; $U \in \mathbb{G}$, $\vec{G}, \vec{H} \in \mathbb{G}^m$.
 $\mathcal{P}$'s private input: $\vec{a}, \vec{b} \in \mathbb{F}^m$.
Output: $\mathcal{P}$: $\pi := (L_{(1)}, \dots, L_{(\log_2 m)}, R_{(1)}, \dots, R_{(\log_2 m)}, \hat{a}, \hat{b})$; $\mathcal{V}$: true or false.

- 1 **foreach** $j \in \{1, \dots, \log_2 m\}$ **do**
- 2 $\mathcal{P}$ computes;
- 3 $(\vec{a}_{\text{LH}} \parallel \vec{a}_{\text{RH}}) \xleftarrow{\text{half}} \vec{a}$; $(\vec{b}_{\text{LH}} \parallel \vec{b}_{\text{RH}}) \xleftarrow{\text{half}} \vec{b}$;
- 4 $(\vec{G}_{\text{LH}} \parallel \vec{G}_{\text{RH}}) \xleftarrow{\text{half}} \vec{G}$; $(\vec{H}_{\text{LH}} \parallel \vec{H}_{\text{RH}}) \xleftarrow{\text{half}} \vec{H}$;
- 5 $L_{(j)} \leftarrow (\vec{a}_{\text{LH}} \bullet \vec{G}_{\text{RH}}) \oplus (\vec{b}_{\text{RH}} \bullet \vec{H}_{\text{LH}}) \oplus (\vec{a}_{\text{LH}} \bullet \vec{b}_{\text{RH}}) U \in \mathbb{G}$;
- 6 $R_{(j)} \leftarrow (\vec{a}_{\text{RH}} \bullet \vec{G}_{\text{LH}}) \oplus (\vec{b}_{\text{LH}} \bullet \vec{H}_{\text{RH}}) \oplus (\vec{a}_{\text{RH}} \bullet \vec{b}_{\text{LH}}) U \in \mathbb{G}$;
- 7 $\mathcal{P} \rightarrow \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}$;
- 8 $\mathcal{V} \rightarrow \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathbb{F}^*$;
- 9 $\mathcal{P}$ updates;
- 10 $\vec{a} \leftarrow \vec{a}_{\text{LH}} + x_{(j)}^{-1} \vec{a}_{\text{RH}} \in \mathbb{F}^{m/2^j}$ $\vec{b} \leftarrow \vec{b}_{\text{LH}} + x_{(j)} \vec{b}_{\text{RH}} \in \mathbb{F}^{m/2^j}$;
- 11 $\vec{G} \leftarrow \vec{G}_{\text{LH}} \oplus x_{(j)} \vec{G}_{\text{RH}} \in \mathbb{G}^{m/2^j}$ $\vec{H} \leftarrow \vec{H}_{\text{LH}} \oplus x_{(j)}^{-1} \vec{H}_{\text{RH}} \in \mathbb{G}^{m/2^j}$;
- 12 $\mathcal{P} \rightarrow \mathcal{V}$: $\hat{a} \leftarrow \vec{a}[1] \in \mathbb{F}$; $\hat{b} \leftarrow \vec{b}[1] \in \mathbb{F}$;
- 13 $\mathcal{V}$ computes $\vec{\xi}, \vec{\xi}^{-1} \in \mathbb{F}^m$ from $x_{(1)}, \dots, x_{(\log_2 m)}$;
- 14 **return** $C \oplus \bigoplus_{j=1}^{\log_2 m} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)}) \stackrel{?}{=} \hat{a} \hat{b} U \oplus \hat{a} (\vec{\xi} \bullet \vec{G}) \oplus \hat{b} (\vec{\xi}^{-1} \bullet \vec{H})$ to $\mathcal{V}$;

E Recapitulation of Prior Work

E.1 Bulletproofs

Bulletproofs [21] offer an argument of knowledge for committed vectors $\vec{a}, \vec{b} \in \mathbb{F}^m$, featuring a communication complexity of $2 \log_2 m$ group elements and 2 field elements for the following relation $\mathcal{R}_{\text{ip}}$:

$$\mathcal{R}_{\text{ip}} = \left\{ \left(C \in \mathbb{G}; U \in \mathbb{G}; \begin{array}{l} \vec{a} \in \mathbb{F}^m, \\ \vec{G} \in \mathbb{G}^m, \vec{H} \in \mathbb{G}^m \end{array} \right) : \left(\vec{a} \in \mathbb{F}^m, \vec{b} \in \mathbb{F}^m \right) \middle| \left(C = (\vec{a} \bullet \vec{b}) U \oplus \langle \vec{a}, \vec{G} \rangle \oplus \langle \vec{b}, \vec{H} \rangle \right) \right\},$$

where $\oplus$ represents group operations in the group $\mathbb{G}$.

Algorithm 4 is an optimized version of Bulletproofs [42]. For any vector $\vec{a} \in \mathbb{F}^\ell$ with even ℓ , we write $(\vec{a}_{\text{LH}} \parallel \vec{a}_{\text{RH}}) \xleftarrow{\text{half}} \vec{a}$ to mean $\vec{a}_{\text{LH}} = (a_1, \dots, a_{\ell/2})$ and $\vec{a}_{\text{RH}} = (a_{\ell/2+1}, \dots, a_\ell)$.

Bulletproofs satisfy $(4, 4, \dots, 4)$ -special soundness because the witness can be extracted from a tree of accepting transcripts with 4 distinct challenges at each node [21, Appendix B].

E.2 The Dory Matrix Commitment Scheme

A two-tier commitment [47] to a matrix $\mathbf{a} \in \mathbb{Z}_p^{n \times n}$ is computed as follows:

- **First-tier commitment:** Apply the Pedersen vector commitment to each column or row of $\mathbf{a}$ using the base vector $\vec{\mathbf{G}} \in \mathbb{G}_1^n$. This results in $\vec{\mathbf{A}} \in \mathbb{G}_1^n$, computed as:

$$\vec{\mathbf{A}} = \vec{\mathbf{G}}^\top \mathbf{a}.$$

- **Second-tier commitment:** Commit to $\vec{\mathbf{A}}$ using AFGHO commitments [2], resulting in $C_a \in \mathbb{G}_T$:

$$C_a = \vec{\mathbf{A}}_G \bullet \vec{\mathbf{H}} = \vec{\mathbf{G}}^\top \mathbf{a} \vec{\mathbf{H}}.$$

We can add a hiding factor $\tilde{a} \in \mathbb{Z}_p$ with the base $\tilde{U} \in \mathbb{G}_T$ to the two-tier commitment as follows:

$$C_a = \vec{\mathbf{G}}^\top \mathbf{a} \vec{\mathbf{H}} \oplus \tilde{a} \tilde{U}.$$

The Dory [59] matrix commitment scheme provides a proof for the vector-matrix-vector product relation as follows:

$$\mathcal{R}_{\text{dory}} = \left\{ C_a \in \mathbb{G}_T, v \in \mathbb{F}, \vec{\mathbf{l}}, \vec{\mathbf{r}} \in \mathbb{F}^n : \mathbf{a} \in \mathbb{F}^{n \times n} \mid v = \vec{\mathbf{l}}^\top \mathbf{a} \vec{\mathbf{r}} \wedge C_a = \vec{\mathbf{G}}^\top \mathbf{a} \vec{\mathbf{H}} \right\}.$$

This proof is achieved by proving the following generalized inner product relations within pairing groups:

$$\exists \vec{\mathbf{A}} \in \mathbb{G}_1^n, \quad \hat{A} = \vec{\mathbf{A}} \bullet \vec{\mathbf{r}} \wedge C_a = \vec{\mathbf{A}} \bullet \vec{\mathbf{H}},$$

$$\exists \vec{\mathbf{a}}_r \in \mathbb{F}^n, \quad v = \vec{\mathbf{a}}_r \bullet \vec{\mathbf{l}} \wedge \hat{A} = \vec{\mathbf{a}}_r \bullet \vec{\mathbf{G}}.$$

These inner products are verified using a folding technique similar to that used in Bulletproofs.

Dory is equivalent to Bulletproofs in $\mathbb{G}_T$, as the two-tier commitment can be expressed as an inner product:

$$C_a = \vec{\mathbf{G}}^\top \mathbf{a} \vec{\mathbf{H}} = \text{vec}(\mathbf{a}) \bullet (\vec{\mathbf{G}} \otimes \vec{\mathbf{H}}).$$

In particular, Dory inherits the special soundness property from Bulletproofs, making it a $2 \log_2 n$ -round $(4, 4, \dots, 4)$ -special-sound protocol.

E.3 Sumcheck on MLEs

State-of-the-art zkSNARKs, *e.g.*, [8,15,31,32,33,45,71,72,89], commonly rely on the multilinear extensions and the sumcheck protocol. Below is a brief overview of how these protocols work.

Definition 24 (MLE [45,71]). For a matrix $\mathbf{a} = \{a_{ij}\} \in \mathbb{F}^{m \times n}$, its MLE is the following multilinear polynomial on $\mu = \log_2 m + \log_2 n$ variables $\mathbf{x} \in \mathbb{F}^\mu$:

$$\begin{aligned} \psi_a(\mathbf{x}) := & \sum_{i=1}^m \sum_{j=1}^n [a_{ij} \prod_{\tau=1}^{\log_2 m} (i_\tau x_\tau + (1 - i_\tau)(1 - x_\tau)) \\ & \cdot \prod_{\tau=1}^{\log_2 n} (j_\tau x_{\tau + \log_2 m} + (1 - j_\tau)(1 - x_{\tau + \log_2 m}))]. \end{aligned}$$

i_τ and j_τ represent the τ -th bit of i and j in little-endian format, respectively.

Algorithm 5. Sumcheck Protocol

Input: $\sigma \in \mathbb{F}; \psi \in \mathbb{F}[X^\mu]$
Output: true or false

- 1 $\sigma_{(0)} \leftarrow \sigma \in \mathbb{F};$
- 2 **foreach** $j \in \{1, \dots, \mu\}$ **do**
- 3 $\mathcal{P}$ computes a univariate polynomial of degree at most d :
 $h^{(j)}(x) \leftarrow \sum_{(x_{j+1}, \dots, x_\mu) \in \{0,1\}^{\mu-j}} \psi(x_{(1)}, \dots, x_{(j-1)}, x, x_{j+1}, \dots, x_\mu)$, with
 $x_{(1)}, \dots, x_{(j-1)}$ denoting challenges from the previous $(j-1)$ iterations;
- 4 $\mathcal{P} \rightarrow \mathcal{V}: h^{(j)}(x) \in \mathbb{F}[X];$
- 5 $\mathcal{V}: \text{if } \neg(\sigma_{(j-1)} \stackrel{?}{=} h^{(j)}(0) + h^{(j)}(1)) \text{ then return false};$
- 6 $\mathcal{V} \rightarrow \mathcal{P}: x_{(j)} \stackrel{\$}{\leftarrow} \mathbb{F}^*;$
- 7 $\mathcal{P}$ and $\mathcal{V}$ compute: $\sigma_{(j)} \leftarrow h^{(j)}(x_{(j)}) \in \mathbb{F};$
- 8 $\mathcal{P}$ runs additional protocols to convince $\mathcal{V}$ that $\sigma_{(\mu)} = \psi(x_{(1)}, x_{(2)}, \dots, x_{(\mu)})$;

The MLE thus defined satisfies, for $\vec{i} \in \{0, 1\}^{\log_2 m}$ and $\vec{j} \in \{0, 1\}^{\log_2 n}$,

$$\psi_a(\vec{i}, \vec{j}) = a_{ij}, \quad \forall i \in [m], j \in [n],$$

where $\vec{i}$ and $\vec{j}$ are the binary representations of the indices i and j , respectively.

Sumcheck protocol. The sumcheck protocol [64] is a proof system rather than an AoK; however, when combined with a PCS, it yields a zkSNARK.

The sumcheck (Algorithm 5) operates on a degree- d and μ -variate polynomial $\psi(\mathbf{x}) \in \mathbb{F}[X^\mu]$ bound to $C_\psi \in \mathbb{C}$ using a PCS pc . It verifies the sum of the polynomial $\psi(\mathbf{x})$ over the boolean hypercube $\mathbf{x} \in \{0, 1\}^\mu$ evaluates to a given value $\sigma \in \mathbb{F}$. Specifically, the sumcheck protocol verifies $\mathcal{R}_{\text{sumcheck}}$ as follows:

$$\mathcal{R}_{\text{sumcheck}} = \left\{ (\sigma \in \mathbb{F}, \psi \in \mathbb{F}[X^\mu]) \mid \left(\sigma = \sum_{\mathbf{x} \in \{0,1\}^\mu} \psi(\mathbf{x}) \right) \right\}.$$

Sumcheck for matrix multiplication. Thaler's protocol [76] verifies matrix multiplication based on the following equivalence on the MLEs of public input matrices [76, Lemma 5]:

$$\begin{aligned} \{ \mathbf{c} = \mathbf{a}\mathbf{b} \mid \text{where } \mathbf{c} \in \mathbb{F}^{n \times n}, \mathbf{a} \in \mathbb{F}^{n \times n}, \mathbf{b} \in \mathbb{F}^{n \times n} \} &\iff \\ \{ \forall \mathbf{y}_a \in \mathbb{F}^{\log_2 n}, \mathbf{y}_b \in \mathbb{F}^{\log_2 n}, \psi_c(\mathbf{y}_a, \mathbf{y}_b) = \sum_{\mathbf{x} \in \{0,1\}^{\log_2 n}} \psi_a(\mathbf{y}_a, \mathbf{x}) \psi_b(\mathbf{x}, \mathbf{y}_b) \}, & \end{aligned}$$

where $\psi_a(\cdot)$, $\psi_b(\cdot)$, and $\psi_c(\cdot)$ are the MLEs of matrices $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$, respectively.

In Thaler's protocol, the verifier sends random $\mathbf{y}_a$ and $\mathbf{y}_b$, and the prover responds with $\sigma = \psi_c(\mathbf{y}_a, \mathbf{y}_b) \in \mathbb{F}$. The polynomial $g(\mathbf{x}) := \psi_a(\mathbf{y}_a, \mathbf{x}) \psi_b(\mathbf{x}, \mathbf{y}_b)$ is a degree-2 multivariate polynomial in $\log_2 n$ variables. Therefore, matrix multiplication can be verified via a sumcheck protocol [64] on the relation:

$$\sigma = \sum_{\mathbf{x} \in \{0,1\}^{\log_2 n}} g(\mathbf{x}).$$

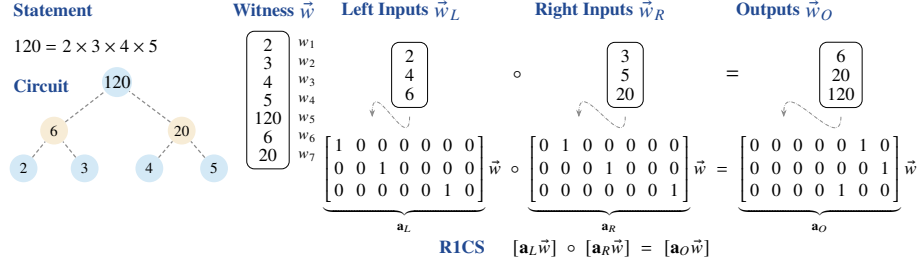


Fig. 6. Example of R1CS relation encoding an arithmetic circuit's structure using sparse matrices $\mathbf{a}_L$, $\mathbf{a}_R$, and $\mathbf{a}_O$

The sumcheck protocol is efficient. Given $\mathbf{y}_a$ and $\mathbf{y}_b$, the prover uses $O(n^2)$ multiplications to compute $\psi_a(\mathbf{y}_a, \mathbf{x})$ and $\psi_b(\mathbf{x}, \mathbf{y}_b)$, followed by $O(n)$ multiplications in sumcheck to reduce matrix multiplication to MLE evaluations.

To transform Thaler's protocol into a zkSNARK compatible with secret input matrices, we combine it with a PCS to verify the evaluations of $\psi_c(\mathbf{y}_a, \mathbf{y}_b)$, $\psi_a(\mathbf{y}_a, \mathbf{x})$, and $\psi_b(\mathbf{x}, \mathbf{y}_b)$. Time complexity of the zkSNARK version of Thaler's protocol depends on the efficiency of the PCS.

E.4 Rank-1 Constraint System (R1CS)

R1CS [7] is a well-known intermediate representation for arithmetic circuit satisfiability, which proves the existence of assignments $\vec{w}$ to all input, output, and multiplication gates in an arithmetic circuit such that all constraints are satisfied. Fig. 6 illustrates the circuit for the statement $120 = 2 \times 3 \times 4 \times 5$. The prover can assign 6 and 20 to intermediate gates, and the verifier checks constraints $2 \times 3 = 6$, $4 \times 5 = 20$, and $6 \times 20 = 120$.

R1CS reduces these three products to vector Hadamard (element-wise) product $\circ$ using three vectors $\vec{w}_L = \mathbf{a}_L \vec{w}$, $\vec{w}_R = \mathbf{a}_R \vec{w}$, and $\vec{w}_O = \mathbf{a}_O \vec{w}$ to represent the left inputs, right inputs, and outputs of the gates, derived from the witness $\vec{w}$ using sparse “rotating” matrices $\{\mathbf{a}_i\}_{i \in \{L, R, O\}}$, respectively (Fig. 6). For example, $(2 \ 4 \ 6)^\top \circ (3 \ 5 \ 20)^\top = (6 \ 20 \ 120)^\top$. Entries at position (3, 6) in $\mathbf{a}_L$ and (1, 6) in $\mathbf{a}_O$ indicate that the left input of the 3-rd multiplication gate and the output of the 1-st gate both correspond to the 6-th entry in $\vec{w}$. Substituting $\{\vec{w}_i\}_{i \in \{L, R, O\}}$ into the Hadamard product equation yields the R1CS relation:

$$[\mathbf{a}_L \vec{w}] \circ [\mathbf{a}_R \vec{w}] = [\mathbf{a}_O \vec{w}].$$

The sparse matrices $\mathbf{a}_i, i \in \{L, R, O\}$ uniquely specify the circuit.

General-purpose zkSNARKs include a pre-processing phase that commits to these sparse matrices. These commitments are crucial for fast verification by allowing the verifier to bypass reading the entire circuit, while also preserving circuit confidentiality.

E.5 Neural Network (NN)

Neural networks are a class of machine learning models designed to approximate complex functions through layered compositions of linear and non-linear transformations. An NN is structured as a sequence of layers, each mapping an input vector to an output vector.

Linear Transformation. In the i -th layer, an input vector $\mathbf{X}_{i-1}$ is multiplied by a weight matrix $\mathbf{W}_i$ and shifted by a bias vector $\mathbf{B}_i$:

$$\mathbf{Z}_i = \mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i, \quad i \in [D],$$

where D is the NN depth. Different NN architectures employ different linear operators. For example, CNNs replace matrix multiplication with convolution as follows:

$$\mathbf{Z}_i = \text{Conv}(\mathbf{W}_i, \mathbf{X}_{i-1}) + \mathbf{B}_i, \quad i \in [D].$$

Non-Linear Activation. The result $\mathbf{Z}_i$ is then passed through a non-linear activation function σ (e.g., sigmoid, ReLU) to obtain the layer output $\mathbf{X}_i$:

$$\mathbf{X}_i = \sigma(\mathbf{Z}_i), \quad i \in [D].$$

Stacking many such layers yields a highly expressive function $\mathbf{Y} = f(\mathbf{X})$, where $\mathbf{X} = \mathbf{X}_0$ and $\mathbf{Y} = \mathbf{X}_D$. The NN parameters $(\mathbf{W}_i, \mathbf{B}_i)$ are learned through training by minimizing a loss function.

Inference vs. Training. *Inference* refers to feedforward evaluation of the NN output $\mathbf{Y}$ on a given input $\mathbf{X}$ with fixed parameters. *Training* refers to updating NN parameters using gradients computed via backpropagation through all layers.

Scalability. Modern NNs typically contain millions to billions of parameters. While this scale enables high expressive power, it also makes training and inference computationally intensive, even in non-cryptographic computation.

Quantized NN. Quantization was introduced as an effective means to improve NN inference efficiency by reducing the numerical precision of NN parameters and computations (e.g., from 32-bit floating point to 8-bit integers). It substantially decreases memory usage and computational cost while typically preserving accuracy within acceptable bounds. Practically, 8-bit quantization has become prevailing for NN inference.

F Security Proofs

F.1 Proof of Theorem 1

Proof. Given public $\vec{\mathbf{a}}$ and $\vec{\mathbf{b}}$, we define $\vec{\mathbf{a}}_j$ and $\vec{\mathbf{b}}_j$ recursively using the initial j random challenges as follows:

$$\begin{aligned} \vec{\mathbf{a}}_{(0)} &\leftarrow \vec{\mathbf{a}}, \quad \vec{\mathbf{b}}_{(0)} \leftarrow \vec{\mathbf{b}}, \\ \vec{\mathbf{a}}_{(j)} &\leftarrow \vec{\mathbf{a}}_{(j-1)\text{LH}} + x_{(j)}^{-1} \vec{\mathbf{a}}_{(j-1)\text{RH}}, \quad \vec{\mathbf{b}}_{(j)} \leftarrow \vec{\mathbf{b}}_{(j-1)\text{LH}} + x_{(j)} \vec{\mathbf{b}}_{(j-1)\text{RH}}, \quad j \in [\log_2 m]. \end{aligned}$$

Moreover, the state functions are defined as follows:

$$\text{State}_0 \leftarrow (c = \vec{a} \bullet \vec{b}), \text{ State}_j \leftarrow \left(c + \sum_{t=1}^j (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}) = \vec{a}_{(j)} \bullet \vec{b}_{(j)} \right).$$

Perfect completeness. If $c = \vec{a} \bullet \vec{b}$, then $\text{State}_0 = \text{true}$.

Given $\text{State}_{j-1} = \text{true}$, we have $\text{State}_j = \text{true}$ since:

$$\begin{aligned} & \vec{a}_{(j)} \bullet \vec{b}_{(j)} \\ &= (\vec{a}_{(j-1)\text{LH}} + x_{(j)}^{-1} \vec{a}_{(j-1)\text{RH}}) \bullet (\vec{b}_{(j-1)\text{LH}} + x_{(j)} \vec{b}_{(j-1)\text{RH}}) \\ &= \vec{a}_{(j-1)} \bullet \vec{b}_{(j-1)} + x_{(j)} \vec{a}_{(j-1)\text{LH}} \bullet \vec{b}_{(j-1)\text{RH}} + x_{(j)}^{-1} \vec{a}_{(j-1)\text{RH}} \bullet \vec{b}_{(j-1)\text{LH}} \\ &= c + \sum_{t=1}^{j-1} (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}) + x_{(j)} \iota_{(i)} + x_{(j)}^{-1} \rho_{(j)} \\ &= c + \sum_{t=1}^j (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}). \end{aligned}$$

By mathematical induction, $\text{State}_\mu = \text{true}$, which is the verification check of LiteBullet proof.

Soundness. If $c \neq \vec{a} \bullet \vec{b}$, then $\text{State}_0 = \text{false}$.

Given $\text{State}_{j-1} = \text{false}$, we define:

$$\gamma := \vec{a}_{(j-1)} \bullet \vec{b}_{(j-1)} - c - \sum_{t=1}^{j-1} (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}) \neq 0.$$

Then,

$$\begin{aligned} & \vec{a}_{(j)} \bullet \vec{b}_{(j)} - c - \sum_{t=1}^j (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}) \\ &= (\vec{a}_{(j-1)\text{LH}} + x_{(j)}^{-1} \vec{a}_{(j-1)\text{RH}}) \bullet (\vec{b}_{(j-1)\text{LH}} + x_{(j)} \vec{b}_{(j-1)\text{RH}}) - c - \sum_{t=1}^j (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}) \\ &= \vec{a}_{(j-1)} \bullet \vec{b}_{(j-1)} - c - \sum_{t=1}^{j-1} (x_{(i)} \iota_{(i)} + x_{(i)}^{-1} \rho_{(i)}) \\ &\quad + x_{(j)} (\vec{a}_{(j-1)\text{LH}} \bullet \vec{b}_{(j-1)\text{RH}} - \iota_{(j)}) + x_{(j)}^{-1} (\vec{a}_{(j-1)\text{RH}} \bullet \vec{b}_{(j-1)\text{LH}} - \rho_{(j)}) \\ &= \gamma + x_{(j)} (\vec{a}_{(j-1)\text{LH}} \bullet \vec{b}_{(j-1)\text{RH}} - \iota_{(j)}) + x_{(j)}^{-1} (\vec{a}_{(j-1)\text{RH}} \bullet \vec{b}_{(j-1)\text{LH}} - \rho_{(j)}). \end{aligned}$$

Since $\gamma \neq 0$ and $x_{(j)}$ is uniformly chosen from the $p - 1$ nonzero elements of $\mathbb{F}^*$, the Schwartz–Zippel lemma implies that the probability of this equation equaling zero is $2/(p - 1)$. Consequently, the probability that $\text{State}_j = \text{true}$ is $2/(p - 1)$, which is the round-by-round soundness error.

According to Lemma 6, the soundness error is given by $2 \log_2 m / (p - 1)$. $\square$

F.2 Proof of Claim 1

Proof. According to the Schwartz–Zippel lemma, the probability that the inner product relation holds while not all projections do is at most $k/(p-1)$. The soundness errors for the inner product proofs are $\frac{2 \log_2(kn^2)}{p-1}$ and $\frac{2 \log_2(n^2)}{p-1}$, respectively. These total soundness errors can be derived using Lemma 6. $\square$

F.3 Proof of Claim 2

Proof. Since, for each operation, the target statement is equivalent to the check relation, we argue by the two implications.

($\Rightarrow$) If the original statement holds, then the check relation holds as well, and the verifier accepts by the perfect completeness of that relation and of LiteBullet.

($\Leftarrow$) If the original statement fails, then the check relation fails. In this case the verifier outputs **false** except with probability at most $\frac{4 \log_2 n}{p-1}$ for multiplication, Hadamard product, and Frobenius inner product by the soundness of LiteBullet; for the remaining Type-1 operations the verifier rejects with certainty.

Each atomic Evalyn uses at most one LiteBullet proof on n^2 -dimensional vectors, with soundness error $\frac{4 \log_2 n}{p-1}$ by Theorem 1. $\square$

F.4 Proof of Claim 3

Proof. For each additional Type-2 operation in a DAG, committing to its output matrix adds one partition to the DAG. Furthermore, each Type-2 operation is proven using a constant number of Type-1 expressions, resulting in a constant number of additional subgraphs.

F.5 Proof of Theorem 2

Proof. *Perfect completeness* stems from that of the atomic proofs.

Soundness is established by demonstrating a negligible soundness error.

For a DAG with a single root, if $f(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}$, the Schwartz–Zippel lemma implies that the probability of $\vec{\mathbf{l}}^\top f(\mathbf{a}_1, \dots, \mathbf{a}_k) \vec{\mathbf{r}} = 0$ for random tensor-structured $\vec{\mathbf{l}}$ and $\vec{\mathbf{r}}$ is at most $1/(p-1)$.

Let the $M' := M + k$ nodes in the DAG be indexed by $i \in [M']$, where $i \in [k]$ corresponds to the leaf nodes, and $i = [M' - J + 1, M']$ corresponds to the J root nodes. Define $b_i := (v_i = \vec{\mathbf{l}}_i^\top \mathbf{a}_i \vec{\mathbf{r}}_i), i \in [M']$. The proof generated by $\textcircled{*}.\text{Prove}$ at the i -th node is denoted by π_i . If the proof is accepted, then for all nodes, π_i is accepted, all BatchProj proofs at Line 10 returns **true**, and the ConcatProj proof at Line 25 returns **true**.

According to Claim 1, if $\exists i \in [k]$ such that $b_i = \text{false}$, then the probability that the ConcatProj proof returns **true** is at most $\frac{k + 4 \log_2 n + 2 \log_2 k}{p-1}$. So:

$$\Pr(\mathcal{V} \text{ accepts} \mid \exists i \in [k], b_i = \text{false}) \leq \frac{k + 4 \log_2 n + 2 \log_2 k}{p-1}.$$

If $b_j = \text{false}$ for some $j \in [M' - J + 1, M']$ while $b_i = \text{true}$ for all $i \in [k]$, there must exist a node $j \in [k + 1, M' - J]$ where $b_j = \text{false}$ and $b_{j_L} = b_{j_R} = \text{true}$, where j_L and j_R are the left and right children of j . Let the messages passed from node j to j_L and j_R be $(v_L, \vec{l}_L, \vec{r}_L)$ and $(v_R, \vec{l}_R, \vec{r}_R)$, respectively. During traversal, **BatchProj** proofs may be used to combine multiple projections, so v_L and v_R may not necessarily equal v_{j_L} and v_{j_R} .

For each node j , assume it has k_j parents. It then receives k_j messages $(v'_{j\tau}, \vec{l}'_{j\tau}, \vec{r}'_{j\tau})$, $\tau \in [k_j]$ from its parent nodes. Let **Event_j** denote the event where $\exists \tau \in [k_j], v'_{j\tau} \neq \vec{l}'_{j\tau}^\top \mathbf{a}_j \vec{r}'_{j\tau}$, but $b_j = \text{true}$. According to Claim 1,

$$\Pr(\mathcal{V} \text{ accepts} \wedge \text{Event}_j) \leq \frac{k_{j_L} + 4 \log_2 n}{p - 1}.$$

Conditional on the event $\neg(\bigvee_{i=1}^{M'-J} \text{Event}_i)$, $b_{j_L} = b_{j_R} = \text{true}$ implies $v_L = \vec{l}_L^\top \mathbf{a}_{j_L} \vec{r}_L \wedge v_R = \vec{l}_R^\top \mathbf{a}_{j_R} \vec{r}_R$. Hence:

$$\begin{aligned} & \Pr(\mathcal{V} \text{ accepts} \wedge b_{j_L} = b_{j_R} = \text{true} \wedge \neg(\bigvee_{i=1}^{M'-J} \text{Event}_i) \mid b_j = \text{false}) \\ & \leq \Pr(\mathcal{V} \text{ accepts} \wedge v_L = \vec{l}_L^\top \mathbf{a}_{j_L} \vec{r}_L \wedge v_R = \vec{l}_R^\top \mathbf{a}_{j_R} \vec{r}_R \mid b_j = \text{false}) \\ & = \Pr(v_L = \vec{l}_L^\top \mathbf{a}_{j_L} \vec{r}_L \wedge v_R = \vec{l}_R^\top \mathbf{a}_{j_R} \vec{r}_R \wedge \pi_j \text{ is accepted} \mid b_j = \text{false}) \\ & \leq \frac{4 \log_2 n}{p - 1}. \end{aligned}$$

In the last step, the inequality follows from Claim 2.

Consequently, the total soundness error is calculated as follows:

$$\begin{aligned} & \Pr(\mathcal{V} \text{ accepts} \mid \exists j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}) \\ & = \Pr(\mathcal{V} \text{ accepts} \wedge \forall i \in [k], b_i = \text{true} \mid \exists j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}) \\ & \quad + \Pr(\mathcal{V} \text{ accepts} \wedge \exists i \in [k], b_i = \text{false} \mid \exists j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}) \\ & \leq \Pr\left(\mathcal{V} \text{ accepts} \wedge \forall i \in [k], b_i = \text{true} \mid \exists j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}\right) \\ & \quad \wedge (\bigvee_{j=M'-J+1}^{M'} b_j = \text{true}) \\ & \quad + \Pr\left(\mathcal{V} \text{ accepts} \wedge \forall i \in [k], b_i = \text{true} \mid \exists j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}\right) \\ & \quad \wedge \neg(\bigvee_{j=M'-J+1}^{M'} b_j = \text{true}) \\ & \quad + \Pr(\mathcal{V} \text{ accepts} \mid \exists i \in [k], b_i = \text{false}) / \left(\frac{p-2}{p-1}\right) \\ & \leq \Pr\left(\mathcal{V} \text{ accepts} \wedge \forall i \in [k], b_i = \text{true} \mid \exists j \in [J], \right. \\ & \quad \left. \wedge (\bigvee_{j=M'-J+1}^{M'} b_j = \text{true}) \wedge \neg(\bigvee_{i=1}^{M'-J} \text{Event}_i) \mid f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}\right) \\ & \quad + \Pr\left(\mathcal{V} \text{ accepts} \wedge \forall i \in [k], b_i = \text{true} \mid \exists j \in [J], \right. \\ & \quad \left. \wedge (\bigvee_{j=M'-J+1}^{M'} b_j = \text{true}) \wedge (\bigvee_{i=1}^{M'-J} \text{Event}_i) \mid f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}\right) \\ & \quad + \Pr\left(\mathcal{V} \text{ accepts} \wedge \forall i \in [k], b_i = \text{true} \mid \exists j \in [J], \right. \\ & \quad \left. \wedge \neg(\bigvee_{j=M'-J+1}^{M'} b_j = \text{true}) \mid f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}\right) \end{aligned}$$

$$\begin{aligned}
& + \Pr(\mathcal{V} \text{ accepts} \mid \exists i \in [k], b_i = \text{false}) / \left(\frac{p-2}{p-1}\right) \\
\leq & \Pr\left(\mathcal{V} \text{ accepts} \wedge \exists j \in [k+1, M'], b_j = \text{false} \wedge b_{j_L} = b_{j_R} = \text{true} \wedge \neg(\bigvee_{i=1}^{M'-J} \text{Event}_i)\right) / \left(\frac{p-2}{p-1}\right) \\
& + \Pr\left(\mathcal{V} \text{ accepts} \wedge \bigvee_{i=1}^{M'-1} \text{Event}_i\right) / \left(\frac{p-2}{p-1}\right) \\
& + \Pr(\mathcal{V} \text{ accepts} \mid \exists i \in [k], b_i = \text{false}) / \left(\frac{p-2}{p-1}\right) \\
& + \Pr\left(\neg(\bigvee_{j=M'-J}^{M'} b_j = \text{true}) \mid \exists j \in [J], f_j(\mathbf{a}_1, \dots, \mathbf{a}_k) \neq \mathbf{0}\right) \\
\leq & \frac{p-1}{p-2} \cdot \sum_{j=k+1}^{M'} \Pr\left(\mathcal{V} \text{ accepts} \wedge b_{j_L} = b_{j_R} = \text{true} \wedge \neg(\bigvee_{i=1}^{M'-J} \text{Event}_i) \mid b_j = \text{false}\right) \\
& + \frac{p-1}{p-2} \cdot \sum_{i=1}^{M'-J} \Pr(\mathcal{V} \text{ accepts} \wedge \text{Event}_i) + \frac{k + 4 \log_2 n + 2 \log_2 k}{p-2} + \frac{J}{p-1} \\
\leq & \frac{4(M'-k) \log_2 n}{p-2} + \frac{\sum_{i=1}^{M'-J} (k_j + 4 \log_2 n)}{p-2} + \frac{k + 4 \log_2 n + 2 \log_2 k}{p-2} + \frac{J}{p-2} \\
\leq & \frac{4(M'-k) \log_2 n + 4(M'-J+1) \log_2 n + 2(M'-k) + k + 2 \log_2 k + J}{p-2} \\
\leq & \frac{10M' \log_2 n}{p-2}, \quad \text{for } n \geq 2.
\end{aligned}$$

The penultimate inequality uses the fact that $\sum_{j=1}^{M'-J} k_j \leq 2(M'-k)$, which reflects that the total number of parents across all nodes equals the total number of edges in the DAG.

Therefore, the soundness error is capped at $\frac{10M' \log_2 n}{p-2} = \frac{10(M+k) \log_2 n}{p-2}$. $\square$

F.6 Proof of Theorem 3

Proof. *Perfect completeness* is inherited from that of Π_{proof} and Π_{open} . *Knowledge soundness* is shown by calculating the knowledge soundness error as follows.

Consider an adversary $\mathcal{P}^*$ that produce an accepting transcript for Π with a probability $\epsilon(u) > \kappa^{\text{ks}}(u)$. We first demonstrate that a PPT extractor, Ext , can interact with $\mathcal{P}^*$ to extract a valid witness (w, w') such that $(u, C_w, w, w') \in \mathcal{R}'$. The relaxed relation $\mathcal{R}'$ is defined as

$$\mathcal{R}' = \left\{ u, C_w : (w, w') \mid \begin{array}{l} [\text{Constraint}(u, w) = \text{true} \wedge \text{Comm}(w) = C_w] \\ \vee [\text{Comm}(w) = \text{Comm}(w') \wedge w \neq w'] \end{array} \right\}$$

Let Ext_{open} be an extractor of Π_{open} . Ext_{open} can extract a witness w such that $C_w = \text{Comm}(w)$ with a probability of $\epsilon(u) - \kappa_{\text{open}}^{\text{ks}}(u)$.

Then, we construct an adversary $\mathcal{P}^{**}$ and an extractor Ext using $\mathcal{P}^*$ and Ext_{open} as follows: $\mathcal{P}^{**}$ first invokes Ext_{open} . If Ext_{open} successfully extracts w and $\text{Constraint}(u, w) = \text{true}$, Ext outputs w . If Ext_{open} successfully extracts w but $\text{Constraint}(u, w) = \text{false}$, then $\mathcal{P}^{**}$ invokes $\mathcal{P}^*$ to produce a proof π' for Π . The success probability of $\mathcal{P}^*$ is independent of Ext_{open} . $\mathcal{P}^{**}$ produces (w, π') with the following probability:

$$\Pr \left(\begin{array}{l} w \leftarrow \text{Ext}_{\text{open}} \\ \wedge \text{Constraint}(u, w) = \text{false} \\ \wedge \pi' \leftarrow \mathcal{P}^*, \pi' \text{ is accepting} \end{array} \right) = \Pr \left(\begin{array}{l} w \leftarrow \text{Ext}_{\text{open}} \\ \wedge \text{Constraint}(u, w) = \text{false} \end{array} \right) \cdot \epsilon(u).$$

By rewinding $\mathcal{P}^{**}$ to the point before the Π_{open} in $\mathcal{P}^*$ is invoked, Ext_{open} of Π_{open} can extract a witness w' such that $C_w = \text{Comm}(w')$ and $F_v(w')$ equals v in π' , with the following probability:

$$\begin{aligned} & \Pr(\text{Constraint}(u, w) = \text{false} \wedge w \leftarrow \text{Ext}_{\text{open}}) \cdot \epsilon(u) - \kappa_{\text{open}}^{\text{ks}}(u) \\ & \leq \Pr(\text{Constraint}(u, w) = \text{false} \wedge (w, w') \leftarrow \text{Ext}) \\ & = \Pr \left(\begin{array}{l} \text{Constraint}(u, w) = \text{false} \\ \wedge (w, w') \leftarrow \text{Ext} \wedge w' = w \end{array} \right) + \Pr \left(\begin{array}{l} \text{Constraint}(u, w) = \text{false} \\ \wedge (w, w') \leftarrow \text{Ext} \wedge w' \neq w \end{array} \right). \end{aligned}$$

If $w' = w$, $F_v(w') = F_v(w)$. Consequently, π'_{proof} serves as an accepting proof for $\Pi_{\text{proof}}(u, w)$. Given the computational soundness error of Π_{proof} is $\kappa_{\text{proof}}^{\text{cs}}(u)$, the first term on the right hand side of the above equation is at most $\kappa_{\text{proof}}^{\text{cs}}(u)$.

Therefore,

$$\begin{aligned} & \Pr(\text{Constraint}(u, w) = \text{false} | w \leftarrow \text{Ext}_{\text{open}}) \cdot \epsilon(u) - \kappa_{\text{open}}^{\text{ks}}(u) \\ & \leq \kappa_{\text{proof}}^{\text{cs}}(u) + \Pr(\text{Constraint}(u, w) = \text{false} \wedge w \neq w' \wedge (w, w') \leftarrow \text{Ext}). \end{aligned}$$

If $w' \neq w$, Ext outputs (w, w') such that $\text{Comm}(w') = \text{Comm}(w) = C_w$. This is a valid witness of $\mathcal{R}'$. The probability that Ext extracts $(u, C_w, w, w') \in \mathcal{R}'$ is:

$$\begin{aligned} & \Pr((w, w') \in \mathcal{R}' \wedge (w, w') \leftarrow \text{Ext}) \\ & = \Pr(\text{Constraint}(u, w) = \text{true} \wedge w \leftarrow \text{Ext}_{\text{open}}) \\ & \quad + \Pr(\text{Constraint}(u, w) = \text{false} \wedge w \neq w' \wedge (w, w') \leftarrow \text{Ext}) \\ & \geq \Pr(\text{Constraint}(u, w) = \text{true} | w \leftarrow \text{Ext}_{\text{open}}) \cdot \Pr(w \leftarrow \text{Ext}_{\text{open}}) \\ & \quad + \Pr(\text{Constraint}(u, w) = \text{false} \wedge w \leftarrow \text{Ext}_{\text{open}}) \cdot \epsilon(u) - \kappa_{\text{open}}^{\text{ks}}(u) - \kappa_{\text{proof}}^{\text{cs}}(u) \\ & = \epsilon(u) - \kappa_{\text{open}}^{\text{ks}}(u) - \Pr(\text{Constraint}(u, w) = \text{false} \wedge w \leftarrow \text{Ext}_{\text{open}}) \\ & \quad + \Pr(\text{Constraint}(u, w) = \text{false} \wedge w \leftarrow \text{Ext}_{\text{open}}) \cdot \epsilon(u) - \kappa_{\text{open}}^{\text{ks}}(u) - \kappa_{\text{proof}}^{\text{cs}}(u) \\ & \geq \epsilon(u) - [\kappa_{\text{proof}}^{\text{cs}}(u) + 2\kappa_{\text{open}}^{\text{ks}}(u)]. \end{aligned}$$

The total knowledge soundness error is bounded by $\kappa_{\text{proof}}^{\text{cs}}(u) + 2\kappa_{\text{open}}^{\text{ks}}(u)$.

As the knowledge soundness error is negligible, Π is knowledge sound for $\mathcal{R}'$.

Due to the binding property of the commitment scheme, the probability that Ext extracts distinct $w \neq w'$ where $\text{Comm}(w) = \text{Comm}(w')$ is negligible (denoted by κ_{com}). So, Π is also knowledge sound for $\mathcal{R}$. The knowledge soundness error is: $\kappa_{\text{proof}}^{\text{cs}}(u) + 2\kappa_{\text{open}}^{\text{ks}}(u) + \kappa_{\text{com}}$. $\square$

F.7 Proof of Corollary 1

Proof. According to Lemmas 9 and 10 in Appendix B.8, the computational soundness error of the non-interactive version of Π_{proof} is $Q \cdot \kappa_{\text{proof}}^{\text{cs}}(u)$, and the knowledge soundness error of the non-interactive version of Π_{open} is $(Q + 1) \cdot \kappa_{\text{open}}^{\text{ks}}(u)$. Then, by applying Theorem 3, we derive Corollary 1.

F.8 Proof of Theorem 4

Proof. *Perfect completeness* is obtained from the perfect completeness of subprotocols. *Knowledge soundness* is demonstrated by showing a negligible knowledge soundness error as follows. If we substitute $\text{Prove}_{\text{open}}$ at Line 6 in Algorithm 3 with a direct proof of $C_v = \text{Comm}(\vec{l}^\top (\mathbf{a}_1 \| \cdots \| \mathbf{a}_k \| \mathbf{c}_1 \| \cdots \| \mathbf{c}_{k'}) \vec{r})$, this modified protocol Π_{proof} serves as a proof for $\mathcal{L}_{\text{main}}$ in Eq. (2). In what follows, we denote $(\mathbf{a}_1 \| \cdots \| \mathbf{a}_k \| \mathbf{c}_1 \| \cdots \| \mathbf{c}_{k'})$ by w and the projection by $v := F_v(w)$.

Let $\mathcal{P}^*$ be an adversary against Π_{proof} that successfully generates an accepting transcript for $(u, w) \notin \mathcal{L}_{\text{main}}$ with a probability of ϵ . Then $\mathcal{P}^*$ is equally capable of producing an accepting transcript π^* for the original protocol Π with the same probability. In this transcript π^* , C_v at Line 2 and C_v at Line 4 satisfy $C_w = \text{Comm}(w)$ and $C_v = \text{Comm}(F_v(w))$.

The PoP protocol Π_{pop} includes a round-by-round sound proof $\Pi_{\text{pop-reduce}}$ and a proof of the opening algorithm $\Pi_{\text{pop-open}}$; we can then apply Corollary 1 to the non-interactive version of Π_{pop} . By fixing C_π and C_v in the transcript π^* at Line 4, Ext_{pop} can extract from $\mathcal{P}^*$ the witness $(\vec{\pi}_{\text{reduce}}, v^*) \in \mathcal{R}_{\text{pop}}$ with a probability of at least:

$$\epsilon - (Q + 1) \cdot (\kappa_{\text{pop-reduce}}^{\text{cs}} - 2\kappa_{\text{pop-open}}^{\text{ks}}) - \kappa_{\pi\text{-com}},$$

where $\kappa_{\pi\text{-com}}$ represents the probability that the binding property is broken, *i.e.*, if $\text{Comm}(v^*) \neq \text{Comm}(F_v(w)) = C_v$.

Thus, the probability that Ext_{pop} extracts $(\vec{\pi}_{\text{reduce}}, v^*) \in \mathcal{R}_{\text{pop}}$ such that $v^* = F_v(w)$ is no less than:

$$\epsilon - (Q + 1) \cdot (\kappa_{\text{pop-reduce}}^{\text{cs}} - 2\kappa_{\text{pop-open}}^{\text{ks}}) - \kappa_{\pi\text{-com}} - \kappa_{v\text{-com}}.$$

On the other hand, given the computational soundness of Π_{reduce} , the aforementioned probability is negligible:

$$\begin{aligned} Q \cdot \kappa_{\text{reduce}}^{\text{cs}} &\geq \Pr(\text{Extract valid } (\vec{\pi}_{\text{reduce}}, v) \wedge v = F_v(w) \mid (u, w) \notin \mathcal{L}_{\text{main}}) \\ &\geq \epsilon - (Q + 1) \cdot (\kappa_{\text{pop-reduce}}^{\text{cs}} - 2\kappa_{\text{pop-open}}^{\text{ks}}) - \kappa_{\pi\text{-com}} - \kappa_{v\text{-com}}. \end{aligned}$$

Hence, $\kappa_{\text{proof}} := \Pr(\mathcal{P}^* \text{ succeeds} \mid (u, w) \notin \mathcal{L}_{\text{main}}) := \epsilon$

$$\leq (Q + 1) \cdot (\kappa_{\text{reduce}}^{\text{cs}} + \kappa_{\text{pop-reduce}}^{\text{cs}} + 2\kappa_{\text{pop-open}}^{\text{ks}}) + \kappa_{\pi\text{-com}} + \kappa_{v\text{-com}}.$$

This κ_{proof} corresponds to the computational soundness error of Π_{proof} .

Then, by applying Corollary 1 to Π_{proof} , we can construct an Ext that extracts $(u, w) \in \mathcal{R}_{\text{main}}$ with a probability of at least:

$$\epsilon - (Q + 1) \cdot (\kappa_{\text{reduce}}^{\text{cs}} + \kappa_{\text{pop-reduce}}^{\text{cs}} + 2\kappa_{\text{pop-open}}^{\text{ks}} + 2\kappa_{\text{open}}^{\text{ks}}) - (\kappa_{\pi\text{-com}} + \kappa_{v\text{-com}} + \kappa_{w\text{-com}}),$$

where $\kappa_{w\text{-com}}$ is the probability that Ext breaks the binding commitment of C_w .

Therefore, the knowledge soundness error for $\mathcal{R}_{\text{main}}$ is:

$$\kappa^{\text{ks}} \leq (Q+1) \cdot (\kappa_{\text{reduce}}^{\text{cs}} + 2\kappa_{\text{open}}^{\text{ks}} + \kappa_{\text{pop-reduce}}^{\text{cs}} + 2\kappa_{\text{pop-open}}^{\text{ks}}) + (\kappa_{\pi\text{-com}} + \kappa_{v\text{-com}} + \kappa_{w\text{-com}}).$$

Let κ_{com} be the probability that the adversary breaks the binding property of any one of the commitments. Then, $\kappa_{\pi\text{-com}} \leq \kappa_{\text{com}}$, $\kappa_{v\text{-com}} \leq \kappa_{\text{com}}$, and $\kappa_{w\text{-com}} \leq \kappa_{\text{com}}$. Consequently:

$$\kappa^{\text{ks}} \leq (Q+1) \cdot (\kappa_{\text{reduce}}^{\text{cs}} + 2\kappa_{\text{open}}^{\text{ks}} + \kappa_{\text{pop-reduce}}^{\text{cs}} + 2\kappa_{\text{pop-open}}^{\text{ks}}) + 3\kappa_{\text{com}}.$$

Zero knowledge is ensured because all prover messages prior to the opening are hiding commitments with independent uniform distributions. The simulator samples these messages at random and uses the matrix-commitment proof simulator to generate an accepting transcript for the overarching protocol. $\square$

F.9 Proof of Claim 4

Proof. Perfect completeness and knowledge soundness follow from Theorems 2 and 3. For zero knowledge, we instantiate with a matrix commitment scheme with zero-knowledge proofs [78] and apply the zero-knowledge transformation [90], which replaces each transcript element with a hiding commitment. With a trapdoor for these commitments, the simulator generates accepting transcripts indistinguishable from faithful ones. $\square$

G Preliminary Experimental Results

We conducted a proof-of-concept experiment demonstrating that our framework achieves practical prover time, succinct proof size, and efficient verification for large NNs. Operationally, a mock MLP neural network with 2^{10} layers was used, each defined by $\mathbf{X}_i = \text{sigmoid}(\mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i)$ for $i \in [2^{10}]$, where each weight matrix $\mathbf{W}_i$ has dimension $2^{10} \times 2^{10}$ and both $\mathbf{B}_i$ and $\mathbf{X}_i$ are 2^{10} -dimensional vectors, with $\mathbf{X}_0$ and $\mathbf{X}_D$ as the NN input and output. This mock NN contains ~ 1 billion parameters, providing a representative benchmark for evaluating Evalyn performance, as the prover time of Evalyn scales linearly with the number of parameters, which is typically measured in billions.

Experimental Setup. Our framework was implemented in Rust and executed on CPUs. Groth16 [48] was used to commit to and prove the PoP circuit, a variant of Dory [59] (Appendix E.2) serves as the matrix commitment and the 2-to-1 Poseidon hash from the Arkworks library instantiates the Fiat-Shamir transformation. Experiments were conducted on a cloud server equipped with an Intel(R) Xeon(R) 6982P-C processor, providing 64 vCPUs (32 cores with 2 threads per core) clocked from 0.8 to 3.9 GHz and 256 GB RAM. Our implementation is available at https://github.com/mirandaprivate/evalyn_asiacrypt.

Performance. Two quantization settings were evaluated. Under 8-bit quantization, the prover time was 34.8 min. Under 16-bit quantization, the prover time

Table 3. Performance on a 2^{10} -layer NN (~ 1 billion parameters)

Layer i : $\mathbf{X}_i \leftarrow \text{sigmoid}(\mathbf{W}_i \mathbf{X}_{i-1} + \mathbf{B}_i)$	8-Bit	16-Bit
Unverified NN inference (no proof)	3.53 s	4.64 s
Setup time	2.44 s	2.43 s
Structure reference string size	22.02 MB	22.02 MB
NN commitment time	2,023.87 s	2,097.80 s
Structure commitment (single thread)	747.07 s	737.30 s
Parameter commitment	1,276.80 s	1,360.50 s
NN commitment size	42.27 KB	42.27 KB
Prover time	2,090.71 s	2,210.88 s
Auxiliary input commitment	733.08 s	755.61 s
Proof reduction ($\mathbb{F}$ operations)	218.80 s	293.16 s
PoP proving (single thread)	775.36 s	765.05 s
Leaf node opening	25.46 s	31.23 s
Fiat-Shamir transform proving	338.01 s	356.83 s
Proof size	98.84 KB	98.84 KB
Verifier time (single thread)	173.19 ms	172.72 ms
Peak RAM	87.42 GB	103.63 GB

Parallelized across 64 CPU threads unless noted “single thread”

was 36.8 min. Detailed results are given in Table 3. The additional overhead for hiding the NN architecture, reflected by the PoP prover time, accounts for $\sim 50\%$ of the total prover time. This ratio can be reduced with a parallel Groth16 implementation for PoP proving.

Pending Support for Realistic NNs. Proofs for architectures such as ResNet and GPT-2 were not implemented because encoding these models requires substantial engineering and the prototype lacks GPU support. Prover time in *Evalyn* scales primarily with parameter count, with only minor variation across architectures. Therefore, times for these NNs can be estimated by scaling the mock-model results by parameter count, normalized to one billion parameters.

Compatible but Unsuitable for Backpropagation. Our framework targets NN inference rather than training. This is a design choice that trades prover time for succinctness. A central goal of *Evalyn* is to maintain succinct zkSNARK proofs so that light-client verifiers (*e.g.*, lightweight blockchain nodes) can check inference efficiently. While verifiable NN inference is in demand in Web3 scenarios, *e.g.*, on-chain compliance tasks such as know your customer and anti-money laundering, succinctness may be less important for privacy-preserving NN training. When parties remain online and interactive, communication-heavy protocols (*e.g.*, Mystique [81]) could achieve faster proving despite lacking succinctness.

Memory Footprint. RAM usage scales with the number of model parameters. The peak occurs when reducing the projections of all leaf nodes into a single large vector projection (Line 25, Algorithm 2), which requires running the *LiteBullet* protocol on large vectors. As a reference, storing 1 billion scalars as 256-bit

field elements requires 32 GB of RAM. Since LiteBullet operates on two such vectors, the memory usage can reach 64 GB, with additional overhead from commitments and intermediate values leading to the reported 87/104 GB peak. Sharding techniques could reduce this memory peak.

Comparison with Unverified Computation. With 8-bit (16-bit) quantization, the prover is $592\times$ ($476\times$) slower than unverified NN inference. In contrast, the verifier is $20\times$ ($27\times$) faster than unverified inference even in a single thread, with larger gains under parallelization. For larger NNs, verifier time grows only logarithmically with model size, further amplifying this advantage. This efficiency asymmetry makes our approach well suited to blockchain settings, where proving is done once by a single node, while all nodes verify.

Impact of Higher-Bit Quantization. Moving from 8-bit to 16-bit quantization increases prover time due to two factors:

- *Increased commitment cost:* Commitment time in the group-based PCS increases with bitwidth.
- *Larger lookup table:* The lookup table size grows from 2^{13} to 2^{18} , increasing both commitment and prover time.

Extending to 32-bit quantization would require commitments to auxiliary vectors of size 2^{32} for lookup proofs, which would sharply increase commitment time and push prover time beyond 10,000 seconds. Evalyn does not support 64-bit quantization. This limitation is acceptable, as many practical deployments target 8–16-bit inference.